

A COOKBOOK FOR NODE.JS DEVELOPERS

Python Backend Cookbook

Django & FastAPI for Node.js developers — learn two Python ecosystems by mapping every concept back to the Node you already know.

Python 3.13+

Django 5

DRF

FastAPI

SQLAlchemy

Pydantic v2

asyncio

Celery

pytest

uv

mypy

LLMs

Written for engineers fluent in JavaScript / TypeScript, Node.js, Express, Prisma and REST. Django is the closest full-stack analog to Rails/Laravel; FastAPI is the closest API-first analog to Express/Nest.

Edition 2026 · Node.js → Python → Django → FastAPI

Table of Contents

How to Use This Book	7
Two frameworks, two Node analogs	7
The one idea that will surprise you most: the GIL and async	8
The Learning Roadmap	8
Quick Start • From Node.js to Python in One Sitting	10
Q.1 Set up in sixty seconds	10
Q.2 Your first API in ten lines.	10
Q.3 The one-hour survival syntax	11
Q.4 Ten things that will trip you up on day one	12
Q.5 Port a tiny Express app to FastAPI.	13
Q.6 Three mental models to carry into the deep chapters	14
Q.7 Your first-hour checklist	15
Part 1 • The Python Language	16
1.1 Variables, types, and the "everything is an object" model	16
1.2 Strings and f-strings	17
1.3 The four core collections.	17
1.4 None, booleans, truthiness	18
1.5 Conditionals, loops, and pattern matching	19
1.6 Functions, defaults, args, *kwargs, keyword args	19
1.7 Lambdas, closures, and higher-order functions	20
1.8 Classes, dataclasses, properties.	20
1.9 Inheritance, protocols, abstract classes	21
1.10 Modules, packages, imports, exceptions.	22
1.11 Context managers, generators, decorators	22
Part 2 • Functional Programming in Python	24
2.1 The big idea: comprehensions over method chains	24
2.2 map, filter, reduce — and why you rarely use them.	25
2.3 functools: partial, reduce, cache, composition	25
2.4 itertools: lazy pipelines and the tools you'll actually use	26
2.5 The Pythonic grouping pattern (no chain needed).	27
2.6 operator: named operators for callbacks	27
2.7 Immutable-style programming	27
Part 3 • The Python Runtime, the GIL, and asyncio	29
3.1 Node's model, restated	29
3.2 The GIL: the one sentence that explains Python concurrency	29
3.3 asyncio: Python's event loop is Node's event loop	30
3.4 Concurrency with coroutines: gather and TaskGroup	31
3.5 The blocking trap: never block the event loop	31
3.6 The three concurrency models — and the deployment split.	32
3.7 EventEmitter and inter-task communication	33
Part 4 • Python Project & Dependency Management	34
4.1 Virtual environments — the concept npm hides.	34

4.2 The tool landscape, mapped to npm	35
4.3 Global CLI tools: pipx & uvx (the real npm -g)	35
4.4 pyproject.toml — the modern manifest	37
4.5 Version constraints and locking	37
4.6 Package structure and imports	38
4.7 Building & publishing a package (vs npm publish)	38
Part 5 • Django for Node Developers	41
5.1 Projects, apps, and settings	41
5.2 The request lifecycle and middleware	42
5.3 Views: function-based and class-based	42
5.4 Models, migrations, and the admin	43
5.5 Management commands and signals	43
Part 6 • Routing (Django, DRF, and FastAPI)	45
6.1 Django URL configuration	45
6.2 DRF routers — CRUD routing like Laravel resources	46
6.3 FastAPI routing — decorators you already know	46
6.4 Route groups, prefixes, and versioning	47
Part 7 • FastAPI	49
7.1 A complete endpoint, annotated	49
7.2 Pydantic — your Zod/Joi, but it powers the whole framework	49
7.3 Dependency injection — Nest-style, function-based	50
7.4 Middleware, exceptions, and background tasks	51
7.5 Async correctness in FastAPI	52
Part 8 • Databases: Django ORM & SQLAlchemy	53
8.1 The four ORMs at a glance	53
8.2 Defining models	53
8.3 Migrations	54
8.4 CRUD side by side	54
8.5 Relationships and the N+1 problem	55
8.6 Transactions, locking, and pagination	56
8.7 Query optimization and indexes	56
Part 9 • Production REST APIs (DRF & FastAPI)	58
9.1 Serializers vs Pydantic — the validation/serialization layer	58
9.2 CRUD: a DRF ViewSet and FastAPI endpoints	58
9.3 Status codes and consistent error envelopes	59
9.4 Pagination, filtering, sorting, search	60
9.5 Idempotency and webhooks	60
Part 10 • Authentication & Authorization	62
10.1 Password hashing	62
10.2 Django authentication: sessions and DRF tokens	62
10.3 FastAPI authentication: dependencies + JWT	63
10.4 Authorization: permissions, roles, policies	64
10.5 Sessions, email verification, password reset	64
Part 11 • Security	66

11.1 SQL injection	66
11.2 CSRF and CORS	66
11.3 XSS and template escaping	67
11.4 Rate limiting	67
11.5 Secrets, HTTPS, and secure cookies	67
11.6 File upload security.	67
11.7 SSRF — the Python-relevant server-side threat	68
11.8 Webhook signature verification	68
11.9 The checklist	68
Part 12 · Application Architecture	70
12.1 Django's default: fat models, thin views	70
12.2 Services and use-cases when logic grows	70
12.3 DTOs, Pydantic schemas, and value objects	71
12.4 Protocols and dependency inversion	71
12.5 Domain events: Django signals vs an explicit bus	72
12.6 Repositories — usually optional	72
Part 13 · Background Jobs (Celery & friends)	74
13.1 The landscape	74
13.2 Defining and dispatching a task	74
13.3 Running workers	75
13.4 Idempotency, retries, and failure handling.	75
13.5 Chains, groups, and chords	76
13.6 async apps: arq / Dramatiq.	76
Part 14 · Async Deep Dive	78
14.1 Coroutines, Tasks, Futures — the precise vocabulary	78
14.2 gather vs TaskGroup (structured concurrency).	78
14.3 Timeouts and cancellation	78
14.4 Async generators and context managers	79
14.5 Mixing blocking code (the critical skill).	79
14.6 Async database and HTTP	80
Part 15 · Caching & Redis	81
15.1 The cache APIs	81
15.2 Cache-aside — the pattern you'll use most	81
15.3 Invalidation.	82
15.4 Locks and distributed locks	82
15.5 Cache stampede (thundering herd)	82
Part 16 · HTTP Clients (requests & httpx)	84
16.1 The three clients, mapped to Axios	84
16.2 Sync (requests / httpx)	84
16.3 Async (httpx) — the FastAPI default	85
16.4 Retries, backoff, and timeouts	85
16.5 Testing without the network	86
Part 17 · Files & Object Storage (S3)	87
17.1 The storage options	87
17.2 Django with django-storages (S3 as the default backend)	87

17.3 FastAPI uploads with boto3.	88
17.4 Private files and presigned (signed) URLs	88
17.5 Streaming and large files.	89
Part 18 • Testing	90
18.1 The tooling map	90
18.2 pytest fixtures — dependency injection for tests.	90
18.3 DRF and Django feature tests	91
18.4 FastAPI tests with TestClient and dependency overrides	91
18.5 Factories, mocking, time, and async	92
18.6 What to test, and how much	92
Part 19 • CLIs & Operational Scripts	94
19.1 Django manage.py	94
19.2 The shell (REPL) — like tinker	95
19.3 Standalone CLIs: Click and Typer.	95
19.4 Running external processes (child_process)	96
Part 20 • Production Deployment	97
20.1 WSGI vs ASGI — the server layer.	97
20.2 The production topology	97
20.3 The deploy sequence.	98
20.4 Configuration and health checks	98
20.5 Docker	99
Part 21 • Performance	100
21.1 The measurable wins, in order	100
21.2 Finding N+1 and slow queries	100
21.3 CPU-bound work and the GIL.	101
21.4 Serialization and connection pooling.	101
21.5 Memory and worker sizing	101
Part 22 • Observability	103
22.1 Structured logging	103
22.2 Request context and correlation IDs	103
22.3 Metrics	104
22.4 Distributed tracing	104
22.5 Error tracking and health checks.	104
22.6 The observability checklist	105
Part 23 • Capstone Projects	106
23.1 Project A — Task Management API	106
23.2 Project B — E-commerce API	107
23.3 Project C — Webhook / Integration Platform	108
Part 24 • Type Checking & Static Typing (mypy & pyright)	111
24.1 The mental model, mapped to TypeScript	111
24.2 The syntax, side by side	111
24.3 Optional and None — Python's one null	112
24.4 Structural typing: Protocol = a TypeScript interface	113
24.5 Typed dicts, dataclasses, and named shapes	114

24.6 Generics	114
24.7 The escape hatches — and when they bite	115
24.8 mypy vs pyright — which checker?	115
Part 25 • Configuration & Settings Management	117
25.1 The problem with raw os.environ.	117
25.2 pydantic-settings — typed config that validates itself	117
25.3 Nested config and prefixes	118
25.4 Per-environment settings (dev / staging / prod)	119
25.5 Secrets	119
Part 26 • Logging & Error Tracking	121
26.1 The standard logging module	121
26.2 Configure once, at startup	122
26.3 Structured logging with structlog	122
26.4 Request context (correlation IDs)	123
26.5 Error tracking with Sentry	123
Part 27 • Python + LLMs	125
27.1 The SDKs, mapped to Node	125
27.2 A basic call — sync and async	125
27.3 Structured output — Pydantic is your Zod	127
27.4 Streaming	128
27.5 Tool calling and a minimal agent loop	129
27.6 Local models — Ollama & Hugging Face	130
27.7 Production patterns: retries, rate limits, cost, and caching	131
27.8 Testing LLM code without burning tokens	132
Part 28 • Master Cheat Sheet & Closing Roadmap	133
24.1 Tooling & ecosystem	133
24.2 Language.	133
24.3 Concurrency & runtime	134
24.4 Web frameworks & HTTP	134
24.5 Data, jobs, infrastructure.	135
24.6 The runtime split, in one table	136
24.7 Closing Roadmap — from here to fluent	136

How to Use This Book

You are a productive Node.js engineer: JavaScript and TypeScript, React on the front, Express/Nest services on the back, Prisma for data, `async/await` in your bones. This book teaches you **two** Python backend ecosystems — **Django** and **FastAPI** — not by teaching programming, but by continuously translating Python into the Node you already know.

Every substantial concept follows the same rhythm:

1. **Concept** — the problem.
2. **Node.js equivalent** — how you already do it.
3. **Python implementation** — how it is done here.
4. **Side-by-side code** — the two, together.
5. **Important differences** — where the mental model breaks.
6. **Common mistakes** — what trips up Node developers.
7. **Production advice** — what matters when it is live.
8. **Exercise** — a small task to make it stick.

Two frameworks, two Node analogs

Python's web world has two dominant server frameworks, and knowing which Node tool each resembles saves you months of confusion:

- **Django** is a batteries-included, convention-driven, full-stack framework: ORM, admin, auth, migrations, templating, all in the box. Its closest Node/JS cousin is **Rails/Laravel** — or, in the Node world, a very opinionated **NestJS** plus a built-in database and admin. You adopt Django's way of doing things and it does a *lot* for you.
- **FastAPI** is a modern, async-first, API-focused microframework built on type hints and **Pydantic**. Its closest Node analog is **Express** (minimal, you compose it) crossed with **NestJS** (dependency injection, decorators, validation from types). It gives you automatic OpenAPI docs and request validation for free, but no ORM, admin, or auth — you choose those.

This book teaches both, and — crucially — the database layer in **two** flavors: Django's built-in ORM (used with Django/DRF) and **SQLAlchemy** (the standalone ORM you pair with FastAPI). Where a topic differs between the two, you get both.

NODE.JS MENTAL MODEL

If you want "Laravel/Rails in Python," that's Django + Django REST Framework (DRF). If you want "Express/Nest in Python with typed validation and async," that's FastAPI + Pydantic + SQLAlchemy. Many teams even run both: Django for the admin/back-office, FastAPI for high-throughput public APIs. You will be fluent in both by the end.

The one idea that will surprise you most: the GIL and async

Node has one event loop and is single-threaded by design; you never think about threads for I/O. Python *has* real threads, but the **Global Interpreter Lock (GIL)** means only one thread executes Python bytecode at a time — so threads help with I/O but not CPU-bound parallelism. Meanwhile Python also has `asyncio`, an event loop that looks almost exactly like Node's. The result: Python gives you *three* concurrency models (async event loop, threads, processes) where Node mostly gives you one. Part 3 makes this precise; it is the highest-leverage chapter in the book.

The Learning Roadmap

Work the levels in order. For each you get: what to read, what to practice, and what you should be able to build afterward.

Level 1 — Language fundamentals (Part 1)

Read: Part 1. *Practice:* port 20 small Node utilities to Python; write a small class hierarchy with type hints and `dataclasses`. *Build:* correct, typed Python modules.

Level 2 — Functional programming (Part 2)

Read: Part 2. *Practice:* rewrite a data-transform module using comprehensions, `itertools`, and `functools`. *Build:* idiomatic Python pipelines (not `filter().map()` chains).

Level 3 — Runtime & concurrency (Part 3)

Read: Part 3 (GIL, `asyncio`, threads, processes). *Practice:* benchmark an I/O-bound vs CPU-bound task across `async`, threads, and multiprocessing. *Build:* a correct model of which concurrency tool fits which workload.

Level 4 — Packaging & frameworks (Parts 4-7)

Read: project management, Django, routing, FastAPI. *Practice:* scaffold both a Django project and a FastAPI app; wire routes and views. *Build:* routed apps in both frameworks.

Level 5 — Database (Part 8)

Read: Django ORM and SQLAlchemy. *Practice:* model Users/Projects/Tasks in both; eliminate N+1s with `select_related` / `selectinload`. *Build:* a relational layer in each ORM.

Level 6 — REST APIs (Part 9)

Read: DRF and FastAPI. *Practice:* CRUD with serializers/Pydantic, pagination, filtering, versioning. *Build:* production-shaped JSON APIs in both.

Level 7 — Auth & security (Parts 10-11)

Read: auth and security. *Practice:* sessions, JWT, permissions, the security checklist. *Build:* APIs safe for the public internet.

Level 8 — Background jobs & async (Parts 13-14)

Read: Celery and the asyncio deep dive. *Practice:* move slow work to Celery; write correct async endpoints. *Build:* responsive services that offload work.

Level 9 — Testing (Part 18)

Read: pytest, DRF/FastAPI test clients. *Practice:* cover your APIs with fixtures and factories. *Build:* a fast, isolated suite.

Level 10 — Production (Part 20)

Read: WSGI/ASGI, Gunicorn/Uvicorn, Docker. *Practice:* containerize the full stack. *Build:* a deployable, observable service.

Level 11 — Advanced (Parts 12, 15-17, 19, 21-22)

Read: architecture, caching, HTTP clients, files/S3, CLIs, performance, observability. *Build:* a maintainable, well-instrumented codebase.

Level 12 — Capstone (Part 23)

Build: the three capstone projects in FastAPI + SQLAlchemy, with the Django/DRF equivalent explained. *Build:* production backends independently.

Level 13 — Advanced tooling & integration (Parts 24-27)

Read: static type checking with mypy/pyright vs TypeScript (Part 24), configuration & settings with pydantic-settings (Part 25), logging & error tracking with structlog and Sentry (Part 26), and integrating LLMs from a Python backend — structured output, tool calling, local models, and cost control (Part 27). *Build:* type-checked, well-configured, observable services with a production-grade LLM feature.

The **master cheat sheet** (Part 28) at the end maps Node → Python → Django → FastAPI across every subsystem — bookmark it.

Quick Start · From Node.js to Python in One Sitting

The rest of this book is a deep, chapter-by-chapter translation of Python and its frameworks. This chapter is different: it is the **fast on-ramp**. In one sitting it gets you from "I've never run Python" to a working, typed, auto-documented FastAPI endpoint — plus the survival syntax and the day-one gotchas that trip up every Node developer. Think of it as *hour zero*, before Level 1 of the roadmap. Skim it, type along, and you'll be productive before you finish your coffee.

Q.1 Set up in sixty seconds

You need a Python and a way to manage dependencies. Use **uv** — one fast tool that installs Python, creates the virtualenv, and manages packages (Part 4 goes deeper). It is the closest thing to a modern `npm`.

SHELL

```
# install uv (macOS/Linux); Windows: use the PowerShell one-liner from the docs
curl -LsSf https://astral.sh/uv/install.sh | sh

uv init hello-api && cd hello-api      # scaffold a project (like npm init)
uv add "fastapi[standard]"             # add a dependency (creates .venv + uv.lock)
```

That's it — no global installs, no manual `virtualenv` activation. `uv run` executes commands inside the project's isolated environment automatically.

NODE.JS MENTAL MODEL

`uv` $\approx$ `npm` / `pnpm`. The project's `.venv` folder $\approx$ `node_modules`. `pyproject.toml` $\approx$ `package.json`. `uv.lock` $\approx$ `package-lock.json`. `uv run x` $\approx$ `npx x`. If you internalize just this mapping, dependency management will feel familiar immediately.

Q.2 Your first API in ten lines

Create `main.py`:

PYTHON

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Task(BaseModel):      # a typed request/response schema (your Zod)
```

```

title: str
done: bool = False

@app.get("/health")
def health():
    return {"status": "ok"}    # dicts are auto-serialized to JSON

@app.post("/tasks", status_code=201)
def create_task(task: Task):  # body is validated against Task automatically
    return task

```

Run it:

SHELL

```
uv run fastapi dev main.py    # hot-reloading dev server on http://127.0.0.1:8000
```

Now open **http://127.0.0.1:8000/docs** — FastAPI generated interactive Swagger documentation from your type hints, with a form to try `POST /tasks`. Send `{"title": "ship it"}` and it validates, fills in `done: false`, and echoes JSON. Send `{"title": 123}` and you get a clean 422 with the exact error — validation you did not write.

PRODUCTION TIP

That `/docs` page is not a toy — it is your API's living contract and manual test surface, free on every project. Keep your Pydantic models accurate and the docs stay accurate. This single feature is often what converts Express developers to FastAPI.

And the Django equivalent, for when you want the batteries-included, admin-first framework (Part 5):

SHELL

```

uv add django
uv run django-admin startproject config .
uv run python manage.py migrate
uv run python manage.py runserver    # http://127.0.0.1:8000 (+ a free /admin UI)

```

Q.3 The one-hour survival syntax

Everything you need to *read* Python code in the first hour. Keep this table open; Part 1 explains the why.

You write in JS/TS	You write in Python
<code>const x = 1;</code> (no semicolons)	<code>x = 1</code>
blocks with <code>{ }</code>	blocks with <code>:</code> and indentation
<code>`Hi \${name}`</code>	<code>f"Hi {name}"</code>
<code>"a" + b</code>	<code>"a" + b</code> (strings) — but <code>f"..."</code> is nicer

You write in JS/TS	You write in Python
<code>{ name: "A" }</code> object	<code>{"name": "A"}</code> dict
<code>["a", "b"]</code> array	<code>["a", "b"]</code> list
<code>obj.prop</code>	<code>obj.prop</code> (same!)
<code>arr.map(f)</code>	<code>[f(x) for x in arr]</code> (comprehension)
<code>arr.filter(p)</code>	<code>[x for x in arr if p(x)]</code>
<code>x === y</code>	<code>x == y</code> (value) / <code>x is None</code> (null check)
<code>x ?? y</code>	<code>x if x is not None else y</code>
<code>a && b, a \\\ b, !a</code>	<code>a and b, a or b, not a</code>
<code>true / false / null</code>	<code>True / False / None</code>
<code>for (const x of xs)</code>	<code>for x in xs:</code>
<code>i++</code>	<code>i += 1</code> (there is no <code>++</code>)
<code>function f(a) {}</code>	<code>def f(a):</code>
<code>(x) => x * 2</code>	<code>lambda x: x * 2</code>
<code>async function / await</code>	<code>async def / await</code> (same idea, Part 3)
<code>try { } catch (e) { }</code>	<code>try: / except Exception as e:</code>
<code>import x from "y"</code>	<code>from y import x</code>
<code>console.log(x)</code>	<code>print(x)</code>
<code>// comment</code>	<code># comment</code>
<code>JSON.stringify(o) / .parse</code>	<code>json.dumps(o) / json.loads</code>

Q.4 Ten things that will trip you up on day one

These are the specific Node-brain reflexes that produce Python bugs. Read them once now and save yourself an afternoon.

#	The trap	The fix
1	Indentation is the syntax — a stray space is an error	pick 4 spaces, let your editor enforce it
2	<code>if items:</code> means "if non-empty " — <code>[]</code> and <code>{}</code> are falsy	opposite of JS; lean into it
3	<code>==</code> vs <code>is</code> — use <code>x is None</code> , not <code>x == None</code>	<code>is</code> for <code>None</code> /identity, <code>==</code> for value
4	Mutable default args: <code>def f(a=[])</code> reuses one list	use <code>def f(a=None): a = a or []</code>
5	No <code>++</code> / <code>--</code>	<code>i += 1</code>
6	<code>self</code> is an explicit first method parameter	<code>def method(self):</code> always
7	Truthy strings: <code>"0"</code> is truthy (unlike PHP), <code>""</code> falsy	explicit checks at boundaries
8	Integer division <code>7 / 2 == 3.5</code> ; use <code>7 // 2</code> for <code>3</code>	<code>/</code> is float div, <code>//</code> is floor div
9	Forgetting the virtualenv → installs go global	use <code>uv run</code> / activate <code>.venv</code>

#	The trap	The fix
10	A coroutine not <code>await</code> ed does nothing (no eager start)	always <code>await</code> or <code>create_task</code> (Part 3)

COMMON MISTAKE

The single most surprising one for Node developers is **#2**: empty collections are falsy. `if not results:` is the idiomatic "if empty" check, and `if results:` means "if it has items." Your JS instinct (arrays are always truthy) is exactly backwards here — internalize it early.

Q.5 Port a tiny Express app to FastAPI

Here is a complete in-memory CRUD in both, side by side, so the shape clicks.

JAVASCRIPT / NODE.JS

```
// Express
import express from "express";
const app = express();
app.use(express.json());
const tasks = [];

app.get("/tasks", (req, res) => res.json(tasks));

app.post("/tasks", (req, res) => {
  const task = { id: tasks.length + 1, title: req.body.title, done: false };
  tasks.push(task);
  res.status(201).json(task);
});

app.get("/tasks/:id", (req, res) => {
  const task = tasks.find((t) => t.id === Number(req.params.id));
  if (!task) return res.status(404).json({ error: "not found" });
  res.json(task);
});

app.listen(8000);
```

PYTHON

```
# FastAPI – same app, with validation and typed responses for free
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()
tasks: list[dict] = []

class TaskIn(BaseModel):
    title: str  # validated: missing/!str -> automatic 422

@app.get("/tasks")
```

```
def list_tasks():
    return tasks  # auto-JSON

@app.post("/tasks", status_code=201)
def create_task(body: TaskIn):  # body parsed + validated, no express.json()
    task = {"id": len(tasks) + 1, "title": body.title, "done": False}
    tasks.append(task)
    return task

@app.get("/tasks/{task_id}")
def get_task(task_id: int):  # path param cast to int (bad input -> 422)
    task = next((t for t in tasks if t["id"] == task_id), None)
    if task is None:
        raise HTTPException(404, "not found")  # -> {"detail": "not found"}
    return task
```

Notice what disappeared: no `express.json()` body-parser wiring, no manual `Number(req.params.id)` casting, no hand-built validation, and you gained Swagger docs. That is the FastAPI value proposition in one screen.

NODE.JS MENTAL MODEL

`req.body` → a typed **Pydantic** parameter. `req.params.id` → a typed function argument (`task_id: int`). `res.status(201).json(x)` → return `x` with `status_code=201`. `res.status(404)` → `raise HTTPException(404, ...)`. Once you see the mapping, porting an Express route is mechanical.

Q.6 Three mental models to carry into the deep chapters

1. **Packaging is npm with a virtualenv.** `uv` = npm; `.venv` = `node_modules`; `pyproject.toml` = `package.json`; `uv.lock` = lockfile. (Part 4)
2. **asyncio is Node's event loop.** `async def / await`, `asyncio.gather` = `Promise.all`. FastAPI is the async/ASGI world; Django/DRF is the sync/WSGI world. But Python also has real threads and processes — and the **GIL** means threads help only I/O, not CPU. (Parts 3, 14)
3. **Types are optional but powerful.** Type hints are TypeScript-style annotations checked by `mypy / pyright` (run like `tsc`), and Pydantic/FastAPI use them *at runtime* for validation and docs. Adopt them from line one. (Parts 1, 7)

WHEN NOT TO REACH PAST THIS CHAPTER YET

Do not try to absorb Django, SQLAlchemy, Celery, and asyncio internals in one day. Get the FastAPI hello-app running, port one small Express route (Q.5), and skim the survival table (Q.3). Come back for the deep parts when you hit a real need — the book is organized so each part stands alone.

Q.7 Your first-hour checklist

Do these seven things and you are officially unblocked:

1. Install `uv`; `uv init` a project; `uv add "fastapi[standard]"`.
2. Create the ten-line `main.py` from Q.2 and run `uv run fastapi dev main.py`.
3. Open `/docs` and successfully `POST` a task through the Swagger form.
4. Trigger a 422 by sending a wrong type — see validation you didn't write.
5. Port the Express CRUD from Q.5 and hit all three routes.
6. Add `uv add --dev mypy ruff` and run `uv run ruff check` + `uv run mypy .`
7. Skim Q.3 and Q.4 once more, then jump to **Part 1** for the language proper.

QUICK-START EXERCISE

Extend the Q.5 FastAPI app: add `PATCH /tasks/{id}` that toggles `done` (404 if missing), a `TaskOut` Pydantic response model with `id / title / done` so responses are typed, and a `GET /tasks?done=true` query filter using a comprehension. Confirm every route appears and works in `/docs`. You have now built, validated, and documented a real API in Python — welcome aboard.

Part 1 · The Python Language

Python and JavaScript are both dynamic, but Python is more strongly typed at runtime (fewer silent coercions), uses **indentation** instead of braces, and ships a much larger standard library. Modern Python (3.13) adds gradual typing via **type hints**, `dataclasses`, pattern matching, and first-class `async`. This part maps Python onto the JavaScript/TypeScript you know.

There is no `let/const` and no `$`; you assign with `=`, and blocks are defined by a colon and indentation, not `{ }`:

PYTHON

```
def greet(name: str) -> str:
    if name:
        return f"Hello, {name}!"
    return "Hello!"
```

1.1 Variables, types, and the "everything is an object" model

Python variables are names bound to objects. Types live on values (like PHP, unlike TypeScript's compile-time types). **Type hints** are optional annotations — they do *not* change runtime behavior, but tools like `mypy/pyright` check them, giving you a TypeScript-like safety net you run separately.

TYPESCRIPT

```
let count: number = 0;
const MAX = 100;           // immutable binding
const name = "Ahmed";
```

PYTHON

```
count: int = 0
MAX = 100                # convention: UPPER_CASE means "constant" (not
                           enforced)
name = "Ahmed"
```

NODE.JS MENTAL MODEL

Type hints are TypeScript types that *don't compile away and aren't enforced at runtime* — they're checked by a separate tool (`mypy`) in CI, like running `tsc --noEmit`. Python runs fine with wrong hints; the checker is your discipline. Treat `mypy/pyright` as mandatory in a serious codebase, the way you treat `tsc`.

1.2 Strings and f-strings

JAVASCRIPT / NODE.JS

```
const greeting = `Hello, ${name}! You have ${3 + 4} messages.`;
```

PYTHON

```
greeting = f"Hello, {name}! You have {3 + 4} messages." # f-string
multi = """triple-quoted
spans lines"""
```

Task	JavaScript	Python
Interpolate	<code>`\${x}`</code>	<code>f"{x}"</code>
Length	<code>s.length</code>	<code>len(s)</code>
Upper	<code>s.toUpperCase()</code>	<code>s.upper()</code>
Contains	<code>s.includes(x)</code>	<code>x in s</code>
Split	<code>s.split(',')</code>	<code>s.split(",")</code>
Join	<code>arr.join(',')</code>	<code>",".join(arr)</code>
Trim	<code>s.trim()</code>	<code>s.strip()</code>
Replace	<code>s.replaceAll(a,b)</code>	<code>s.replace(a, b)</code>
Slice	<code>s.slice(1,3)</code>	<code>s[1:3]</code> (slicing)

Python methods live *on* the object (`s.upper()`) — closer to JS than PHP's free functions. `len()` is the notable exception (a builtin, not a method).

1.3 The four core collections

Where JS has arrays and objects (plus `Map/Set`), Python has **four** distinct built-in collections, and choosing the right one is idiomatic:

PYTHON

```
nums    = [1, 2, 3]           # list -> JS array (mutable, ordered)
point   = (10, 20)           # tuple -> immutable fixed-size sequence
unique  = {1, 2, 3}           # set -> JS Set
user    = {"name": "Ahmed", "age": 31} # dict -> JS object / Map
```

JAVASCRIPT / NODE.JS

```
const nums = [1, 2, 3];
const point = [10, 20]; // JS has no real tuple
const unique = new Set([1, 2, 3]);
const user = { name: "Ahmed", age: 31 };
```

Python	JavaScript	Mutable?	Use for
<code>list</code>	<code>Array</code>	yes	ordered, growable sequence
<code>tuple</code>	<i>(none)</i>	no	fixed, heterogeneous record; dict keys
<code>set</code>	<code>Set</code>	yes	uniqueness, membership tests
<code>dict</code>	<code>Object</code> / <code>Map</code>	yes	key→value mapping

Access and common operations:

PYTHON

```

user["name"]           # dict access (KeyError if missing)
user.get("email", "none") # safe access with default
nums.append(4); nums[0]; nums[-1] # -1 = last element (nice!)
nums[1:3]              # slicing -> [2, 3]
"name" in user         # key membership

```

NODE.JS MENTAL MODEL

Two Python collection features have no clean JS analog and you will love both: negative indexing (`nums[-1]` is the last element) and slicing (`nums[1:3]` , `nums[::-1]` to reverse). Also, `dict` preserves insertion order (guaranteed since 3.7), just like a JS object — so iteration order is predictable.

1.4 None, booleans, truthiness

`None` is Python's `null` / `undefined` (there is only one). Booleans are `True` / `False` (capitalized). Truthiness resembles JS but with collection rules: empty containers are falsy.

PYTHON

```

x = None
if x is None: ...           # identity check for None (idiomatic)
value = maybe or "default"  # like JS ||
value = maybe if cond else "other" # ternary (word order differs!)

```

Value	JS falsy?	Python falsy?
<code>""</code>	yes	yes
<code>0</code> , <code>0.0</code>	yes	yes
<code>[]</code> , <code>{}</code> , <code>()</code>	no (truthy)	yes (empty = falsy)
<code>None</code> / <code>null</code>	yes	yes

COMMON MISTAKE

Two traps: (1) `[]` and `{}` are **truthy in JS but falsy in Python**, so `if items:` means "if non-empty" — the opposite of your JS reflex. (2) Use `is None`, not `== None`, for null checks (identity vs equality). And the ternary reads value-first: `a if cond else b`, not `cond ? a : b`.

1.5 Conditionals, loops, and pattern matching

PYTHON

```
for item in items:           # like for...of
    print(item)
for i, item in enumerate(items): # index + value
    ...
for key, val in user.items():   # dict iteration
    ...
while cond:
    ...
```

There is no C-style `for(;;)`; you iterate over sequences or `range(n)`. Python 3.10+ has **structural pattern matching** (`match/case`) — far more powerful than a `switch`, closer to Rust/Scala matching:

PYTHON

```
match command.split():
    case ["go", direction]:
        move(direction)
    case ["drop", *items]:      # capture the rest
        drop(items)
    case {"type": "user", "id": uid}: # match dict shape
        load(uid)
    case _:                     # default
        unknown()
```

1.6 Functions, defaults, *args*, *kwargs, keyword args

JAVASCRIPT / NODE.JS

```
function makeUser(name, age = 18, ...roles) {
    return { name, age, roles };
}
```

PYTHON

```
def make_user(name: str, age: int = 18, *roles: str, admin: bool = False) -> dict:
    return {"name": name, "age": age, "roles": roles, "admin": admin}
```

```
make_user("Ada", 30, "billing", admin=True)  # keyword argument
make_user(name="Ada")                       # all args can be keyword args
```

- `*args` collects extra **positional** args into a tuple (JS rest params).
- `**kwargs` collects extra **keyword** args into a dict (no JS analog — JS uses an options object).
- Any parameter can be passed by name; this is idiomatic and improves readability.

COMMON MISTAKE

Mutable default arguments. `def f(items=[])` reuses the *same* list across calls — a notorious Python bug. Use `None` as the sentinel: `def f(items=None): items = items or []`. There is no JS equivalent because JS evaluates defaults per call; Python evaluates them once at definition.

1.7 Lambdas, closures, and higher-order functions

Functions are first-class. `lambda` is the arrow function, but limited to a single expression (use a `def` for anything longer):

JAVASCRIPT / NODE.JS

```
const double = (x) => x * 2;
const add = (a) => (b) => a + b;  // curried closure
```

PYTHON

```
double = lambda x: x * 2
def add(a):
    def inner(b):
        return a + b          # closes over a
    return inner
sorted(users, key=lambda u: u["age"])  # lambda as a callback
```

Python closures capture variables by reference (like JS), but assigning to an outer variable inside a nested function needs `nonlocal` (or `global`) — Python makes rebinding explicit.

1.8 Classes, dataclasses, properties

Python classes are conventional OOP. `self` is the explicit first parameter of every method (Python never hides it). `__init__` is the constructor.

TYPESCRIPT

```
class Money {
    constructor(public readonly amount: number, public readonly currency = "USD") {}
    format() { return `${this.amount} ${this.currency}`; }
}
```

PYTHON

```

from dataclasses import dataclass

@dataclass(frozen=True)           # frozen = immutable (like readonly fields)
class Money:
    amount: int
    currency: str = "USD"
    def format(self) -> str:
        return f"{self.amount} {self.currency}"

m = Money(100, "EUR")
m.format()                        # method call

```

`@dataclass` auto-generates `__init__`, `__repr__`, and `__eq__` from the typed fields — the equivalent of TypeScript's constructor shorthand plus value equality. `frozen=True` makes instances immutable. This is the idiomatic way to write data-holding classes.

Concept	TypeScript	Python
Instance field	<code>this.x</code>	<code>self.x</code>
Constructor	<code>constructor()</code>	<code>__init__(self)</code>
Static method	<code>static m()</code>	<code>@staticmethod</code> / <code>@classmethod</code>
Getter	<code>get x()</code>	<code>@property</code>
Immutable data	<code>readonly</code>	<code>@dataclass(frozen=True)</code>
toString	<code>toString()</code>	<code>__repr__</code> / <code>__str__</code>
Value equality	manual	<code>dataclass</code> <code>__eq__</code>

1.9 Inheritance, protocols, abstract classes

PYTHON

```

from abc import ABC, abstractmethod
from typing import Protocol

class Repository(ABC):           # abstract base class
    @abstractmethod
    def save(self, x) -> None: ...

class Serializable(Protocol):    # structural typing (like a TS interface)
    def to_dict(self) -> dict: ...

```

Protocols are Python's structural interfaces — duck typing made checkable, exactly like a TypeScript `interface`: any class with a matching `to_dict` satisfies `Serializable` without explicitly inheriting it. **ABCs** are nominal abstract classes you `extend`. Composition over inheritance applies here just as it does in your Node code.

1.10 Modules, packages, imports, exceptions

PYTHON

```
# app/services/billing.py
from app.models import User      # absolute import
from decimal import Decimal
import json

class Billing:
    def charge(self, user: User, cents: int) -> None:
        if cents <= 0:
            raise ValueError("amount must be positive")
```

PYTHON

```
try:
    billing.charge(user, amount)
except PaymentDeclined as e:      # catch by type (like PHP; unlike JS single
    handle(e)                     catch)
except (ValueError, KeyError) as e: # catch multiple
    report(e)
else:
    commit()                      # runs if no exception
finally:
    lock.release()
```

Imports are by module path (a package is a directory with an `__init__.py`, or a namespace package). There is no default export; you import named symbols. The `else` clause on `try` (runs only if no exception) has no JS equivalent.

1.11 Context managers, generators, decorators

Context managers (`with`) guarantee setup/teardown — the equivalent of a `try/finally` for resources (files, DB connections, locks), and far cleaner than Node's manual cleanup:

PYTHON

```
with open("big.csv") as f:        # auto-closes even on exception
    for line in f:
        process(line)
```

Generators (`yield`) give lazy iteration, exactly like JS `function*`:

PYTHON

```
def read_lines(path: str):
    with open(path) as f:
```

```
for line in f:
    yield line.strip()    # one line at a time, O(1) memory
```

Decorators (`@`) wrap a function with behavior — closer to real JS/TS decorators than PHP attributes; they are functions that take a function and return a new one:

PYTHON

```
from functools import wraps

def timed(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = fn(*args, **kwargs)
        log.info("took %.3fs", time.perf_counter() - start)
        return result
    return wrapper

@timed
def generate_report(): ...    # now timed on every call
```

Decorators are everywhere in FastAPI (`@app.get(...)`) and Django, so understand them well.

PRODUCTION TIP

Adopt type hints from day one and run `mypy / pyright` in CI. Python's dynamism is a productivity boost and a footgun; hints + a checker recover most of the safety you rely on in TypeScript, and they make Pydantic and FastAPI (which *use* the hints at runtime) work their magic.

EXERCISE 1.1

Port a Node module to Python: a `frozen Money` dataclass with an `add` method returning a new `Money`; a `Currency Enum`; a generator `read_ids` that lazily yields ints from a file; and a `@timed` decorator. Add type hints throughout and make `mypy` pass with zero errors.

Part 2 · Functional Programming in Python

You reach for `filter().map().reduce()` in JavaScript by reflex. Python supports every functional idea, but its *idiomatic* style is different: instead of chaining higher-order methods, Python prefers **comprehensions** and lazy **generators**, with `functools / itertools / operator` filling the gaps. This part rewires your reflexes so your Python reads like Python, not transliterated JavaScript.

2.1 The big idea: comprehensions over method chains

The single most important stylistic shift. Where JS chains `filter` then `map`, Python expresses both in one **comprehension** — a readable, one-pass construct.

JAVASCRIPT / NODE.JS

```
const activeNames = users
  .filter((u) => u.active)
  .map((u) => u.name);
```

PYTHON

```
active_names = [u["name"] for u in users if u["active"]]
```

Read it as: *"the name of each user, for each user in users, if the user is active."* Comprehensions exist for every collection:

PYTHON

```
squares    = [x * x for x in range(10)]           # list comprehension
evens_set  = {x for x in nums if x % 2 == 0}      # set comprehension
by_id      = {u["id"]: u for u in users}          # dict comprehension
lazy       = (x * x for x in range(10_000_000))  # generator (lazy, O(1) memory)
```

NODE.JS MENTAL MODEL

A list comprehension `[f(x) for x in xs if cond(x)]` is `xs.filter(cond).map(f)`, fused into one expression that Python developers read instantly. Reaching for `list(map(f, filter(cond, xs)))` marks you as a newcomer — idiomatic Python uses the comprehension. Save `map / filter` for when you already have a named function to pass.

2.2 map, filter, reduce — and why you rarely use them

They exist, but return lazy iterators and read worse than comprehensions. `reduce` even moved *out* of builtins into `functools` as a gentle hint to use a loop or comprehension instead.

PYTHON

```
from functools import reduce
list(map(lambda x: x * 2, nums))      # works, but prefer [x*2 for x in nums]
list(filter(lambda x: x > 10, nums))  # prefer [x for x in nums if x > 10]
total = reduce(lambda a, b: a + b, nums, 0)  # prefer sum(nums)
```

Python provides specialized builtins that replace most `reduce` uses and read far better:

You want	JS	Python idiom
map	<code>arr.map(f)</code>	<code>[f(x) for x in arr]</code>
filter	<code>arr.filter(p)</code>	<code>[x for x in arr if p(x)]</code>
sum	<code>arr.reduce((a,b)=>a+b,0)</code>	<code>sum(arr)</code>
max / min	<code>Math.max(...arr)</code>	<code>max(arr) / min(arr)</code>
any / every	<code>arr.some/every(p)</code>	<code>any(...) / all(...)</code>
sort by key	<code>arr.sort((a,b)=>...)</code>	<code>sorted(arr, key=...)</code>
unique	<code>[...new Set(arr)]</code>	<code>set(arr) / dict.fromkeys</code>
count	<code>arr.length</code>	<code>len(arr)</code>
flatten	<code>arr.flat()</code>	<code>chain.from_iterable(arr)</code>

PYTHON

```
any(u["active"] for u in users)      # some
all(u["verified"] for u in users)    # every
sorted(users, key=lambda u: u["age"], reverse=True)  # sort desc by age
max(orders, key=lambda o: o["total"])  # the order with the biggest total
```

2.3 functools: partial, reduce, cache, composition

`functools` is the standard-library home for higher-order helpers:

PYTHON

```
from functools import partial, reduce, lru_cache

add = lambda a, b: a + b
add_tax = partial(add, 0.15)      # partial application (bakes in first arg)

@lru_cache(maxsize=None)          # memoize pure functions
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
```

`partial` is partial application without hand-rolling a closure — closer to lodash's `_.partial` than anything built into JS. `lru_cache` is memoization as a one-line decorator (a per-process cache; see Part 15 for cross-process caching).

There is no built-in `compose` / `pipe`, but they are trivial:

PYTHON

```
def pipe(*fns):
    def run(x):
        for f in fns:
            x = f(x)
        return x
    return run

clean = pipe(str.strip, str.lower, slugify)
clean(" Hello World ") # "hello-world"
```

2.4 itertools: lazy pipelines and the tools you'll actually use

`itertools` is Python's answer to lodash's lazy sequence operators. Everything is a **lazy iterator** — nothing materializes until you consume it, so you can process huge or infinite streams in constant memory (the Part 1 generator lesson, industrialized).

PYTHON

```
from itertools import chain, islice, groupby, count, batched

chain.from_iterable(list_of_lists) # flatten one level
islice(huge_generator, 0, 1000)    # take the first 1000, lazily
batched(rows, 500)                 # chunk into groups of 500 (3.12+)
count(1)                           # infinite 1, 2, 3, ...
```

PYTHON

```
# group consecutive items by a key (input must be pre-sorted by that key)
from itertools import groupby
orders.sort(key=lambda o: o["customer_id"])
for customer_id, group in groupby(orders, key=lambda o: o["customer_id"]):
    revenue = sum(o["total"] for o in group)
```

COMMON MISTAKE

`itertools.groupby` groups **consecutive** equal keys — it does *not* sort for you, unlike Laravel's `groupBy` or a manual dict grouping. You must `sort` by the key first, or you'll get fragmented groups. For unsorted grouping, use a `defaultdict(list)` instead. Also: a generator is consumed once — iterate it twice and the second pass is empty.

2.5 The Pythonic grouping pattern (no chain needed)

For the "group and aggregate" task you'd write with `lodash` `_.groupBy`, Python uses `collections.defaultdict` or `Counter`:

PYTHON

```
from collections import defaultdict, Counter

by_customer = defaultdict(list)
for o in orders:
    by_customer[o["customer_id"]].append(o)    # auto-creates the list
revenue = {cid: sum(o["total"] for o in os) for cid, os in by_customer.items()}

status_counts = Counter(o["status"] for o in orders)    # {"paid": 42, "pending": 3}
top3 = status_counts.most_common(3)
```

2.6 operator: named operators for callbacks

`operator` gives you function versions of operators and attribute/item getters — cleaner than lambdas as sort/map keys:

PYTHON

```
from operator import itemgetter, attrgetter

sorted(users, key=itemgetter("age"))    # instead of key=lambda u: u["age"]
sorted(objs, key=attrgetter("created_at")) # instead of key=lambda o: o.created_at
```

2.7 Immutable-style programming

Python doesn't enforce immutability, but you lean into it with `frozen` dataclasses (Part 1), tuples, and `frozenset`, and by *returning new values* rather than mutating. This is the same discipline you apply in functional JS — Python just doesn't give you `Object.freeze` deep-freezing.

WHEN NOT TO FORCE FUNCTIONAL STYLE

Python rewards *readability* over point-free cleverness. A `for` loop with a clear body is more Pythonic than a nested `reduce / map / filter` expression, and a comprehension beats both when it fits on a line or two. Do not port a heavily-chained `lodash/RxJS` pipeline literally — translate it to comprehensions + `itertools`, and drop to an explicit loop when the logic branches. "Flat is better than nested" is in the language's own Zen.

EXERCISE 2.1

Given `orders` (each with `customer`, `total`, `paid`), produce the top 5 customers by paid revenue as a list of `(name, revenue)` tuples, sorted descending — using a comprehension for the filter/extract, a `defaultdict` for grouping, and `sorted(..., key=itemgetter(1), reverse=True)[:5]`. Then rewrite the summation of a 10-million-row file using a generator and confirm memory stays flat.

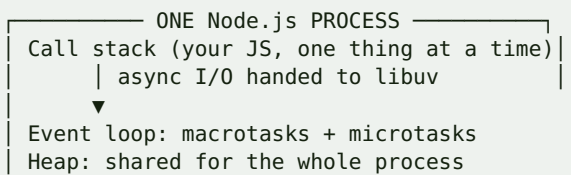
Part 3 · The Python Runtime, the GIL, and asyncio

This is the most important conceptual chapter in the book. Node gave you exactly one concurrency model — a single-threaded event loop — and hid the hard parts. Python gives you **three** models (an asyncio event loop, OS threads, and processes) plus one famous constraint (the GIL) that decides which model fits which workload. Master this chapter and you will always know *where* to run a piece of work.

3.1 Node's model, restated

A Node process runs your JavaScript on one thread with a **libuv** event loop. I/O is non-blocking: you `await`, the loop serves other work, your callback runs when the result is ready. CPU-bound work blocks everything, so you offload it to Worker Threads or child processes.

DIAGRAM



3.2 The GIL: the one sentence that explains Python concurrency

CPython (the standard interpreter) has a **Global Interpreter Lock**: a mutex that allows **only one thread to execute Python bytecode at a time**. Threads are real OS threads, but they take turns holding the GIL. The consequence is precise and worth memorizing:

- **I/O-bound work scales with threads.** While a thread waits on a socket or disk, it *releases* the GIL, so other threads run. Threads overlap I/O well.
- **CPU-bound work does NOT scale with threads.** Two threads crunching numbers can't run in parallel — they fight over the one GIL — so you get no speedup (often a slowdown). For CPU parallelism you need **multiple processes**.

DIAGRAM

GIL: only one thread runs Python bytecode at a time

I/O-bound (threads help):
 T1 —wait socket— (GIL free)
 T2 ■ run ■ —wait—
 T3 ■ run ■

CPU-bound (threads DON'T help):
 T1 ■ compute ■ (holds GIL)
 T2 ...waiting for GIL...
 T3 ...waiting for GIL...

→ overlap = throughput

→ serialized = no speedup
use PROCESSES instead

NODE.JS MENTAL MODEL

Node sidesteps this whole issue by being single-threaded and pushing CPU work to Worker Threads/processes. Python *lets* you use threads but the GIL means they only help for I/O — the same conclusion Node reaches, arrived at differently. For CPU parallelism, both ecosystems end up at "use more processes." (Note: Python 3.13 ships an experimental *free-threaded* no-GIL build, but the mainstream deployment target still has the GIL — design for it.)

3.3 asyncio: Python's event loop is Node's event loop

`asyncio` gives Python a single-threaded event loop that behaves almost exactly like Node's, with syntax you already know. `async def` is `async function`, `await` is `await`, and coroutines are lazy Promises.

JAVASCRIPT / NODE.JS

```
async function loadUser(id) {
  const user = await db.users.find(id);
  const orders = await db.orders.where({ userId: id });
  return { user, orders };
}
```

PYTHON

```
async def load_user(id: int) -> dict:
    user = await db.users.find(id)
    orders = await db.orders.where(user_id=id)
    return {"user": user, "orders": orders}
```

Node	Python asyncio
<code>async function</code>	<code>async def</code>
<code>await x</code>	<code>await x</code>
a <code>Promise</code>	a coroutine / <code>Task</code> / <code>Future</code>
<code>Promise.all([...])</code>	<code>asyncio.gather(...)</code> / <code>TaskGroup</code>
<code>Promise.race</code>	<code>asyncio.wait(..., FIRST_COMPLETED)</code>
<code>setTimeout(fn, ms)</code>	<code>asyncio.sleep(s)</code> (awaitable)
<code>queueMicrotask</code>	<code>loop.call_soon</code>
unhandled rejection	unretrieved-exception warning
<code>AbortController</code>	<code>Task.cancel()</code> / <code>asyncio.timeout</code>
<code>EventEmitter</code>	<code>asyncio.Event</code> / <code>Queue</code> , or a library

3.4 Concurrency with coroutines: gather and TaskGroup

The direct analog of `Promise.all`. Modern Python (3.11+) prefers `TaskGroup` for structured concurrency (automatic cancellation if one fails — safer than `gather`).

JAVASCRIPT / NODE.JS

```
const [a, b, c] = await Promise.all([fetch(u1), fetch(u2), fetch(u3)]);
```

PYTHON

```
import asyncio

# classic: gather (like Promise.all; returns results in order)
a, b, c = await asyncio.gather(fetch(u1), fetch(u2), fetch(u3))

# modern: TaskGroup (structured; if one task raises, the rest are cancelled)
async with asyncio.TaskGroup() as tg:
    ta = tg.create_task(fetch(u1))
    tb = tg.create_task(fetch(u2))
a, b = ta.result(), tb.result()
```

Coroutine vocabulary: a **coroutine** is what `async def` returns (a lazy Promise — it does nothing until awaited or scheduled). A **Task** is a coroutine scheduled to run on the loop (like a Promise already executing). A **Future** is a low-level awaitable result placeholder. You mostly use coroutines and Tasks.

COMMON MISTAKE

Calling an `async def` without `await` does **nothing** — you get a coroutine object and a "coroutine was never awaited" warning, not execution. Unlike a JS Promise (which starts running immediately when created), a Python coroutine is inert until awaited or wrapped in a Task. If you want it to run concurrently *now*, use `asyncio.create_task(coro())`.

3.5 The blocking trap: never block the event loop

Exactly like Node, one synchronous/blocking call freezes the entire asyncio loop. Calling a **blocking** library (a sync DB driver, `requests`, `time.sleep`, heavy CPU) inside an `async def` stalls every other coroutine.

PYTHON

```
# WRONG inside async code – blocks the whole loop
import time, requests
async def bad():
    time.sleep(5)           # blocks EVERYTHING (use asyncio.sleep)
    r = requests.get(url)   # blocks EVERYTHING (use httpx async)

# RIGHT – await async equivalents, or offload blocking work to a thread
async def good():
```

```
await asyncio.sleep(5)
async with httpx.AsyncClient() as c:
    r = await c.get(url)
result = await asyncio.to_thread(cpu_or_blocking_fn, arg)  # run off-loop
```

`asyncio.to_thread` runs a blocking function in a thread pool and awaits it — the escape hatch for using a sync library from async code (the GIL lets the I/O thread release while it waits). For CPU-bound work, offload to a **process** pool instead (Part 14).

3.6 The three concurrency models — and the deployment split

Workload	Best tool	Why
Many concurrent I/O ops (HTTP, DB)	asyncio	one thread, thousands of awaits
Blocking I/O in sync code	threads (<code>ThreadPoolExecutor</code>)	GIL releases on I/O
CPU-bound parallelism	processes (<code>ProcessPoolExecutor</code> , Celery)	bypass the GIL
Slow work off the request	Celery/RQ (Part 13)	durable queue + workers

This drives a fork in the web frameworks, which Parts 5–7 build on:

- **Django (traditional) and DRF run under WSGI — synchronous.** Concurrency comes from running **many worker processes/threads** (Gunicorn), the same "shared-nothing, process-per-request" shape as classic PHP. Each request runs top-to-bottom, blocking I/O and all. Django also has an async mode (ASGI), but the ecosystem is predominantly sync.
- **FastAPI runs under ASGI — asynchronous.** One Uvicorn worker handles many concurrent requests on an event loop, Node-style, *if* your code is `async` and non-blocking end to end.

DIAGRAM



NODE.JS MENTAL MODEL

FastAPI/ASGI is the Node model you know: one event loop, `await` everything, scale by never blocking. Django/WSGI is the classic PHP-FPM model: many synchronous worker processes, scale by adding workers. Choosing FastAPI vs Django is partly choosing which runtime you want — and if you pick FastAPI, you must keep the whole request path non-blocking (`async` DB driver, `httpx` not `requests`), or you lose the benefit exactly as you would in Node.

3.7 EventEmitter and inter-task communication

Node's `EventEmitter` maps to a few asyncio tools depending on need:

PYTHON

```
import asyncio
event = asyncio.Event()           # signal/wait between coroutines
queue = asyncio.Queue()           # producer/consumer pipeline (backpressure)

async def producer():
    await queue.put(item)
async def consumer():
    item = await queue.get()       # awaits until something is available
```

For application-level domain events (Part 12), Django uses **signals** and FastAPI apps use a small pub/sub or a task queue — not a process-global emitter, because (for WSGI) there is no shared long-lived process to hold listeners.

WHEN NOT TO USE ASYNC

Async is not free speed. If your framework is Django/WSGI, writing `async` views against sync ORM calls buys you nothing and adds footguns — stay sync and scale with workers. Even in FastAPI, a purely CPU-bound endpoint gains nothing from `async` (it blocks the loop) — offload it to a process/Celery. Reach for async when you have **many concurrent I/O waits**; otherwise sync code is simpler and just as fast.

EXERCISE 3.1

(1) Write three functions that fetch a URL: one sync-sequential, one with threads, one with `asyncio.gather` + `httpx`; time all three and explain the results via the GIL. (2) Take a CPU-bound function (e.g. hashing 1M items), run it across threads vs a `ProcessPoolExecutor`, and show why only processes speed it up. (3) In two sentences, explain why a `requests.get()` inside a FastAPI `async def` endpoint hurts throughput and what to use instead.

Part 4 · Python Project & Dependency Management

Node standardized on npm early, so dependency management is a solved, single-tool problem. Python's story is messier: multiple tools coexist, and the ecosystem is mid-migration toward `pyproject.toml` and the fast newcomer **uv**. This part maps it all to npm and tells you what to actually use in 2026.

4.1 Virtual environments — the concept npm hides

The biggest conceptual difference: Node installs dependencies **locally** in `node_modules/` per project automatically. Python, by default, installs packages **globally** (into the interpreter), which causes version conflicts across projects. The fix is a **virtual environment** — an isolated per-project Python with its own installed packages. It is `node_modules`, but you must create and "activate" it.

So what is `.venv`? It is just a directory (conventionally named `.venv`) that holds a **copy (or link) of the Python interpreter** plus its own `site-packages/` folder — the place installed packages actually land. When it is "active," the shell puts `.venv/bin` first on your `PATH`, so `python` and `pip` resolve to the project's private copies instead of the system ones. Nothing magic: deactivating just removes it from `PATH` again.

SHELL

```
python -m venv .venv          # create an isolated environment (a .venv/ dir)
source .venv/bin/activate     # activate it (Windows: .venv\Scripts\activate)
which python                  # -> ../myproject/.venv/bin/python (not /usr/
bin)                           bin)
pip install django            # now installs INTO .venv/lib/.../site-packages
deactivate                    # leave it — PATH goes back to normal
```

DIAGRAM

```
myproject/
  .venv/                ← the virtual environment (git-ignored, like node_modules)
    bin/python          ← the project's private interpreter
    lib/python3.13/site-packages/ ← installed deps live here
  pyproject.toml
  src/...
```

NODE.JS MENTAL MODEL

A virtualenv is `node_modules` plus a scoped `node / npx` — but explicit. `node_modules` sits in your project and Node finds it automatically; a `.venv` sits in your project too, but Python only finds it once it's on `PATH` (via activation, or because a tool like `uv` / Poetry runs Python through it for you). Forgetting to activate it — and installing into the system Python — is the classic beginner mistake, the equivalent of `npm install -g` for everything. Tools like **uv** and **Poetry** create and use the venv for you, so you rarely activate by hand.

4.2 The tool landscape, mapped to npm

npm / Node	pip (classic)	Poetry	uv (recommended)
<code>package.json</code>	<code>requirements.txt</code>	<code>pyproject.toml</code>	<code>pyproject.toml</code>
<code>package-lock.json</code>	<i>(none by default)</i>	<code>poetry.lock</code>	<code>uv.lock</code>
<code>node_modules/</code>	the venv	the venv	<code>.venv</code>
<code>npm install</code>	<code>pip install -r req.txt</code>	<code>poetry install</code>	<code>uv sync</code>
<code>npm install pkg</code>	<code>pip install pkg</code>	<code>poetry add pkg</code>	<code>uv add pkg</code>
<code>npm ci</code>	<code>pip install -r</code> (no lock)	<code>poetry install</code>	<code>uv sync --frozen</code>
<code>npx tool</code>	—	<code>poetry run tool</code>	<code>uvx tool</code> / <code>uv run tool</code>
<code>npm run x</code>	—	<code>poetry run ...</code>	<code>uv run ...</code>
<code>npm install -g cli</code>	<code>pipx install cli</code>	—	<code>uv tool install cli</code>
registry: npm	PyPI	PyPI	PyPI

Recommendation for 2026: use **uv**. It is a single, extremely fast (Rust) tool that manages the Python version, the virtualenv, dependencies, and locking — the closest thing Python has to a modern, batteries-included npm/pnpm.

SHELL

```

uv init myapp && cd myapp           # scaffold with pyproject.toml
uv add django djangoestframework    # add deps (creates/updates uv.lock + .venv)
uv add --dev pytest ruff mpy        # dev dependencies
uv run python manage.py runserver    # run inside the managed env (no activate)
uv sync --frozen                     # reproducible install from the lock (like npm
ci)
```

4.3 Global CLI tools: pipx & uvx (the *real* `npm -g`)

Node blurs two very different things under one command. `npm install lodash` adds a **library** to your project; `npm install -g prettier` installs a **command-line application** you run from anywhere. Python keeps these separate, and matching them up is the single most common point of confusion:

What you want	Node	Python (classic)	Python (uv)
A library <i>for this project</i>	<code>npm i axios</code>	<code>pip install httpx</code> (in a venv)	<code>uv add httpx</code>
A CLI tool available <i>everywhere</i>	<code>npm i -g prettier</code>	<code>pipx install ruff</code>	<code>uv tool install ruff</code>
Run a CLI <i>once</i> , no install	<code>npx prettier .</code>	<code>pipx run ruff .</code>	<code>uvx ruff .</code>

The problem `pipx` solves: if you `pip install` a CLI tool into your global Python, its dependencies pollute the global environment and collide with the next tool you install. **`pipx` gives each CLI app its own hidden virtualenv**, then exposes just the command on your `PATH`. It is exactly `npm -g`, but isolated so tools can never break each other.

SHELL

```

pipx install ruff                # ruff + its deps in a private venv, `ruff` on
PATH
pipx install "poetry"           # each tool isolated from the others
pipx list                       # what you've installed globally
pipx run cowsay "hi"            # download-run-discard (like npx), nothing
installed
pipx upgrade-all                # keep your global tools current

```

`uvx` is uv's built-in equivalent of `pipx run` (and `uv tool install` the equivalent of `pipx install`). If you already use uv, you get this for free — no separate `pipx` needed:

SHELL

```

uvx ruff check .                # ephemeral: fetch ruff, run it, discard (= npx)
uv tool install ruff            # persistent global install (= npm i -g / pipx
install)
uv tool list                     # list globally installed tools

```

WHICH DO I USE?

Inside a project, dependencies go through `uv add` / `pip install` into the project's `.venv`. For standalone developer tools you want on every project (formatters, linters, `httpie`, `pre-commit`), use `uv tool install` (or `pipx install`) so they live *outside* any one project. Rule of thumb: if you `import` it in code, it's a project dependency; if you only *run* it from a terminal, it's a global tool.

COMMON MISTAKE

Running `pip install ruff` with no venv active installs Ruff **and its dependency tree** into your system Python. Do that with a handful of tools and you get the dependency conflicts venvs were invented to prevent ("I upgraded tool A and tool B stopped working"). Global *tools* → `pipx` / `uv tool`; global `pip install` is almost never what you want.

4.4 pyproject.toml — the modern manifest

`pyproject.toml` is Python's `package.json`: project metadata, dependencies, and tool configuration in one file (the community standard that replaced `setup.py / requirements.txt`).

TOML

```
[project]
name = "myapp"
version = "0.1.0"
requires-python = ">=3.13"
dependencies = [
    "django>=5.1",
    "djangoRESTframework>=3.15",
    "psycpg[binary]>=3.2",
]

[dependency-groups]
dev = ["pytest>=8", "ruff>=0.6", "mypy>=1.11"]

[tool.ruff]
line-length = 100

[tool.pytest.ini_options]
DJANGO_SETTINGS_MODULE = "myapp.settings"
```

NODE.JS MENTAL MODEL

`[project.dependencies]` is `"dependencies"`; `[dependency-groups].dev` is `"devDependencies"`. The big difference: `pyproject.toml` also hosts *tool* config under `[tool.*]` — Ruff, mypy, pytest, coverage — the way you might scatter `.eslintrc`, `.prettierrc`, and `jest.config.js` across a Node repo. One file, less clutter.

4.5 Version constraints and locking

Python semver constraints resemble npm's but use different operators:

Constraint	Meaning	npm analog
<code>>=5.1</code>	at least 5.1	<code>>=5.1</code>
<code>~5.1</code>	<code>>=5.1, <6.0</code> ("compatible release")	<code>^5.1</code> (roughly)
<code>==5.1.*</code>	any patch of 5.1	<code>~5.1</code>
<code>>=5, <6</code>	explicit range	same

PRODUCTION TIP

Commit the lockfile (`uv.lock` / `poetry.lock`) and deploy with a frozen install (`uv sync --frozen`), never a fresh resolve on the server — the same discipline as `npm ci` . A bare `requirements.txt` without pinned transitive dependencies is *not* a lockfile; use a real locking tool for reproducible builds. For pip-only setups, use `pip-tools` to compile a fully-pinned `requirements.txt` from a high-level input.

4.6 Package structure and imports

A Python **package** is a directory of modules; imports are absolute from the project root. The modern convention is a `src/` **layout** — your importable code lives under `src/`, which prevents accidentally importing from the working directory instead of the installed package (a subtle bug the flat layout invites).

DIAGRAM

```
myapp/
  pyproject.toml
  uv.lock
  src/myapp/
    __init__.py      ← marks the directory as a package (can be empty)
    settings.py
    api/
      __init__.py
      routes.py
  tests/
```

PYTHON

```
# absolute imports from the package root – always prefer these
from myapp.api.routes import router
from myapp.settings import DATABASE_URL

# relative imports (within the package) are allowed but read worse:
from .routes import router          # "." = current package
```

NODE.JS MENTAL MODEL

`__init__.py` is roughly the folder's `index.js` — it makes a directory an importable unit and can re-export names. `from myapp.api import x` is `import { x } from "myapp/api"`. There is no per-file `module.exports`; every top-level name in a module is importable unless prefixed with `_` by convention. Python has no `node_modules`-style resolution walk — imports resolve against installed packages and your `src/`, which is exactly why the `src/` layout and an *installed* project (`uv sync` / `pip install -e .`) matter.

4.7 Building & publishing a package (vs `npm publish`)

Say you want to share a library — the Python equivalent of `npm publish`. Three pieces matter: a **build backend**, the **artifacts** it produces, and the **publish** step.

1. Declare a build backend in `pyproject.toml`. This is the tool that turns your source into installable artifacts — there is no single built-in one, so you name it (Hatchling is a good modern default; uv uses it by scaffold). This is the rough analog of the `"files"/"main"/` packaging behavior npm bakes in.

TOML

```
[project]
name = "mytoolkit"                # the PyPI name (like "name" in package.json)
version = "1.2.0"
description = "A small toolkit."
readme = "README.md"
requires-python = ">=3.11"
dependencies = ["httpx>=0.27"]
authors = [{ name = "Ada Dev", email = "ada@example.com" }]
license = "MIT"

[project.scripts]                # <-- console entry points (the `bin` field)
mytool = "mytoolkit.cli:main"    # creates a `mytool` command -> calls main()

[project.urls]
Homepage = "https://github.com/ada/mytoolkit"

[build-system]                   # the build backend (no npm-style default)
requires = ["hatchling"]
build-backend = "hatchling.build"
```

2. Understand the two artifacts. `npm publish` uploads one tarball. Python publishes two:

Artifact	What it is	npm analog
sdist (<code>.tar.gz</code>)	source distribution — the code + metadata, built on install	the npm source tarball
wheel (<code>.whl</code>)	pre-built, installs with no build step (fast)	a prebuilt/binary package

`[project.scripts]` is the important mapping for Node developers: it is `package.json`'s `"bin"`. Each line `name = "module:function"` installs a real executable named `name` that calls that function — so after `pip install mytoolkit`, the user simply runs `mytool`. No shebang wrangling.

3. Build and publish — the `npm publish` sequence:

SHELL

```
uv build                # produces dist/*.whl and dist/*.tar.gz
uv publish              # upload to PyPI (prompts for a token)

# classic-tool equivalent:
python -m build          # build sdist + wheel
twine upload dist/*      # upload (twine = the publish client)
```

SHELL

```
# test the whole thing against TestPyPI first (like npm's dry-run instincts):
uv publish --publish-url https://test.pypi.org/legacy/
```

PRODUCTION TIP

Authenticate with a **scoped PyPI API token**, not your password, and store it in `~/.pypirc` or CI secrets (`UV_PUBLISH_TOKEN` / `TWINE_PASSWORD`). Enable **Trusted Publishing** (OIDC) from GitHub Actions to publish with no long-lived token at all — the modern equivalent of npm's provenance/2FA publishing. And remember: **PyPI releases are immutable** — you can *yank* a version but never overwrite it, so bump the version for every upload (unlike the occasional `npm unpublish` within 72h).

WHEN NOT TO PUBLISH TO PYPI

Most backend code is an application, not a library — it never gets published. For sharing across a few internal services, prefer a **private index** (a self-hosted index, GitLab/GitHub package registry, or AWS CodeArtifact) or install straight from Git (`uv add "git+https://github.com/org/repo"`), the way you'd depend on a private Git package in Node. Publish to public PyPI only for genuinely public libraries.

COMMON MISTAKE

Four classics for Node developers: (1) installing into the system Python by forgetting to activate the venv (or just use `uv run`, which never forgets); (2) committing `.venv/` to git — gitignore it like `node_modules`; (3) treating an unpinned `requirements.txt` as reproducible — it is not; two installs a week apart can pull different transitive versions, so use a lock; (4) reusing a version number on PyPI — it will be rejected, because releases are immutable.

EXERCISE 4.1

Initialize a project with `uv`, add `fastapi`, `uvicorn`, and dev deps (`pytest`, `ruff`, `mypy`), and run a one-line "hello" app with `uv run`. Inspect `pyproject.toml` and `uv.lock`. Then reproduce the environment from scratch with `uv sync --frozen` and confirm identical versions — the Python equivalent of `npm ci`. Finally, install Ruff *globally* with `uv tool install ruff` (or `pipx install ruff`) and confirm it runs from a different directory — the `npm i -g` analog.

EXERCISE 4.2

Turn a small module into a publishable package: add a `[build-system]` with Hatchling, expose a `[project.scripts]` entry point that prints "hello from my package", run `uv build`, and inspect the generated `dist/*.whl` (it's a zip — open it and read the metadata). Install the wheel into a fresh venv and confirm the console command works. (*Optional*: upload to **TestPyPI** and `pip install` it back from there.)*

Part 5 · Django for Node Developers

Django is the "batteries included" full-stack framework: ORM, migrations, admin, auth, forms, templating, and a strong project structure, all shipped together. For a Node developer the closest analog is **Rails/Laravel** — or a very opinionated **NestJS** with a built-in database and a free admin UI. This part maps Django's architecture; Part 6 covers routing and Part 8 the ORM in depth.

5.1 Projects, apps, and settings

Django splits code into a **project** (the deployable site) containing multiple **apps** (reusable feature modules). This is more structure than a typical Express repo, closer to Nest modules.

SHELL

```
uv run django-admin startproject config .      # the project ("config")
uv run python manage.py startapp projects      # a feature app
```

DIAGRAM

manage.py	– the CLI entrypoint (like a project's npm scripts)
config/	
settings.py	– all configuration (DB, apps, middleware, ...)
urls.py	– root URL routing (the "router")
wsgi.py / asgi.py	– server entrypoints (Part 20)
projects/	– a feature app
models.py	– ORM models
views.py	– request handlers ("controllers")
urls.py	– this app's routes
admin.py	– register models in the admin UI
migrations/	– generated schema migrations
apps.py	– app config

`settings.py` is the central config module (a Python file, not JSON). You register apps in `INSTALLED_APPS`, middleware in `MIDDLEWARE`, and read secrets from the environment.

PYTHON

```
# config/settings.py
INSTALLED_APPS = [
    "django.contrib.admin", "django.contrib.auth", "django.contrib.contenttypes",
    "rest_framework", "projects",          # third-party + your apps
]
DATABASES = {"default": {
    "ENGINE": "django.db.backends.postgresql",
    "NAME": os.environ["DB_NAME"], "USER": os.environ["DB_USER"],
    "PASSWORD": os.environ["DB_PASSWORD"], "HOST": os.environ["DB_HOST"],
}}
```

NODE.JS MENTAL MODEL

An "app" is a feature module you can plug into any project — think a Nest module (`models.py` + `views.py` + `urls.py` + `admin.py`), except Django also generates a full CRUD admin UI for its models for free. That admin alone is often the reason teams pick Django: a back-office you didn't have to build.

5.2 The request lifecycle and middleware

Django processes each request through a **middleware stack**, then routes to a **view**, then back out — the onion model you know from Express, but middleware is a class with hooks rather than `(req, res, next)`.

DIAGRAM

```
request → middleware 1 → middleware 2 → URL resolver → view (your handler)
response ← middleware 1 ← middleware 2 ←
```

PYTHON

```
class TimingMiddleware:
    def __init__(self, get_response):      # set up once (like Express closure)
        self.get_response = get_response
    def __call__(self, request):          # runs per request
        start = time.perf_counter()
        response = self.get_response(request)    # == next(req)
        response["X-Elapsed"] = f"{time.perf_counter()-start:.3f}"
        return response
```

JAVASCRIPT / NODE.JS

```
// Express middleware equivalent
app.use((req, res, next) => {
  const start = performance.now();
  res.on("finish", () => res.set("X-Elapsed", `${performance.now()-start}`));
  next();
});
```

5.3 Views: function-based and class-based

A **view** is Django's controller — a function (or class) taking a `request` and returning a `response`. Plain Django views return HTML or `JsonResponse`; for APIs you almost always use **Django REST Framework** (Part 9), which layers serializers and viewsets on top.

PYTHON

```
from django.http import JsonResponse

def project_list(request):                # function-based view
```

```
projects = Project.objects.filter(owner=request.user).values("id", "name")
return JsonResponse({"data": list(projects)})
```

Class-based views (and DRF's viewsets) provide reusable CRUD behavior — the equivalent of Laravel's resource controllers or a Nest CRUD controller.

5.4 Models, migrations, and the admin

Django's ORM defines schema **in the model class** (like Prisma's schema, but in Python), and it **generates migrations by diffing** models against the last migration — much closer to Prisma's workflow than Laravel's hand-written migrations.

PYTHON

```
# projects/models.py
from django.db import models

class Project(models.Model):
    owner = models.ForeignKey("auth.User", on_delete=models.CASCADE,
                             related_name="projects")
    name = models.CharField(max_length=120)
    status = models.CharField(max_length=20, default="active", db_index=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        indexes = [models.Index(fields=["owner", "status"])]
```

SHELL

```
python manage.py makemigrations      # generate migration from model changes (diff)
python manage.py migrate             # apply migrations
python manage.py createsuperuser     # make an admin login
```

Register the model in `admin.py` and you get a full CRUD web UI for it — no code:

PYTHON

```
from django.contrib import admin
from .models import Project
admin.site.register(Project)      # instant admin CRUD at /admin
```

5.5 Management commands and signals

`manage.py` is Django's CLI (Part 19): `runserver`, `migrate`, `shell`, and your own commands.

Signals are Django's domain-event mechanism — decoupled hooks fired on model save/delete (the analog of Laravel model events / a Node `EventEmitter`), covered in Part 12.

PYTHON

```

from django.db.models.signals import post_save
from django.dispatch import receiver

@receiver(post_save, sender=Project)
def on_project_created(sender, instance, created, **kwargs):
    if created:
        notify_owner(instance)          # side effect on create

```

PRODUCTION TIP

Split settings by environment (`settings/base.py` , `settings/prod.py`) or drive everything from env vars, and **never** run with `DEBUG=True` in production (it leaks stack traces and settings — the exact analog of Laravel's `APP_DEBUG`). Use `django-environ` or `pydantic-settings` to read and validate configuration from the environment at startup.

WHEN NOT TO REACH FOR PLAIN DJANGO VIEWS

For a JSON API, don't hand-roll `JsonResponse` in function views — use **Django REST Framework** (Part 9) for serialization, validation, pagination, and auth, or choose **FastAPI** (Part 7) if you want async and typed request bodies. Plain Django views shine for server-rendered HTML and the admin, not for API surfaces.

EXERCISE 5.1

Scaffold a Django project and a `projects` app. Define the `Project` model, run `makemigrations / migrate` , register it in the admin, create a superuser, and add a project through the admin UI. Add a `TimingMiddleware` and confirm the `X-Elapsed` header appears. Note how much you got — schema, migrations, and a CRUD UI — without writing a controller.

Part 6 · Routing (Django, DRF, and FastAPI)

Routing is where the two Python frameworks diverge most visibly. Django uses an explicit URL-pattern list; FastAPI uses Express/Nest-style decorators on handler functions. This part shows both and maps each back to Express and Laravel.

6.1 Django URL configuration

Django routes live in `urls.py` files as an explicit `urlpatterns` list. The root `config/urls.py` `include()`s each app's `urls.py` — the equivalent of mounting Express sub-routers.

PYTHON

```
# config/urls.py (root)
from django.urls import path, include
urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/projects/", include("projects.urls")),  # mount the app's routes
]

# projects/urls.py
from django.urls import path
from . import views
urlpatterns = [
    path("", views.project_list),  # GET/POST /api/projects/
    path("<int:pk>/", views.project_detail),  # /api/projects/42/
    path("<slug:slug>/", views.by_slug),  # typed path converters
]
```

JAVASCRIPT / NODE.JS

```
// Express equivalent
const projects = express.Router();
projects.get("/", list);
projects.get("/:id", detail);
app.use("/api/projects", projects);
```

Path converters (`<int:pk>`, `<slug:slug>`, `<uuid:id>`) validate and cast the segment — Django's version of a route constraint. Unlike Express, the HTTP method is *not* in the URL pattern; a Django view inspects `request.method` (or, in DRF/CBVs, method-named handlers do it for you).

Feature	Express	Django	FastAPI
Path param	<code>/:id</code>	<code><int:pk></code>	<code>/{id}</code>

Feature	Express	Django	FastAPI
Method binding	<code>router.get</code>	in the view / DRF	<code>@app.get</code>
Mount sub-routes	<code>app.use('/x', r)</code>	<code>include('x.urls')</code>	<code>include_router</code>
Named routes	manual	<code>name=</code> + <code>reverse()</code>	operation ids
Typed/validated param	manual	path converter	type hint + validation

6.2 DRF routers — CRUD routing like Laravel resources

Django REST Framework adds **routers** that generate CRUD routes from a **ViewSet** — the direct analog of Laravel's `apiResource` or a Nest CRUD controller. You register a viewset and get the full set of endpoints.

PYTHON

```
# projects/urls.py with DRF
from rest_framework.routers import DefaultRouter
from .views import ProjectViewSet

router = DefaultRouter()
router.register("projects", ProjectViewSet, basename="project")
urlpatterns = router.urls
```

This generates:

Verb	URL	ViewSet method
GET	<code>/projects/</code>	<code>list</code>
POST	<code>/projects/</code>	<code>create</code>
GET	<code>/projects/{pk}/</code>	<code>retrieve</code>
PUT/PATCH	<code>/projects/{pk}/</code>	<code>update</code> / <code>partial_update</code>
DELETE	<code>/projects/{pk}/</code>	<code>destroy</code>

6.3 FastAPI routing — decorators you already know

FastAPI routing is Express/Nest-style: decorate a handler with the method and path. Path and query parameters become **typed function arguments**, and FastAPI validates and documents them automatically.

PYTHON

```
from fastapi import FastAPI, APIRouter, Query

router = APIRouter(prefix="/projects", tags=["projects"])

@router.get("/{project_id}")
async def get_project(project_id: int, include: str | None = Query(None)):
```

```
#           ^path param (validated int) ^query param with default
...

@router.post("/", status_code=201)
async def create_project(payload: ProjectCreate): # Pydantic body (Part 7)
    ...

app = FastAPI()
app.include_router(router) # mount, like app.use
```

JAVASCRIPT / NODE.JS

```
// Express – but you validate params by hand; FastAPI does it from the types
router.get("/projects/:id", (req, res) => { const id = Number(req.params.id); });
```

NODE.JS MENTAL MODEL

FastAPI routing feels exactly like Express/Nest, with one upgrade: because parameters are **typed function arguments**, FastAPI validates them (a non-int `project_id` returns 422 automatically) and generates OpenAPI docs — no `Number(req.params.id)` casting, no Zod schema for params. The type is the validation. Django's explicit `urlpatterns` list feels more foreign but is just a different spelling of the same route table.

6.4 Route groups, prefixes, and versioning

- **Django:** nest `include()` calls with path prefixes; `path("api/v1/", include("api.v1.urls"))`.
- **DRF:** register viewsets under a prefixed router; namespace versions by app/module.
- **FastAPI:** `APIRouter(prefix="/v1", dependencies=[Depends(auth)])` groups routes with a shared prefix, tags, and dependencies (middleware-like) — the cleanest of the three for grouping.

PYTHON

```
v1 = APIRouter(prefix="/v1", dependencies=[Depends(require_auth)])
v1.include_router(projects_router)
v1.include_router(tasks_router)
app.include_router(v1)
```

COMMON MISTAKE

Django URL patterns are matched **in order, top to bottom**, and a greedy earlier pattern can shadow a later one (e.g. a `<slug>` route before a literal `/new/` route swallows `/new/`). Order specific/literal routes before general ones. In FastAPI the same applies: declare `/projects/search` before `/projects/{project_id}` or "search" is parsed as an id.

EXERCISE 6.1

Build the same "projects + nested tasks" route surface three ways: (1) Django `urlpatterns` with `include()` and path converters; (2) a DRF `DefaultRouter` with a `ProjectViewSet`; (3) a FastAPI `APIRouter` with a `/v1` prefix and an auth dependency. Compare how each handles a typed `project_id` and a missing record.

Part 7 · FastAPI

FastAPI is the Python framework that will feel most familiar and most delightful to a Node developer: it is async-first, built on type hints, validates requests with **Pydantic**, injects dependencies like Nest, and generates interactive OpenAPI docs for free. If Express had TypeScript-native validation, DI, and Swagger built in, it would be FastAPI.

7.1 A complete endpoint, annotated

PYTHON

```
from fastapi import FastAPI, Depends, HTTPException, status
from pydantic import BaseModel, Field

app = FastAPI(title="Projects API")

class ProjectCreate(BaseModel):
    name: str = Field(min_length=2, max_length=120)
    status: str = "active"

class ProjectOut(BaseModel):
    id: int
    name: str
    status: str

@app.post("/projects", response_model=ProjectOut, status_code=201)
async def create_project(payload: ProjectCreate,
                        user: User = Depends(get_current_user)): # DI
    project = await repo.create(owner_id=user.id, **payload.model_dump())
    return project # auto-serialized to ProjectOut
```

Four things happened for free that you hand-build in Express: the JSON body was **validated** against `ProjectCreate` (a bad payload returns 422 with details), the response was **serialized and filtered** to `ProjectOut` (extra fields dropped), `get_current_user` was **injected** as a dependency, and the endpoint appears in **auto-generated Swagger docs** at `/docs`.

7.2 Pydantic — your Zod/Joi, but it powers the whole framework

Pydantic models are typed, validated data classes. FastAPI uses them for request bodies, response models, query/params, and settings. Pydantic v2 is fast (Rust core) and does coercion + validation from type hints.

JAVASCRIPT / NODE.JS

```
// Express + Zod
const Schema = z.object({
  email: z.string().email(),
  age: z.number().int().min(0).default(18),
});
const parsed = Schema.parse(req.body);
```

PYTHON

```
from pydantic import BaseModel, EmailStr, Field

class UserIn(BaseModel):
    email: EmailStr
    age: int = Field(default=18, ge=0)
    tags: list[str] = []

UserIn.model_validate(data)          # validate (raises ValidationError)
UserIn(**data).model_dump()          # -> dict
UserIn(**data).model_dump_json()     # -> JSON string
```

Zod (Node)	Pydantic (Python)
<code>z.object({...})</code>	<code>class X(BaseModel)</code>
<code>z.string().email()</code>	<code>email: EmailStr</code>
<code>z.number().min(0)</code>	<code>Field(ge=0)</code>
<code>.default(18)</code>	<code>= 18 / Field(default=18)</code>
<code>.parse(x)</code> (throws)	<code>Model.model_validate(x)</code>
<code>.safeParse(x)</code>	wrap in try/except <code>ValidationError</code>
infer TS type	the class <i>is</i> the type

NODE.JS MENTAL MODEL

Pydantic is Zod, except the schema **is** the class and the type simultaneously — no separate `z.infer<>` step. And because FastAPI reads those types at runtime, one class definition gives you validation, serialization, and OpenAPI schema all at once. It's the DRY dream you keep reaching for with Zod + DTOs + Swagger decorators in Node.

7.3 Dependency injection — Nest-style, function-based

FastAPI's `Depends` is a clean DI system. A dependency is any callable; FastAPI resolves it (and its own sub-dependencies), caches it per request, and injects the result. This replaces Express middleware for most "prepare something for the handler" needs — auth, DB sessions, pagination params.

PYTHON

```

async def get_db() -> AsyncSession:                # a dependency (yields a resource)
    async with SessionLocal() as session:
        yield session                                # teardown after the request

async def get_current_user(token: str = Depends(oauth2_scheme),
                             db: AsyncSession = Depends(get_db)) -> User:
    user = await db.get(User, decode(token))
    if not user:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED)
    return user

@app.get("/me")
async def me(user: User = Depends(get_current_user)):    # injected
    return user

```

Dependencies compose (one can depend on another), can be applied to whole routers (`APIRouter(dependencies=[...])`), and are trivial to override in tests (`app.dependency_overrides[get_db] = fake_db`) — the DI testability you value in Nest.

7.4 Middleware, exceptions, and background tasks

PYTHON

```

# middleware (ASGI-level; runs around every request)
@app.middleware("http")
async def add_timing(request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    response.headers["X-Elapsed"] = f"{time.perf_counter()-start:.3f}"
    return response

# central exception handling -> consistent error envelopes (Part 9)
@app.exception_handler(DomainError)
async def domain_error_handler(request, exc: DomainError):
    return JSONResponse(status_code=exc.status, content={"error": {"message":
str(exc)}})

# fire-and-forget work after the response (light tasks only)
@app.post("/signup")
async def signup(payload: UserIn, tasks: BackgroundTasks):
    user = await create_user(payload)
    tasks.add_task(send_welcome_email, user.email)    # runs after response
    return {"id": user.id}

```

WHEN NOT TO USE BACKGROUNDTASKS

`BackgroundTasks` runs in the **same process** after the response — fine for a quick, best-effort email, but it has no retries, no durability, and dies if the process restarts. For anything important or slow (payments, external syncs, heavy processing), use **Celery** (Part 13).

`BackgroundTasks` is `setImmediate`, not a real queue.

7.5 Async correctness in FastAPI

FastAPI's throughput advantage (Part 3) only holds if the request path never blocks the event loop. Use **async** database drivers and HTTP clients end-to-end; if you must call a blocking library, offload it.

PYTHON

```
# GOOD: async all the way
async with httpx.AsyncClient() as client:
    r = await client.get(url)                # non-blocking

# If a dependency is blocking (sync ORM, requests), either:
# - define the endpoint as `def` (FastAPI runs it in a threadpool), or
# - offload: result = await run_in_threadpool(blocking_fn)
```

FastAPI is clever here: a `def` (non-async) endpoint is automatically run in a thread pool so it doesn't block the loop; an `async def` endpoint runs on the loop and must not block. Mixing them wrongly (blocking call inside `async def`) is the #1 FastAPI performance bug.

COMMON MISTAKE

Writing `async def` endpoints and then calling a **sync** ORM (`Session`, Django ORM) or `requests.get()` inside them — this blocks the whole event loop and serializes all requests (Part 3). Either go fully async (async SQLAlchemy + `httpx`) or use plain `def` endpoints so FastAPI thread-pools them. Don't half-async.

PRODUCTION TIP

Lean on the auto-generated docs (`/docs` Swagger, `/redoc`) as your API's living contract and manual test surface. Set `response_model` on every endpoint so responses are validated and internal fields never leak, and define a `Settings(BaseSettings)` Pydantic model to load/validate env config at startup (fail fast on misconfiguration).

EXERCISE 7.1

Build a FastAPI `projects` resource: `ProjectCreate` / `ProjectOut` Pydantic models, a `get_db` and `get_current_user` dependency, POST (201) with validation, GET one (404 via `HTTPException`), and a central handler mapping a `ProjectLocked` domain error to 409. Open `/docs` and exercise it. Add a test that overrides `get_current_user` and `get_db` with fakes.

Part 8 · Databases: Django ORM & SQLAlchemy

Python has two dominant ORMs, and which one you use is tied to your framework choice: **Django ORM** (Active Record-ish, used with Django/DRF) and **SQLAlchemy** (Data Mapper, the standard pairing with FastAPI). You know Prisma (explicit, functional) and now Eloquent (Active Record). This part maps all four so you can move between them.

8.1 The four ORMs at a glance

Trait	Prisma	Eloquent	Django ORM	SQLAlchemy
Pattern	data-mapper-ish	Active Record	Active Record-ish	Data Mapper
Schema	<code>schema.prisma</code>	migrations+model	model class	model class
Migrations	diff-generated	hand-written	diff-generated	Alembic (mostly hand)
A row is	POJO	model w/ methods	model instance	mapped instance + Session
Query style	functional	fluent	<code>Model.objects</code> chain	<code>select()</code> + Session
Async	yes	no (sync)	partial	yes (async engine)
Best with	Node	Laravel	Django/DRF	FastAPI

NODE.JS MENTAL MODEL

Django ORM ≈ Eloquent: `Project.objects.filter(...)` reads like a sentence and the model carries `.save()`. SQLAlchemy ≈ Prisma in spirit (explicit, a session/unit-of-work you commit) but more powerful and more verbose. Pick Django ORM for Django apps, SQLAlchemy for FastAPI apps.

8.2 Defining models

PYTHON

```
# Django ORM
class Project(models.Model):
    owner = models.ForeignKey(User, on_delete=models.CASCADE,
related_name="projects")
    name = models.CharField(max_length=120)
    status = models.CharField(max_length=20, default="active", db_index=True)
    created_at = models.DateTimeField(auto_now_add=True)
```

PYTHON

```
# SQLAlchemy 2.0 (typed, declarative)
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy import ForeignKey

class Project(Base):
    __tablename__ = "projects"
    id: Mapped[int] = mapped_column(primary_key=True)
    owner_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    name: Mapped[str] = mapped_column(String(120))
    status: Mapped[str] = mapped_column(default="active", index=True)
    owner: Mapped["User"] = relationship(back_populates="projects")
    tasks: Mapped[list["Task"]] = relationship(back_populates="project")
```

8.3 Migrations

- **Django:** `makemigrations` diffs your models and generates migration files; `migrate` applies them — very close to Prisma's flow.
- **SQLAlchemy:** uses **Alembic**; `alembic revision --autogenerate` diffs models against the DB, but you review/edit the generated script (less magical, more control).

SHELL

```
# Django
python manage.py makemigrations && python manage.py migrate
# SQLAlchemy + Alembic
alembic revision --autogenerate -m "create projects" && alembic upgrade head
```

8.4 CRUD side by side

PYTHON

```
# Django ORM
active = Project.objects.filter(status="active", owner=user).order_by("-created_at")[:20]
p = Project.objects.create(owner=user, name="X")
p.name = "Y"; p.save()
Project.objects.filter(id=5).update(name="Y") # bulk update, no instance
p.delete()
one = Project.objects.get(pk=5) # raises DoesNotExist if missing
first = Project.objects.filter(name="X").first() # or None
```

PYTHON

```
# SQLAlchemy 2.0 (async session)
from sqlalchemy import select

stmt = (select(Project)
        .where(Project.status == "active", Project.owner_id == user.id)
        .order_by(Project.created_at.desc()).limit(20))
```

```

active = (await session.scalars(stmt)).all()

p = Project(owner_id=user.id, name="X")
session.add(p); await session.commit()           # unit of work: add, then commit
p.name = "Y"; await session.commit()             # dirty tracking auto-updates
await session.delete(p); await session.commit()
one = await session.get(Project, 5)               # by PK, or None

```

Prisma	Django ORM	SQLAlchemy
<code>findMany({where})</code>	<code>.objects.filter(...)</code>	<code>select().where(...)</code>
<code>findUnique({where:{id}})</code>	<code>.objects.get(pk=id)</code>	<code>session.get(M, id)</code>
<code>create({data})</code>	<code>.objects.create(...)</code>	<code>session.add(M(...)) + commit</code>
<code>update({where,data})</code>	<code>.filter(...).update(...)</code>	<code>mutate + commit</code>
<code>delete</code>	<code>.delete()</code>	<code>session.delete() + commit</code>
<code>count</code>	<code>.count()</code>	<code>select(func.count())</code>
<code>transaction</code>	<code>atomic()</code>	<code>session.begin()</code>

NODE.JS MENTAL MODEL

SQLAlchemy's **Session** is a unit-of-work: you `add` / `mutate` objects, then `commit()` flushes all changes in one transaction — like Prisma's `$transaction` but as the default working mode. Django ORM auto-saves per `.save()` (Active Record), closer to Eloquent. The Session's identity map also means fetching the same row twice returns the same object.

8.5 Relationships and the N+1 problem

Same N+1 trap as every ORM (Parts 7 of the PHP book): accessing a relation lazily fires a query per row. Both ORMs have eager-loading tools; know their names cold.

PYTHON

```

# Django: N+1 – one query per project for .owner
for p in Project.objects.all():
    print(p.owner.name)           # BAD: 1 + N queries

# Django: eager load (2 queries)
Project.objects.select_related("owner")           # FK/one-to-one -> SQL JOIN
Project.objects.prefetch_related("tasks")         # many -> second query + join
in Python
Project.objects.select_related("owner").prefetch_related("tasks__assignee")

```

PYTHON

```

# SQLAlchemy: eager load
from sqlalchemy.orm import selectinload, joinedload
select(Project).options(joinedload(Project.owner))           # JOIN (to-one)
select(Project).options(selectinload(Project.tasks))         # second IN query (to-

```

```
many)
select(Project).options(selectinload(Project.tasks).selectinload(Task.assigned))
```

Concern	Prisma	Django	SQLAlchemy
Eager to-one	<code>include</code>	<code>select_related</code>	<code>joinedload</code>
Eager to-many	<code>include</code>	<code>prefetch_related</code>	<code>selectinload</code>
Nested	nested <code>include</code>	<code>"a__b"</code>	chained loaders

COMMON MISTAKE

The N+1 is the #1 ORM performance bug in Django and SQLAlchemy exactly as in Prisma/Eloquent, and it hides with small dev datasets. In Django reach for `select_related` (to-one, JOIN) vs `prefetch_related` (to-many, extra query) — using the wrong one silently keeps the N+1. In SQLAlchemy prefer `selectinload` for collections. Install **django-debug-toolbar** or log SQL to see the query count.

8.6 Transactions, locking, and pagination

PYTHON

```
# Django
from django.db import transaction
with transaction.atomic():
    account = Account.objects.select_for_update().get(pk=order.account_id) # row
    lock
    account.balance -= order.total; account.save()
    order.status = "paid"; order.save()

# Django pagination
from django.core.paginator import Paginator
page = Paginator(Project.objects.all(), 20).page(2)
```

PYTHON

```
# SQLAlchemy
async with session.begin(): # transaction block
    stmt = select(Account).where(Account.id == order.account_id).with_for_update()
    account = (await session.scalars(stmt)).one() # SELECT ... FOR UPDATE
    account.balance -= order.total
    order.status = "paid"
# keyset/offset pagination via .limit().offset() or a keyset on an indexed column
```

8.7 Query optimization and indexes

Add indexes to columns you filter, sort, or join on (Django `db_index=True` / `Meta.indexes`; SQLAlchemy `index=True` / `Index(...)`). Use `.only()` / `.defer()` (Django) or `load_only` (SQLAlchemy) to select fewer columns on hot paths, and `.values()` / `select(cols)` when you

don't need full model instances. Profile with the debug toolbar or `EXPLAIN` before optimizing (Part 21).

WHEN NOT TO DROP TO RAW SQL

Both ORMs express the vast majority of queries clearly; use raw SQL (`Model.objects.raw(...)` / `session.execute(text(...))`) only for genuine reporting aggregates, window functions, or measured hot paths — and always bind parameters, never f-string user input into SQL (Part 11). Don't hand-write SQL to avoid learning the query API.

EXERCISE 8.1

Model `User` → `Projects` → `Tasks` (→ `assignee`) in **both** Django ORM and SQLAlchemy. Write a query that loads a user's active projects with tasks and each task's assignee in a bounded number of queries — prove it with SQL logging (`select_related` + `prefetch_related` in Django; `joinedload` + `selectinload` in SQLAlchemy). Wrap a balance-transfer in a transaction with a row lock and test the oversell race.

Part 9 · Production REST APIs (DRF & FastAPI)

You know REST from Express. This part builds production-grade APIs in both Python stacks: **Django REST Framework** (serializers, viewsets — the Laravel-resource experience) and **FastAPI** (Pydantic models, typed endpoints — the Nest/Express experience). The production concerns — status codes, error envelopes, pagination, filtering, versioning, idempotency, webhooks — are the same; the spelling differs.

9.1 Serializers vs Pydantic — the validation/serialization layer

DRF **serializers** validate input and shape output (like a combined Zod schema + DTO). FastAPI uses **Pydantic** models for the same job. Both replace the hand-rolled `res.json(pick(user, [...]))` you write in Express.

PYTHON

```
# DRF serializer
from rest_framework import serializers

class ProjectSerializer(serializers.ModelSerializer):
    task_count = serializers.IntegerField(source="tasks.count", read_only=True)
    class Meta:
        model = Project
        fields = ["id", "name", "status", "task_count", "created_at"]
        read_only_fields = ["id", "created_at"]
```

PYTHON

```
# FastAPI / Pydantic
class ProjectOut(BaseModel):
    id: int
    name: str
    status: str
    task_count: int = 0
    model_config = {"from_attributes": True}  # read straight from ORM objects
```

9.2 CRUD: a DRF ViewSet and FastAPI endpoints

A DRF **ModelViewSet** gives full CRUD from a queryset + serializer — the Laravel-resource experience:

PYTHON

```
class ProjectViewSet(viewsets.ModelViewSet):
    serializer_class = ProjectSerializer
    permission_classes = [IsAuthenticated]
    def get_queryset(self):
        return (Project.objects.filter(owner=self.request.user)
                .select_related("owner").prefetch_related("tasks"))
    def perform_create(self, serializer):
        serializer.save(owner=self.request.user)  # inject owner
```

FastAPI spells out each endpoint but with typed bodies and DI:

PYTHON

```
@router.post("", response_model=ProjectOut, status_code=201)
async def create(payload: ProjectCreate, user=Depends(current_user),
                 db: AsyncSession = Depends(get_db)):
    project = Project(owner_id=user.id, **payload.model_dump())
    db.add(project); await db.commit(); await db.refresh(project)
    return project
```

9.3 Status codes and consistent error envelopes

Both frameworks emit correct codes for common failures; you standardize the error *shape* centrally.

Situation	Code	DRF	FastAPI
Created	201	<code>status=201</code> on create	<code>status_code=201</code>
Deleted	204	default on destroy	<code>status_code=204</code>
Validation	422/400	serializer errors (400)	Pydantic 422 (automatic)
Unauthorized	401	auth classes	dependency raises 401
Forbidden	403	permission classes	dependency raises 403
Not found	404	<code>get_object_or_404</code>	raise <code>HTTPException(404)</code>
Rate limited	429	throttle classes	slowapi / middleware

PYTHON

```
# FastAPI: central handler -> one error shape (Part 7)
@app.exception_handler(DomainError)
async def handle_domain(request, exc: DomainError):
    return JSONResponse(status_code=exc.status,
                        content={"error": {"code": exc.code, "message": str(exc)}})
```

PYTHON

```
# DRF: custom exception handler in settings -> REST_FRAMEWORK["EXCEPTION_HANDLER"]
def custom_exception_handler(exc, context):
```

```

response = drf_default_handler(exc, context)
if response is not None:
    response.data = {"error": {"code": response.status_code, "detail":
response.data}}
return response

```

9.4 Pagination, filtering, sorting, search

- **DRF** ships pagination classes (`PageNumberPagination` , `CursorPagination`) and `django-filter` for allow-listed filtering/sorting — set them and they apply to every list endpoint.
- **FastAPI** has no built-in pagination; use `fastapi-pagination` or accept `limit / offset / cursor` query params and apply them to the query yourself.

PYTHON

```

# DRF settings – global pagination
REST_FRAMEWORK = {
    "DEFAULT_PAGINATION_CLASS": "rest_framework.pagination.PageNumberPagination",
    "PAGE_SIZE": 20,
    "DEFAULT_FILTER_BACKENDS": ["django_filters.rest_framework.DjangoFilterBackend",
                               "rest_framework.filters.OrderingFilter"],
}

```

PYTHON

```

# FastAPI – explicit, allow-listed sort (never trust raw column names)
@router.get("", response_model=list[ProjectOut])
async def list_projects(status: str | None = None, sort: str = "created_at",
                        limit: int = Query(20, le=100), offset: int = 0,
                        db: AsyncSession = Depends(get_db)):
    allowed = {"created_at": Project.created_at, "name": Project.name}
    col = allowed.get(sort, Project.created_at)
    stmt = select(Project).order_by(col.desc()).limit(limit).offset(offset)
    if status:
        stmt = stmt.where(Project.status == status)
    return (await db.scalars(stmt)).all()

```

9.5 Idempotency and webhooks

Same principles as the PHP book (Part 9 there): make unsafe `POST` s idempotent with an `Idempotency-Key` stored in Redis/DB; for inbound webhooks, **verify the signature, dedupe, respond fast, process async** (Celery, Part 13).

PYTHON

```

# FastAPI idempotency dependency
async def idempotent(request: Request, idem_key: str | None = Header(None),
                    redis=Depends(get_redis)):

    if not idem_key:
        return None

```

```

cached = await redis.get(f"idem:{idem_key}")
if cached:
    raise ReplayResponse(json.loads(cached))    # short-circuit with stored
result
return idem_key    # handler stores the response under this key on success

```

PYTHON

```

# Webhook receive: verify HMAC on the RAW body, dedupe, enqueue (Parts 11, 13)
@app.post("/webhooks/stripe")
async def stripe_webhook(request: Request):
    body = await request.body()                # RAW bytes
    expected = hmac.new(secret, body, hashlib.sha256).hexdigest()
    if not hmac.compare_digest(expected, request.headers.get("Stripe-Signature",
    "")):
        raise HTTPException(400, "bad signature")
    event_id = json.loads(body)["id"]
    if await already_processed(event_id):        # dedupe (at-least-once)
        return Response(status_code=204)
    process_webhook.delay(event_id)              # Celery, async
    return Response(status_code=204)            # ack fast

```

COMMON MISTAKE

Doing webhook work synchronously and returning slowly — providers retry and you double-process. Verify → dedupe → enqueue → 2xx immediately, every time. And for idempotency, verify the HMAC against the **raw** request body with `hmac.compare_digest` (timing-safe), never against re-serialized JSON.

PRODUCTION TIP

Set `response_model` (FastAPI) / explicit serializer `fields` (DRF) on every endpoint so internal columns (`password_hash` , internal flags) can never leak into responses. Define domain exceptions and map them to status codes in one central handler so business logic never constructs HTTP responses directly.

EXERCISE 9.1

Build the Projects list endpoint in **both** DRF (ViewSet + serializer + global pagination + `django-filter`) and FastAPI (typed query params + allow-listed sort + explicit limit/offset). Add a central error envelope to each and an idempotency guard on create. Verify a retrieved create with the same key yields one record.

Part 10 · Authentication & Authorization

Django ships a complete auth system (users, sessions, permissions, an admin login) out of the box — like Laravel. FastAPI ships *nothing* and expects you to assemble auth from dependencies — like Express. This part covers both, keeping the **authentication** (who are you) vs **authorization** (what may you do) distinction clean.

10.1 Password hashing

JAVASCRIPT / NODE.JS

```
import bcrypt from "bcrypt";
const hash = await bcrypt.hash(password, 12);
```

PYTHON

```
# Django: built in – uses PBKDF2/argon2, handled by the User model
from django.contrib.auth.hashers import make_password, check_password
# FastAPI: use passlib (or argon2-cffi) yourself
from passlib.context import CryptContext
pwd = CryptContext(schemes=["argon2", "bcrypt"], deprecated="auto")
hash = pwd.hash(password); ok = pwd.verify(password, hash)
```

10.2 Django authentication: sessions and DRF tokens

Django has a `User` model, session middleware, and login machinery ready. For DRF APIs you choose an **authentication class**:

Method	Node analog	DRF class
Session (cookie)	<code>express-session</code>	<code>SessionAuthentication</code>
Token (DB, opaque)	opaque bearer token	<code>TokenAuthentication</code>
JWT	<code>jsonwebtoken</code>	<code>rest_framework_simplejwt</code>
OAuth2 provider	own OAuth server	<code>django-oauth-toolkit</code>

PYTHON

```
# settings.py – DRF global auth + permissions
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ],
}
```

```
"DEFAULT_PERMISSION_CLASSES": ["rest_framework.permissions.IsAuthenticated"],
}
```

PYTHON

```
# JWT login endpoints come from simplejwt
urlpatterns += [
    path("api/token/", TokenObtainPairView.as_view()),      # POST creds ->
    access+refresh
    path("api/token/refresh/", TokenRefreshView.as_view()),
]
```

10.3 FastAPI authentication: dependencies + JWT

FastAPI gives you the OAuth2/bearer *plumbing* (`OAuth2PasswordBearer`) and you implement token issuance/verification, typically with `python-jose` or `pyjwt`. Auth becomes a **dependency** injected into protected routes.

PYTHON

```
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
import jwt

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

@app.post("/token")
async def login(form: OAuth2PasswordRequestForm = Depends(), db=Depends(get_db)):
    user = await authenticate(db, form.username, form.password)
    if not user:
        raise HTTPException(401, "invalid credentials")
    token = jwt.encode({"sub": str(user.id), "exp": expiry()}, SECRET, "HS256")
    return {"access_token": token, "token_type": "bearer"}

async def current_user(token: str = Depends(oauth2_scheme), db=Depends(get_db)) ->
User:
    try:
        payload = jwt.decode(token, SECRET, algorithms=["HS256"])
    except jwt.PyJWTError:
        raise HTTPException(401, "invalid token")
    user = await db.get(User, int(payload["sub"]))
    if not user:
        raise HTTPException(401)
    return user
```

PYTHON

```
@app.get("/me")
async def me(user: User = Depends(current_user)):    # protected by injection
    return user
```

NODE.JS MENTAL MODEL

FastAPI auth is exactly your Express/Nest JWT setup: a login route mints a token, a dependency (guard) verifies the bearer and loads the user, and protected routes declare that dependency. Django/DRF is the Laravel-style "framework provides users, sessions, and pluggable auth classes." JWTs are stateless (can't revoke before expiry) — add a short expiry + refresh tokens, or a token denylist in Redis if you need instant revocation, same tradeoff you weigh in Node.

10.4 Authorization: permissions, roles, policies

- **Django/DRF**: object-level and view-level **permission classes** plus the built-in permissions/groups system. A permission class is checked before the view runs — the analog of a Laravel Policy or a Nest guard.
- **FastAPI**: authorization is just more dependencies (or checks inside them) — a `require_role("admin")` dependency, or an explicit check in the handler.

PYTHON

```
# DRF object-level permission (owner-only edit)
class IsOwner(BasePermission):
    def has_object_permission(self, request, view, obj):
        return obj.owner_id == request.user.id

class ProjectViewSet(viewsets.ModelViewSet):
    permission_classes = [IsAuthenticated, IsOwner]
```

PYTHON

```
# FastAPI: a dependency that enforces a rule
def require_role(role: str):
    async def checker(user: User = Depends(current_user)) -> User:
        if role not in user.roles:
            raise HTTPException(403, "forbidden")
        return user
    return checker

@app.delete("/projects/{pid}")
async def delete_project(pid: int, user=Depends(require_role("admin"))):
    ...
```

For rich RBAC, Django has groups/permissions built in; FastAPI apps use a small roles/permissions table or a library like `casbin`.

10.5 Sessions, email verification, password reset

Django provides password reset, email verification, and session management as built-in views/flows (configure and go) — the Laravel-like batteries. FastAPI requires you to build

these (token table, email sending via Part 13, expiry checks), giving you full control at the cost of assembly.

COMMON MISTAKE

Confusing authn and authz: gating everything with only "is authenticated" and forgetting per-object checks, so any logged-in user can edit any record. Enforce **object-level** permissions (DRF `has_object_permission` / a FastAPI dependency that checks ownership) — the same discipline as a Laravel Policy. Also: never log tokens or store them in plaintext; hash opaque DB tokens.

PRODUCTION TIP

Give access tokens a short lifetime (minutes) with refresh tokens, and rate-limit the login endpoint hard (Part 11) against credential stuffing. Prefer the framework's vetted auth (Django auth, simplejwt, passlib) over hand-rolled crypto — rolling your own token/hash scheme is how security bugs are born.

EXERCISE 10.1

Implement JWT auth in **FastAPI** (login issues access+refresh, `current_user` dependency verifies) and in **DRF** (simplejwt + `IsAuthenticated`). Add an owner-only permission so user B gets 403 editing user A's project and 401 without a token. Add a `require_role("admin")` path and test all three outcomes.

Part 11 · Security

Django is famously "secure by default" — it closes most common holes unless you opt out. FastAPI, being minimal, leaves more to you. This part is a practical checklist per threat, with the Node habit and the Python control for each.

11.1 SQL injection

Both ORMs parameterize queries. You only get injectable by building raw SQL with string interpolation.

PYTHON

```
Project.objects.filter(name=user_input)           # SAFE (parameterized)
session.execute(select(Project).where(Project.name == user_input)) # SAFE
session.execute(text("SELECT * FROM p WHERE name = :n"), {"n": user_input}) # SAFE
session.execute(text(f"SELECT * FROM p WHERE name = '{user_input}'")) # DANGER
```

COMMON MISTAKE

Using `text(f"... {value}")` or `.raw("... %s" % value)` — f-strings and `%` formatting bypass parameterization. Always pass bind parameters (`:name` / `%s` with a params tuple). And allow-list column/ `order_by` names from user input (Part 9); identifiers can't be bound as parameters.

11.2 CSRF and CORS

- **Django** enables **CSRF protection** by default for session-based POSTs (middleware + template token). Token/JWT APIs (no cookies) don't need CSRF; DRF session auth still does. Do not blindly disable CSRF.
- **CORS**: use `django-cors-headers` (Django) or `CORSMiddleware` (FastAPI) with **explicit** origins in production — never `*` with credentials.

PYTHON

```
# FastAPI CORS
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(CORSMiddleware,
    allow_origins=["https://app.example.com"], allow_credentials=True,
    allow_methods=["*"], allow_headers=["*"])
```

11.3 XSS and template escaping

For JSON APIs, XSS is mostly a client concern. If you render **Django templates**, `{{ value }}` auto-escapes HTML (safe); `{{ value|safe }}` and `mark_safe()` do not — never wrap user content in them without sanitizing.

11.4 Rate limiting

Not built into Django core or FastAPI; add a library.

PYTHON

```
# FastAPI with slowapi (Redis-backed)
from slowapi import Limiter
limiter = Limiter(key_func=get_remote_address)

@app.post("/login")
@limiter.limit("5/minute")           # blunt credential stuffing
async def login(...): ...
```

DRF has built-in **throttling** classes (`AnonRateThrottle`, `UserRateThrottle`, scoped throttles) configured in settings — the closest to Laravel's `throttle`.

11.5 Secrets, HTTPS, and secure cookies

Read secrets from the environment (validate them at startup with `pydantic-settings` / `django-environ`); never commit them. In production: force HTTPS, and set secure cookie flags.

PYTHON

```
# Django production settings
DEBUG = False                                # CRITICAL – never True in prod
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_HSTS_SECONDS = 31536000
SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https") # behind a proxy
```

11.6 File upload security

Validate content type and size; store under generated names; keep user uploads off any path where code can execute; put private files on private storage and serve via signed URLs (Part 17).

PYTHON

```
# FastAPI upload with size/type checks
@app.post("/avatar")
```

```

async def upload(file: UploadFile):
    if file.content_type not in {"image/jpeg", "image/png", "image/webp"}:
        raise HTTPException(415, "unsupported type")
    data = await file.read()
    if len(data) > 2 * 1024 * 1024:
        raise HTTPException(413, "too large")
    key = f"avatars/{uuid4()}_{Path(file.filename).suffix}" # generated name
    await storage.put(key, data)

```

11.7 SSRF — the Python-relevant server-side threat

Server-Side Request Forgery: if your server fetches a user-supplied URL (webhooks, "import from URL", link previews), an attacker can point it at internal services (<http://169.254.169.254/> cloud metadata, `localhost` admin ports). This bites Python backends often. Defenses: allow-list schemes/hosts, resolve and reject private/loopback IP ranges, disable redirects to internal hosts, and set timeouts.

PYTHON

```

import ipaddress, socket
def assert_public_url(url: str) -> None:
    host = urlparse(url).hostname
    ip = ipaddress.ip_address(socket.gethostbyname(host))
    if ip.is_private or ip.is_loopback or ip.is_link_local:
        raise ValueError("blocked internal address") # stops SSRF to metadata/localhost

```

11.8 Webhook signature verification

PYTHON

```

import hmac, hashlib
expected = hmac.new(secret, raw_body, hashlib.sha256).hexdigest()
if not hmac.compare_digest(expected, provided_signature): # timing-safe
    raise HTTPException(400)

```

11.9 The checklist

Control	Django default	FastAPI
SQL injection	safe (ORM)	safe (ORM); watch <code>text()</code>
CSRF	on (sessions)	N/A for token APIs
XSS (templates)	auto-escaped	N/A (JSON)
Password hashing	built in	passlib/argon2
Rate limiting	DRF throttling	slowapi (add)
CORS	add cors-headers	CORSMiddleware

Control	Django default	FastAPI
Secrets	env	pydantic-settings
HTTPS / secure cookies	settings flags	proxy + config
SSRF	your job	your job
Dependency CVEs	<code>pip-audit</code>	<code>pip-audit</code> / <code>uv</code>

WHEN NOT TO TRUST DEFAULTS

Django's defaults are strong but only if you don't turn them off: disabling CSRF "to make the SPA work," setting `ALLOWED_HOSTS=['*']`, or leaving `DEBUG=True` each reopen a hole. FastAPI has *few* defaults, so nearly everything (CORS, rate limiting, security headers, HTTPS) is opt-in — audit that you actually added each control.

EXERCISE 11.1

Harden an endpoint that fetches a user-supplied URL with the SSRF guard above and prove it blocks `http://169.254.169.254/` and `http://localhost`. Add rate limiting to login (slowapi or DRF throttle), verify HMAC on a webhook with `compare_digest`, and run `pip-audit`, resolving advisories. Confirm a `text(f"...{x}")` query is injectable and fix it with bind params.

Part 12 · Application Architecture

Python gives you the same layering choices as any backend — views/controllers, services, repositories, DTOs, domain events — and the same temptation to over-engineer. Django nudges you toward "fat models, thin views"; FastAPI leaves structure entirely to you. This part is a decision guide, including when *not* to add a layer.

12.1 Django's default: fat models, thin views

Django convention keeps business logic close to models and data, with views as thin orchestrators — the mirror of Laravel's "thin controllers." For simple CRUD, a DRF `ViewSet` + `serializer` is the whole architecture; you do not need a service layer.

PYTHON

```
class ProjectViewSet(viewsets.ModelViewSet):
    serializer_class = ProjectSerializer
    def get_queryset(self):
        return Project.objects.filter(owner=self.request.user)
    def perform_create(self, serializer):
        serializer.save(owner=self.request.user)      # thin; no service needed
```

WHEN NOT TO ADD A SERVICE LAYER

If a view is a few lines of orchestration with no branching logic, a `ProjectService.create()` that just calls `Project.objects.create()` is pure ceremony. Add a service/use-case only when logic is non-trivial, reused across entry points (API + CLI + Celery task), or worth isolating for testing. Premature service layers are the most common over-engineering in both Django and FastAPI codebases.

12.2 Services and use-cases when logic grows

When a use case spans multiple steps (create records, charge, enqueue, notify), extract a **service function** or a single-purpose **use-case** class. In FastAPI this pairs naturally with dependency injection.

PYTHON

```
class CreateProject:
    def __init__(self, db: AsyncSession, events: EventBus):
        self.db, self.events = db, events
    async def __call__(self, owner: User, data: ProjectCreate) -> Project:
        async with self.db.begin():                      # one transaction
            project = Project(owner_id=owner.id, **data.model_dump())
            self.db.add(project)
```

```

    await self.db.flush()
    await self.events.publish(ProjectCreated(project.id))
    return project

```

PYTHON

```

# FastAPI wires it via Depends; the endpoint stays thin
@router.post("", status_code=201)
async def create(payload: ProjectCreate, action: CreateProject = Depends(),
                user=Depends(current_user)):
    return await action(user, payload)

```

NODE.JS MENTAL MODEL

This is your Node instinct exactly: thin route handlers delegating to a `services/` / `use-cases/` layer. FastAPI's `Depends` is Nest's DI; Django's convention is Rails/Laravel's fat-model. The layering judgment is identical across all of them — only the wiring mechanics differ.

12.3 DTOs, Pydantic schemas, and value objects

FastAPI already gives you DTOs for free: **Pydantic models** are your typed request/response objects. For internal domain values with invariants, use a `frozen` dataclass or a Pydantic model as a **value object** (Part 1).

PYTHON

```

from dataclasses import dataclass
@dataclass(frozen=True)
class Money:
    cents: int
    currency: str = "USD"
    def add(self, other: "Money") -> "Money":
        if other.currency != self.currency:
            raise ValueError("currency mismatch")
        return Money(self.cents + other.cents, self.currency)

```

Keep API schemas (Pydantic `*In` / `*Out`) separate from domain/ORM models — the same "don't leak your database into your API" rule you follow in Node.

12.4 Protocols and dependency inversion

For swappable integrations (payment gateways, SMS, storage), define a **Protocol** (structural interface, Part 1) and inject an implementation — the Python way to invert dependencies without a heavy DI framework.

PYTHON

```

from typing import Protocol

```

```
class PaymentGateway(Protocol):
    async def charge(self, cents: int) -> str: ...

class StripeGateway:                                # satisfies the Protocol structurally
    async def charge(self, cents: int) -> str: ...
class FakeGateway:                                   # test double
    async def charge(self, cents: int) -> str: return "ch_fake"

# FastAPI: choose the implementation via a dependency (override in tests)
def get_gateway() -> PaymentGateway: return StripeGateway()
```

12.5 Domain events: Django signals vs an explicit bus

Django signals (`post_save`, custom signals) are the built-in observer mechanism — decoupled side effects fired on model changes (Part 5). They are convenient but *implicit*: reading a model's `save()`, you can't see what listeners fire.

PYTHON

```
from django.dispatch import Signal
project_created = Signal()
project_created.send(sender=Project, instance=project) # fire
```

PYTHON

```
@receiver(project_created)
def send_welcome(sender, instance, **kwargs):
    send_project_email.delay(instance.id)                # Celery task (Part 13)
```

FastAPI has no built-in signals; use a small event bus or just call a use-case, and offload async side effects to Celery.

WHEN NOT TO USE SIGNALS/EVENTS

Signals add the same indirection as any event system: side effects become invisible at the call site, and ordering/failure handling gets murky. Reserve them for genuinely cross-cutting concerns (audit logs, cache invalidation, notifications). For the core steps of a use case, prefer explicit calls in a service — a linear, readable flow beats a scavenger hunt through `receivers`. Django signals in particular are easy to overuse.

12.6 Repositories — usually optional

As in Laravel, wrapping the ORM in repository interfaces is often redundant: Django's `Manager / QuerySet` and SQLAlchemy's `Session` already are your data layer, and both are easily faked in tests. Add repositories only for genuine needs: multiple data backends, strict domain/persistence separation (DDD), or keeping ORM types out of a pure domain core.

Situation	Reach for
Simple CRUD	ViewSet/endpoint + serializer/Pydantic + model
Multi-step use case	+ a service / use-case class
Reused domain value with rules	+ a frozen dataclass value object
Swappable integration	+ a Protocol + injected impl
Decoupled side effects	+ signals / event bus (offloaded to Celery)
Multiple data sources / strict DDD	+ repositories (rarely)

PRODUCTION TIP

Let architecture emerge. Ship the thin ViewSet/endpoint; extract a use-case the first time logic branches or gets reused by a Celery task or CLI command. Every abstraction is a tax paid on every read — add them when duplication or testing pain justifies the cost, not preemptively.

EXERCISE 12.1

Refactor a fat endpoint (create record → attach relation → notify → analytics) into a `CreateProject` use-case wrapped in a transaction, move notification+analytics into a signal/event handler that dispatches Celery tasks, and leave the endpoint at two lines. Define a `PaymentGateway` Protocol with real+fake implementations and inject the fake in a test.

Part 13 · Background Jobs (Celery & friends)

You know BullMQ: producers push jobs to Redis, workers consume them with retries, backoff, and scheduling. Python's dominant equivalent is **Celery**, a mature distributed task queue. This part maps Celery (and the lighter RQ / Dramatiq) to BullMQ and Laravel Queues, and covers the reliability discipline that is identical everywhere.

13.1 The landscape

Tool	Best for	Node/PHP analog
Celery	full-featured, distributed, scheduling, chains	BullMQ / Laravel Queue
RQ	simple Redis queue, minimal	small BullMQ setup
Dramatiq	simpler than Celery, good defaults	BullMQ-ish
arq	async-native (asyncio) queue	BullMQ in an async app
Celery Beat	scheduled/periodic tasks	node-cron / Laravel Scheduler

Celery is the default choice for Django/DRF and sync stacks; **arq** or Dramatiq fit async FastAPI apps better (Celery is sync-first, though usable from FastAPI).

13.2 Defining and dispatching a task

JAVASCRIPT / NODE.JS

```
// BullMQ
await emailQueue.add("welcome", { userId }, { attempts: 5, backoff: 2000 });
new Worker("email", async (job) => { await sendWelcome(job.data.userId); });
```

PYTHON

```
# Celery
from celery import Celery
app = Celery("myapp", broker="redis://localhost:6379/0",
            backend="redis://localhost:6379/1")

@app.task(bind=True, max_retries=5, acks_late=True)
def send_welcome(self, user_id: int):
    try:
        mailer.send_welcome(user_id)
    except SMTPException as exc:
        raise self.retry(exc=exc, countdown=2 ** self.request.retries) # exp backoff
```

```
# dispatch (producer side)
send_welcome.delay(user.id) # enqueue
send_welcome.apply_async(args=[user.id], countdown=300, queue="emails") # delayed +
routed
```

BullMQ / Laravel	Celery
<code>queue.add(job, data)</code>	<code>task.delay(...)</code> / <code>apply_async(...)</code>
worker process	<code>celery -A app worker</code>
<code>attempts</code>	<code>max_retries</code>
<code>backoff</code>	<code>self.retry(countdown=...)</code>
delayed job	<code>countdown=</code> / <code>eta=</code>
named queues	<code>queue=</code> routing
flows / chains	<code>chain(...)</code> , <code>group(...)</code> , <code>chord(...)</code>
repeat / cron	Celery Beat
Bull Board	Flower dashboard

13.3 Running workers

SHELL

```
celery -A myapp worker --loglevel=info --concurrency=4 -Q emails,default
celery -A myapp beat --loglevel=info # the scheduler process (periodic
tasks)
celery -A myapp flower # web dashboard (like Bull Board)
```

Like BullMQ workers and Laravel `queue:work`, Celery workers are **long-lived processes** — they hold imported code in memory and must be **restarted on deploy** to pick up new task code.

COMMON MISTAKE

Deploying new task code and not restarting Celery workers, so they run the old logic (same trap as Laravel's `queue:restart` and BullMQ workers). Your deploy must restart workers. Also: by default Celery acknowledges a task *before* running it (`acks_early`), so a worker crash loses the task — set `acks_late=True` for at-least-once delivery on important work, and make tasks idempotent to tolerate the resulting re-runs.

13.4 Idempotency, retries, and failure handling

Because tasks can run more than once (retries, at-least-once delivery), they **must be idempotent** — the single most important queue discipline, identical to BullMQ/Laravel.

PYTHON

```
@app.task(bind=True, max_retries=5, acks_late=True)
def charge_order(self, order_id: int):
    order = Order.objects.get(pk=order_id)
    if order.status == "charged":           # idempotency guard: already done
        return
    charge_id = gateway.charge(order.total_cents) # external side effect
    order.status = "charged"; order.charge_id = charge_id; order.save()
```

Failed tasks (after retries) go to a dead-letter queue or are logged; Flower and `task.apply_async(retry=...)` policies control retry behavior. Use `autoretry_for`, `retry_backoff=True`, and `retry_jitter=True` for exponential backoff with jitter (avoid synchronized retry stampedes).

13.5 Chains, groups, and chords

PYTHON

```
from celery import chain, group, chord

chain(charge_order.s(order_id), send_receipt.s(), notify_warehouse.s())() #
sequential
group(recompute_stats.s(uid) for uid in ids)()                             # parallel
chord((process.s(i) for i in items), finalize.s())()
# parallel + callback
```

These map to BullMQ flows / Laravel job chains and batches: `chain` = sequence, `group` = parallel fan-out, `chord` = parallel work with an aggregate callback.

13.6 async apps: arq / Dramatiq

If your app is FastAPI/async, Celery's sync core is a slight impedance mismatch. **arq** is asyncio-native (tasks are `async def`, runs on the event loop) and integrates cleanly with async DB drivers and `httpx`:

PYTHON

```
async def send_welcome(ctx, user_id: int):
    await mailer.send(user_id)           # fully async task

class WorkerSettings:
    functions = [send_welcome]
    redis_settings = RedisSettings()
    # enqueue: await redis.enqueue_job("send_welcome", user.id)
```

WHEN NOT TO REACH FOR CELERY

For a small app that needs one or two background tasks, full Celery (broker + result backend + beat + Flower) is heavy. **RQ** or **Dramatiq** are simpler; an async app is better served by **arq**. And FastAPI's `BackgroundTasks` (Part 7) suffices for a fire-and-forget email that can afford to be lost. Match the tool to the reliability you actually need.

PRODUCTION TIP

Route tasks to named queues by urgency (`emails` , `webhooks` , `default` , `reports`) and run dedicated workers per queue so a slow report batch never starves user-facing emails — the same queue-topology thinking as BullMQ. Set `acks_late=True` + idempotent tasks for anything that must not be lost, and monitor with Flower (queue depth, failures, latency).

EXERCISE 13.1

Move welcome-email and an external CRM sync off the registration path into Celery tasks. Make the CRM sync idempotent (guard on an external id) with `max_retries=5` , `retry_backoff` , `retry_jitter` , `acks_late` . Route it to a `webhooks` queue. Force a failure, inspect it in Flower, fix and retry. Confirm redeploying requires restarting the worker to pick up the change.

Part 14 · Async Deep Dive

Part 3 established that `asyncio` is Node's event loop with familiar syntax. This part goes deeper: the concurrency primitives, cancellation and timeouts, `async` generators and context managers, mixing blocking code, and — most importantly — **when `async` helps and when it is pure overhead**.

14.1 Coroutines, Tasks, Futures — the precise vocabulary

- **Coroutine**: what `async def` returns. Inert until awaited or scheduled (a *lazy* Promise). `x = coro()` runs nothing.
- **Task**: a coroutine scheduled on the loop, running concurrently (an *executing* Promise). `t = asyncio.create_task(coro())`.
- **Future**: a low-level awaitable result placeholder; you rarely create these.

PYTHON

```
coro = fetch(url)                # nothing runs yet
task = asyncio.create_task(coro) # now it's running concurrently
result = await task              # await its completion
```

14.2 gather vs TaskGroup (structured concurrency)

PYTHON

```
# gather: run concurrently, collect results in order (like Promise.all)
results = await asyncio.gather(fetch(a), fetch(b), fetch(c))
# with return_exceptions=True, failures come back as values instead of raising
results = await asyncio.gather(*coros, return_exceptions=True) # like allSettled

# TaskGroup (3.11+): structured – if ANY task raises, the rest are cancelled
async with asyncio.TaskGroup() as tg:
    tasks = [tg.create_task(fetch(u)) for u in urls]
    values = [t.result() for t in tasks] # all succeeded (or the group raised)
```

Prefer **TaskGroup** for correctness: it guarantees no orphaned tasks keep running after a sibling fails — the structured-concurrency guarantee `Promise.all` lacks (a rejected `Promise.all` leaves the others running).

14.3 Timeouts and cancellation

PYTHON

```
# timeout (3.11+): cancels the block if it exceeds the budget
try:
    async with asyncio.timeout(5):
        data = await slow_call()
except TimeoutError:
    ...

# cancellation propagates as CancelledError – clean up in finally
task = asyncio.create_task(work())
task.cancel()
```

Node	Python asyncio
<code>Promise.all</code>	<code>gather</code> / <code>TaskGroup</code>
<code>Promise.allSettled</code>	<code>gather(return_exceptions=True)</code>
<code>Promise.race</code>	<code>asyncio.wait(..., FIRST_COMPLETED)</code>
<code>AbortController</code> / <code>signal</code>	<code>Task.cancel()</code>
<code>AbortSignal.timeout(ms)</code>	<code>asyncio.timeout(s)</code>
<code>setTimeout</code>	<code>asyncio.sleep</code> / <code>call_later</code>

14.4 Async generators and context managers

Just like Node's `for await...of` and async disposables:

PYTHON

```
async def stream_rows(query):
    async with db.stream(query) as result:          # async context manager
        async for row in result:                   # async generator / iterator
            yield transform(row)

async for row in stream_rows(q):                   # consume lazily
    await handle(row)
```

Use these to stream large query results or paginated APIs without buffering everything in memory — the async cousin of Part 2's generators.

14.5 Mixing blocking code (the critical skill)

Real apps mix async and blocking libraries. The rules:

- **Blocking I/O (sync DB driver, `requests`, file I/O):** offload to a thread — the GIL releases during I/O so the loop stays free.
- **CPU-bound work (parsing, hashing, image work):** offload to a **process** — threads won't parallelize past the GIL (Part 3).

PYTHON

```
# blocking I/O -> thread (GIL releases while waiting)
result = await asyncio.to_thread(sync_db_call, arg)

# CPU-bound -> process pool (bypass the GIL)
loop = asyncio.get_running_loop()
with ProcessPoolExecutor() as pool:
    result = await loop.run_in_executor(pool, cpu_heavy_fn, data)
```

COMMON MISTAKE

Calling a blocking function directly inside an `async def` (a sync ORM query, `requests.get`, `time.sleep`, a big loop) freezes the **entire** event loop — every concurrent request stalls until it returns (Part 3). Symptoms: "async app is mysteriously slow under load." Fix: `await` an async equivalent, or offload with `to_thread` (I/O) / `run_in_executor` (CPU).

14.6 Async database and HTTP

To keep a FastAPI request path non-blocking end to end, use async drivers: `asyncpg` / `psycopg` (async) via SQLAlchemy's async engine, `redis.asyncio`, and `httpx.AsyncClient` (Part 16). One sync call anywhere on the path forfeits the benefit.

PYTHON

```
async with httpx.AsyncClient(timeout=10) as client:
    r = await client.get(url)                # non-blocking HTTP
async with SessionLocal() as session:        # async DB session
    user = await session.get(User, uid)
```

WHEN NOT TO USE ASYNC

Async pays off with **many concurrent I/O waits**. It does *not* help — and adds real complexity and footguns — when: your framework is Django/WSGI (stay sync, scale with workers); your workload is CPU-bound (use processes); or your endpoints do one quick query and return (sync is simpler and equally fast). "Async everywhere" is a cargo cult; choose it for concurrency, not as a default. A blocking sync codebase behind Gunicorn workers is a perfectly good, boring, scalable choice.

EXERCISE 14.1

Fetch 20 URLs three ways — sequential, `gather`, and `TaskGroup` with a per-request `asyncio.timeout` — and compare timings and failure behavior (kill one URL and observe how each handles it). Then take a CPU-bound function and offload it via `run_in_executor(ProcessPoolExecutor())`, proving the loop stays responsive to other requests during the crunch.

Part 15 · Caching & Redis

Whether your app is Django/WSGI (many processes) or FastAPI/ASGI (one process, but you'll run several), caching lives in **Redis**, shared across all workers — just like PHP and unlike a single long-lived Node process where an in-memory `Map` can persist. This part maps the caching patterns you know onto Django's cache framework and direct Redis use.

15.1 The cache APIs

PYTHON

```
# Django cache framework (backend configured in settings: Redis)
from django.core.cache import cache
cache.set("user:1", user, timeout=3600)    # TTL seconds
user = cache.get("user:1")
cache.delete("user:1")
cache.get_or_set("stats", expensive_fn, timeout=600)  # cache-aside in one call
cache.incr("views:1")                          # atomic counter
```

PYTHON

```
# Direct Redis (redis-py; async variant for FastAPI)
import redis.asyncio as redis
r = redis.from_url("redis://localhost")
await r.set("user:1", json.dumps(user), ex=3600)
raw = await r.get("user:1")
```

15.2 Cache-aside — the pattern you'll use most

JAVASCRIPT / NODE.JS

```
// Node, hand-rolled
let stats = JSON.parse(await redis.get("stats"));
if (!stats) { stats = await compute(); await redis.setex("stats", 600,
JSON.stringify(stats)); }
```

PYTHON

```
# Django: get_or_set does the whole dance
stats = cache.get_or_set("dashboard:stats", compute_stats, timeout=600)

# FastAPI/async: a tiny helper
async def cached(key, ttl, coro):
    if (v := await r.get(key)) is not None:
        return json.loads(v)
    value = await coro()
```

```
await r.set(key, json.dumps(value), ex=ttl)
return value
```

NODE.JS MENTAL MODEL

`cache.get_or_set(key, fn, ttl)` is the memoize-with-TTL helper you always write in Node, standardized on Redis. Because it's external, every worker and every server shares one cache — no per-instance inconsistency to reason about, unlike a process-local `Map` in Node that each instance holds separately.

15.3 Invalidation

Strategy	How	When
TTL expiry	<code>timeout=</code>	tolerably stale data
Explicit delete	<code>cache.delete(key)</code> on write	precise
Signal-driven	delete in a <code>post_save</code> signal (Part 12)	tie cache to data
Versioned keys	embed a version in the key	bulk invalidate by bump
Key prefix flush	delete by pattern (Redis SCAN)	group invalidation

PYTHON

```
@receiver(post_save, sender=Project)
def bust_project_cache(sender, instance, **kwargs):
    cache.delete(f"project:{instance.id}") # invalidate on write
```

15.4 Locks and distributed locks

Multiple workers/processes run concurrently, so "only one may do X" needs a **distributed lock** in Redis (the Redlock pattern you know from Node).

PYTHON

```
# redis-py provides a lock primitive
async with r.lock("report:generate", timeout=120):
    await generate_expensive_report() # exclusive across all workers
```

15.5 Cache stampede (thundering herd)

When a hot key expires, many concurrent requests miss and all recompute at once, hammering the database — same failure as everywhere. Defenses: a lock around the recompute (only one rebuilds, others wait/serve stale), jittered TTLs so keys don't expire simultaneously, and stale-while-revalidate (serve slightly-stale data while one request refreshes).

PYTHON

```

async def cached_swr(key, ttl, stale_ttl, coro):
    v = await r.get(key)
    if v is not None:
        return json.loads(v)  # fresh or within stale window
    async with r.lock(f"lock:{key}", timeout=30):  # only one rebuilds
        v = await r.get(key)  # double-check after acquiring
        if v is not None: return json.loads(v)
        value = await coro()
        await r.set(key, json.dumps(value), ex=ttl)
        return value

```

COMMON MISTAKE

Choosing Django's **local-memory** cache backend (`LocMemCache`) in production — it is per-process, so each Gunicorn worker has its own cache and they never share or invalidate together, causing baffling inconsistencies. Use the **Redis** backend in any multi-worker deployment. Also: caching a hot key with a short TTL and no stampede protection spikes the DB at every expiry.

WHEN NOT TO CACHE

Don't cache to hide a slow query you could fix with an index or eager loading (Parts 8, 21) — caching adds an invalidation problem; an index does not. Cache genuinely expensive, read-heavy, tolerably-stale data. Reaching for Redis before profiling often optimizes the wrong thing and adds a consistency bug you didn't have before.

EXERCISE 15.1

Cache a dashboard-stats endpoint with `get_or_set` (10-min TTL) and invalidate it via a `post_save` signal on `Order`. Simulate a stampede at expiry and mitigate it with the lock-based stale-while-revalidate helper. Add a Redis lock around a "generate monthly report" task so two requests can't build it at once. Confirm a `LocMemCache` backend fails to share across two workers.

Part 16 · HTTP Clients (requests & httpx)

Calling other services in Python means choosing between `requests` (the classic, synchronous, ubiquitous client) and `httpx` (modern, supports both sync *and* async with the same API). For anything in an async app, use `httpx`. Both map cleanly onto Axios.

16.1 The three clients, mapped to Axios

Axios (Node)	requests (sync)	httpx (sync + async)
<code>axios.get(url)</code>	<code>requests.get(url)</code>	<code>httpx.get(url)</code> / <code>await client.get</code>
<code>params: {}</code>	<code>params={}</code>	<code>params={}</code>
<code>headers: {}</code>	<code>headers={}</code>	<code>headers={}</code>
<code>timeout</code>	<code>timeout=</code>	<code>timeout=</code>
Bearer auth	<code>headers={"Authorization": ...}</code>	same / <code>auth=</code>
<code>axios.all</code>	threads / futures	<code>asyncio.gather</code> (async)
interceptors	session hooks	event hooks / transport
throws on non-2xx	<code>raise_for_status()</code>	<code>raise_for_status()</code>

16.2 Sync (requests / httpx)

JAVASCRIPT / NODE.JS

```
const { data } = await axios.get("https://api.x.dev/users", {
  params: { page: 2 }, headers: { Authorization: `Bearer ${token}` }, timeout: 5000,
});
```

PYTHON

```
import httpx
r = httpx.get("https://api.x.dev/users",
              params={"page": 2},
              headers={"Authorization": f"Bearer {token}"},
              timeout=5.0)
r.raise_for_status()           # raise on 4xx/5xx (like axios default)
data = r.json()
```

Use a **client/session** to reuse connections (keep-alive) and set defaults once — the equivalent of an Axios instance:

PYTHON

```
with httpx.Client(base_url="https://api.x.dev", timeout=10,
                  headers={"Authorization": f"Bearer {token}"}) as client:
    users = client.get("/users").raise_for_status().json()
```

16.3 Async (httpx) — the FastAPI default

PYTHON

```
async with httpx.AsyncClient(timeout=10) as client:
    r = await client.get(url)  # non-blocking (Part 14)
    r.raise_for_status()
    data = r.json()
```

Concurrent requests use `asyncio.gather` — the `Promise.all` / `axios.all` analog:

PYTHON

```
async with httpx.AsyncClient() as client:
    responses = await asyncio.gather(
        client.get("/users"), client.get("/plans"), client.get("/flags"))
    users, plans, flags = (resp.json() for resp in responses)
```

COMMON MISTAKE

Using `requests` (synchronous) inside a FastAPI `async def` — it blocks the event loop and serializes every request (Part 3/14). In async code use `httpx.AsyncClient`; if you must use a sync client, offload with `asyncio.to_thread`. Also: creating a new `Client` per request wastes connections — reuse a shared client (via a FastAPI dependency / app state).

16.4 Retries, backoff, and timeouts

`requests` / `httpx` don't retry by default. Add retries via `httpx` transport retries (connection-level) plus an application-level retry (e.g. `tenacity`) for status-based backoff.

PYTHON

```
from tenacity import retry, wait_exponential, stop_after_attempt,
retry_if_exception_type

@retry(wait=wait_exponential(multiplier=0.25, max=8),
       stop=stop_after_attempt(5),
       retry=retry_if_exception_type(httpx.TransportError))  # only connection
errors
async def fetch(client, url):
    r = await client.get(url, timeout=httpx.Timeout(10.0, connect=3.0))
    r.raise_for_status()
    return r.json()
```

PRODUCTION TIP

Always set **both** a connect timeout and a total timeout on every outbound call — a vendor with no timeout ties up your workers (WSGI) or a slot on the loop (ASGI) and cascades into an outage. Retry only **idempotent** requests, with exponential backoff + jitter to avoid stampeding a struggling downstream. Wrap external APIs in a small typed client class so you can inject and fake it in tests.

16.5 Testing without the network

Use `respx` (for `httpx`) or `responses` (for `requests`) to intercept and stub outbound calls — the `nock` / `msw` analog — so tests are fast and deterministic.

PYTHON

```
import respx, httpx

@respx.mock
async def test_fetch_users():
    respx.get("https://api.x.dev/users").mock(
        return_value=httpx.Response(200, json={"data": [{"id": 1}]})
    )
    result = await fetch_users()
    assert result == [{"id": 1}]
```

WHEN NOT TO CALL SYNCHRONOUSLY

If an outbound call isn't needed to build the response (CRM sync, analytics, outbound webhook), don't do it inline — dispatch a Celery task (Part 13). A third-party call on the request path couples your latency and uptime to theirs, whether you're on WSGI or ASGI.

EXERCISE 16.1

Wrap a third-party API in a `WeatherClient` using a shared `httpx.AsyncClient` with `connect+total` timeouts and `tenacity` retries. Add a method that fetches three cities concurrently with `gather`. Write a `respx` test asserting the auth header is sent and that a 500 triggers retries. Then move a non-critical analytics POST into a Celery task.

Part 17 · Files & Object Storage (S3)

File handling in Python centers on **boto3** (the AWS SDK) for direct S3 access, and **django-storages** for a pluggable storage backend in Django (the analog of Laravel's Storage abstraction). FastAPI apps typically use boto3/aioboto3 directly. This part maps uploads, private files, and signed URLs onto what you do in Node.

17.1 The storage options

Task	Node	Python
Upload parsing	<code>multer</code>	Django <code>request.FILES</code> / FastAPI <code>UploadFile</code>
S3 SDK	<code>@aws-sdk/client-s3</code>	boto3 (sync) / aioboto3 (async)
Pluggable disks	—	django-storages (Django)
Signed URL	<code>getSignedUrl</code>	<code>generate_presigned_url</code>
Local dev storage	<code>fs</code>	<code>FileSystemStorage</code> / local dir

17.2 Django with django-storages (S3 as the default backend)

Configure S3 as the storage backend and Django's `FileField/ImageField` and `default_storage` transparently read/write S3 — the closest thing to Laravel's disk abstraction.

PYTHON

```
# settings.py
STORAGES = {"default": {"BACKEND": "storages.backends.s3.S3Storage",
                        "OPTIONS": {"bucket_name": "my-bucket", "region_name": "us-east-1"}}}

# a model field just works against S3
class Document(models.Model):
    file = models.FileField(upload_to="docs/") # saved to S3 on .save()
```

PYTHON

```
from django.core.files.storage import default_storage
path = default_storage.save("avatars/x.png", content) # returns the stored name
url = default_storage.url(path)
```

17.3 FastAPI uploads with boto3

PYTHON

```
import boto3
from uuid import uuid4
s3 = boto3.client("s3")

@app.post("/avatar")
async def upload(file: UploadFile):
    if file.content_type not in {"image/jpeg", "image/png", "image/webp"}:
        raise HTTPException(415, "unsupported type")
    data = await file.read()
    if len(data) > 2 * 1024 * 1024:
        raise HTTPException(413, "too large")
    key = f"avatars/{uuid4()}_{Path(file.filename).suffix}" # generated name
    await asyncio.to_thread(s3.put_object, Bucket="my-bucket", Key=key, Body=data)
    return {"key": key}
```

Note the `asyncio.to_thread` — boto3 is synchronous, so in an async endpoint you offload it to a thread (Part 14) rather than blocking the loop, or use **aioboto3** for a fully async client.

17.4 Private files and presigned (signed) URLs

Never expose private files publicly. Store them in a private bucket and hand out **presigned URLs** that expire; the client downloads directly from S3.

PYTHON

```
url = s3.generate_presigned_url(
    "get_object",
    Params={"Bucket": "my-bucket", "Key": key,
            "ResponseContentDisposition": 'attachment; filename="invoice.pdf"'},
    ExpiresIn=300, # 5 minutes
)
```

JAVASCRIPT / NODE.JS

```
// Node equivalent
const url = await getSignedUrl(s3, new GetObjectCommand({ Bucket, Key }), {
  expiresIn: 300 });
```

NODE.JS MENTAL MODEL

`generate_presigned_url` is exactly `getSignedUrl` from the AWS SDK — a time-limited, signed link to a private object. Offload the download to S3; never stream large private files through your Python workers (they'd tie up a Gunicorn worker or block the event loop). For presigned **uploads**, generate a `put_object` /POST policy URL so the browser uploads straight to S3.

17.5 Streaming and large files

Stream large downloads instead of buffering; for big uploads use S3 multipart or presigned direct uploads. Heavy processing (image resizing, PDF generation) goes on a **Celery task** (Part 13), not the request.

PYTHON

```
# Celery task for thumbnailing (off the request path)
@app.task
def make_thumbnail(key: str):
    data = s3.get_object(Bucket="b", Key=key)["Body"].read()
    thumb = Image.open(io.BytesIO(data)); thumb.thumbnail((400, 400))
    out = io.BytesIO(); thumb.save(out, "WEBP")
    s3.put_object(Bucket="b", Key=key.replace(".", "_thumb."), Body=out.getvalue())
```

COMMON MISTAKE

Processing images or generating PDFs synchronously in the upload request — a few large files exhaust your workers and time out. Accept the upload, store the original, dispatch a Celery task for derivatives, and return immediately. And validate content type/size and use a generated key — never trust the client's filename (path traversal / overwrite risk).

EXERCISE 17.1

Build an avatar upload in FastAPI (validate type/size, store to S3 with a generated key via `to_thread`, dispatch a `make_thumbnail` Celery task) and the Django equivalent with a `FileField` + `django-storages`. Serve private documents via a 5-minute presigned URL and confirm the link 403s after expiry.

Part 18 · Testing

You test Node with Jest and Supertest. Python's standard is **pytest** — concise, fixture-driven, and with a huge plugin ecosystem — plus each framework's test client: Django's `TestCase` / `APIClient` (DRF) and FastAPI's `TestClient`. This part maps them and shows the patterns that make Python test suites fast and isolated.

18.1 The tooling map

Node	Python
Jest / Vitest	pytest (or <code>unittest</code>)
<code>describe / it</code>	plain <code>def test_*</code> functions
<code>expect(x).toBe(y)</code>	<code>assert x == y</code> (pytest rewrites asserts)
<code>beforeEach / setup</code>	fixtures (<code>@pytest.fixture</code>)
Supertest	Django <code>APIClient</code> / FastAPI <code>TestClient</code>
<code>jest.mock</code>	<code>unittest.mock</code> / <code>pytest-mock</code> (mocker)
fake timers	<code>freezegun</code> / <code>time-machine</code>
factories	<code>factory_boy</code> / <code>model_bakery</code>
nock / msw	<code>respx</code> (httpx) / <code>responses</code> (requests)
coverage	<code>pytest-cov</code>

18.2 pytest fixtures — dependency injection for tests

Fixtures are pytest's superpower: reusable setup injected by parameter name (the inverse of `beforeEach` globals). They compose and have scopes.

PYTHON

```
import pytest

@pytest.fixture
def user(db):
    # `db` fixture = database access
    return User.objects.create(email="a@x.dev")

@pytest.fixture
def client(user):
    c = APIClient(); c.force_authenticate(user) # like setting an auth header
    return c

def test_create_project(client):
    # fixtures injected by name
    resp = client.post("/api/projects/", {"name": "Launch"}, format="json")
```

```
assert resp.status_code == 201
assert resp.json()["name"] == "Launch"
```

18.3 DRF and Django feature tests

`pytest-django` gives you a transactional test DB that rolls back per test (isolated + fast, like Laravel's `RefreshDatabase`).

PYTHON

```
@pytest.mark.django_db
def test_forbids_editing_others_project(client, other_user):
    project = Project.objects.create(owner=other_user, name="Theirs")
    resp = client.put(f"/api/projects/{project.id}/", {"name": "Hijack"},
format="json")
    assert resp.status_code == 403                # object-level permission (Part 10)

@pytest.mark.django_db
def test_validation_error(client):
    resp = client.post("/api/projects/", {"name": ""}, format="json")
    assert resp.status_code == 400
    assert "name" in resp.json()
```

18.4 FastAPI tests with TestClient and dependency overrides

FastAPI's biggest testing win: **override dependencies** to swap the DB, auth, or external clients for fakes — the DI testability you value in Nest.

PYTHON

```
from fastapi.testclient import TestClient

def fake_user():
    return User(id=1, email="a@x.dev")

app.dependency_overrides[get_current_user] = fake_user    # inject a fake
app.dependency_overrides[get_db] = get_test_db

client = TestClient(app)

def test_create_project():
    resp = client.post("/projects", json={"name": "Launch"})
    assert resp.status_code == 201
    assert resp.json()["name"] == "Launch"

# async endpoints: use httpx.ASGITransport + pytest-asyncio for true async tests
```

JAVASCRIPT / NODE.JS

```
// Supertest equivalent
it("creates a project", async () => {
  const res = await request(app).post("/projects").send({ name: "Launch" });
  expect(res.status).toBe(201);
});
```

18.5 Factories, mocking, time, and async

PYTHON

```
# factory_boy – realistic model instances (like a typed fixture builder)
class ProjectFactory(factory.django.DjangoModelFactory):
    class Meta: model = Project
    name = factory.Faker("sentence", nb_words=3)
    owner = factory.SubFactory(UserFactory)

ProjectFactory.create_batch(10) # ten projects

# mock a collaborator (pytest-mock)
def test_charges(mockers):
    gateway = mockers.patch("app.services.gateway.charge", return_value="ch_1")
    place_order(order)
    gateway.assert_called_once()

# freeze time
from freezegun import freeze_time
@freeze_time("2026-01-01")
def test_expiry(): ...

# async tests
@pytest.mark.asyncio
async def test_async_fetch():
    result = await fetch_users()
    assert result
```

18.6 What to test, and how much

Layer	Test type	Priority
Endpoints (auth, validation, happy path, 403/404)	API/feature	highest
Business logic (use-cases, value objects)	unit	high
Permissions / authorization	API/unit	high
Celery tasks (enqueue + task logic)	unit + integration	medium
External clients	unit with <code>respx</code>	medium
Serializers / Pydantic schemas	light coverage	low

PRODUCTION TIP

Bias toward **API-level tests** through the real client (`APIClient` / `TestClient`): one test covers routing, validation, auth, the handler, the ORM, and serialization, and survives internal refactors. Use factories over fixtures files, a transactional/rolled-back DB for speed, and `dependency_overrides` /mocks to keep external services out of the suite. Run against Postgres in CI if you use Postgres-specific features (not SQLite).

COMMON MISTAKE

Forgetting `@pytest.mark.django_db` (test can't touch the DB) or hitting real external services in tests (slow, flaky, rate-limited). Mark DB tests, stub HTTP with `respx` / `responses` , and mock Celery (`task.delay` with `CELERY_TASK_ALWAYS_EAGER=True` or by asserting the call) so tests are fast and deterministic.

EXERCISE 18.1

Cover a Projects+Tasks API with pytest: create (201), validation (400/422), unauthorized edit (403), missing record (404), pagination shape. Use `factory_boy` for data, a rolled-back DB, and — in FastAPI — `dependency_overrides` for auth and DB. Assert a welcome task is *enqueued* (not executed) on registration. Target a sub-second suite.

Part 19 · CLIs & Operational Scripts

Laravel has Artisan; Node has npm scripts + commander/yargs. Python splits this into **Django's** `manage.py` (for Django apps) and standalone CLI libraries — **Click** and **Typer** — for everything else. This part maps them and shows the right home for operational scripts (backfills, imports, maintenance).

19.1 Django manage.py

`manage.py` is Django's built-in CLI: run the server, migrations, a REPL, and your own management commands.

Task	Laravel Artisan	Django manage.py
Dev server	<code>artisan serve</code>	<code>manage.py runserver</code>
Migrations	<code>artisan migrate</code>	<code>manage.py migrate</code>
Make migration	<code>artisan make:migration</code>	<code>manage.py makemigrations</code>
REPL	<code>artisan tinker</code>	<code>manage.py shell</code> (or <code>shell_plus</code>)
Custom command	<code>artisan make:command</code>	a <code>management/commands/*.py</code>
Create admin	—	<code>manage.py createsuperuser</code>

PYTHON

```
# projects/management/commands/prune_exports.py
from django.core.management.base import BaseCommand

class Command(BaseCommand):
    help = "Delete export files older than N days"

    def add_arguments(self, parser):
        parser.add_argument("--days", type=int, default=30)
        parser.add_argument("--dry-run", action="store_true")

    def handle(self, *args, **opts):
        cutoff = timezone.now() - timedelta(days=opts["days"])
        qs = Export.objects.filter(created_at__lt=cutoff)
        self.stdout.write(f"Found {qs.count()} old exports")
        if opts["dry_run"]:
            return
        for export in qs.iterator():  # streaming, low memory (Part 2)
            default_storage.delete(export.file.name)
            export.delete()
        self.stdout.write(self.style.SUCCESS("Done"))
```

SHELL

```
python manage.py prune_exports --days 60 --dry-run
```

19.2 The shell (REPL) — like tinker

SHELL

```
python manage.py shell          # a Python REPL with Django loaded
# with django-extensions: shell_plus auto-imports your models
```

PYTHON

```
>>> Project.objects.filter(status="active").count()
>>> send_welcome.delay(User.objects.first().id)  # dispatch a Celery task
```

PRODUCTION TIP

Use the shell for local/staging debugging, but for production data fixes write a **management command** (or a Django data migration) — repeatable, reviewable, testable, and supporting `--dry-run` — rather than ad-hoc shell mutations. Treat the production shell like `psql` on prod: permitted, but with great care.

19.3 Standalone CLIs: Click and Typer

For non-Django apps (a FastAPI service, a library, a data tool), use **Click** (mature, decorator-based) or **Typer** (built on Click, uses type hints — the FastAPI of CLIs). Typer will feel instantly familiar after FastAPI.

PYTHON

```
# Typer – arguments/options come from type hints, like FastAPI
import typer
app = typer.Typer()

@app.command()
def import_users(file: str, dry_run: bool = False, chunk: int = 500):
    """Import users from a CSV in chunks."""
    total = 0
    for batch in read_csv_in_batches(file, chunk):  # generator (Part 2)
        if not dry_run:
            User.objects.bulk_create([User(**row) for row in batch])
            total += len(batch)
            typer.echo(f"processed {total}")
    typer.secho(f"Done: {total} users", fg="green")

if __name__ == "__main__":
    app()
```

SHELL

```
python -m myapp.cli import-users users.csv --dry-run --chunk 1000
```

JAVASCRIPT / NODE.JS

```
// Node equivalent: commander/yargs + a bin script in package.json
```

Node	Python
commander / yargs	Click / Typer
bin script in package.json	[project.scripts] in pyproject.toml
process.argv parsing	decorators + type hints
chalk (colors)	typer.secho / rich

19.4 Running external processes (child_process)

Python's `subprocess` is Node's `child_process`; for concurrent processes use `asyncio.create_subprocess_exec` or a `ProcessPoolExecutor` (Part 14).

PYTHON

```
import subprocess
result = subprocess.run(["ffmpeg", "-i", "in.mp4", "out.webm"],
                        capture_output=True, text=True, timeout=60)
if result.returncode != 0:
    raise RuntimeError(result.stderr)
```

WHEN NOT TO DO HEAVY WORK IN A CLI COMMAND

A management/CLI command should *orchestrate*, not necessarily *do* long heavy work synchronously — for large jobs, have the command dispatch Celery tasks (Part 13) so the work runs on monitored workers with retries. Reserve inline processing for genuinely one-off maintenance where a queue is overkill.

EXERCISE 19.1

Write an `import_users` management command (Django) *and* a Typer CLI that stream a CSV with a generator, validate rows, and `bulk_create` in chunks, with `--dry-run` and colored output. Add a test invoking the command (`call_command('import_users', ...)`) and asserting the created count. Schedule a `--dry-run` nightly via Celery Beat (Part 13).

Part 20 · Production Deployment

Deploying Python web apps hinges on one distinction from Part 3: **WSGI vs ASGI**. Django/DRF are typically WSGI (synchronous), run by **Gunicorn** with many workers; FastAPI is ASGI (async), run by **Uvicorn** (often supervised by Gunicorn). Around either sits Nginx, Postgres, Redis, and Celery workers — a small fleet, like the Laravel deployment.

20.1 WSGI vs ASGI — the server layer

Concept	Node analog	WSGI (Django/DRF)	ASGI (FastAPI)
Interface	—	WSGI (sync)	ASGI (async)
Server	<code>node /PM2</code>	Gunicorn	Uvicorn / Hypercorn
Concurrency	event loop	many worker processes	event loop per worker
Scale by	not blocking	adding processes/threads	not blocking + workers
Blocking code	offload	fine (each worker is sync)	must not block loop

SHELL

```
# Django (WSGI) under Gunicorn: N sync workers
gunicorn config.wsgi:application --workers 4 --bind 0.0.0.0:8000 --timeout 60

# FastAPI (ASGI) under Uvicorn workers managed by Gunicorn
gunicorn app.main:app -k uvicorn.workers.UvicornWorker --workers 4 --bind 0.0.0.0:8000

# FastAPI directly with Uvicorn (simple)
uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

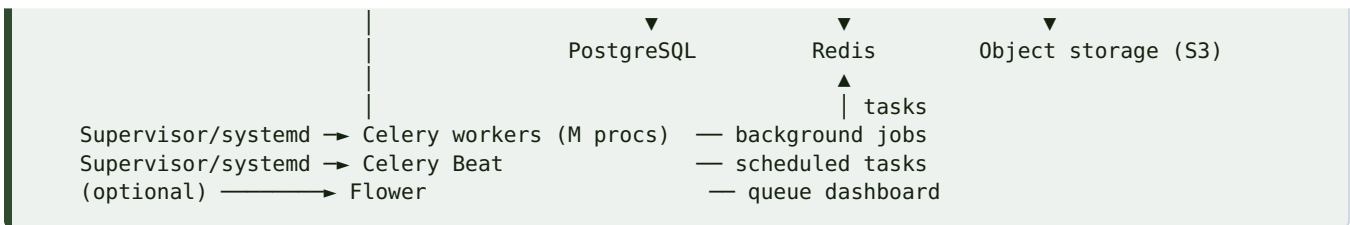
NODE.JS MENTAL MODEL

Gunicorn with N sync workers is the PHP-FPM / cluster-of-Node-processes model — scale throughput by adding processes, each handling one request at a time. Uvicorn (ASGI) is the single-event-loop Node model — one worker handles many concurrent requests, *if* your code never blocks (Part 14). A common production setup runs several Uvicorn workers (Gunicorn-managed) to use all CPU cores *and* get async concurrency within each.

20.2 The production topology

DIAGRAM

```
Internet → Nginx → Gunicorn/Uvicorn (N workers) → app (Django/FastAPI)
```



Static files: Django collects them (`collectstatic`) and serves via Nginx or **WhiteNoise**; FastAPI is API-only so this rarely applies.

20.3 The deploy sequence

SHELL

```

uv sync --frozen                                # reproducible install (like npm ci)
python manage.py migrate --noinput               # apply migrations (Django)
alembic upgrade head                             # apply migrations (SQLAlchemy/FastAPI)
python manage.py collectstatic --noinput         # gather static assets (Django)
# restart app server (Gunicorn/Uvicorn) AND Celery workers (new code, Part 13)
systemctl restart myapp celery-worker celery-beat
  
```

Zero-downtime migrations follow the same expand→migrate→contract rule as the Laravel book: add nullable columns/new tables first, deploy code that writes to both, backfill via a task, then contract (switch reads, drop old, add constraints) in a later deploy — never a breaking schema change in one step.

20.4 Configuration and health checks

PYTHON

```

# pydantic-settings validates env config at startup (fail fast)
from pydantic_settings import BaseSettings
class Settings(BaseSettings):
    database_url: str
    redis_url: str
    secret_key: str
    debug: bool = False
settings = Settings()                        # raises if a required env var is missing
  
```

PYTHON

```

# FastAPI health/readiness endpoints for the load balancer/orchestrator
@app.get("/health")
async def health(): return {"status": "ok"}

@app.get("/ready")
async def ready(db=Depends(get_db)):
    await db.execute(text("SELECT 1")) # check dependencies are reachable
    return {"status": "ready"}
  
```

Ship logs as **structured JSON** to stdout (Part 22); never run Django with `DEBUG=True` in production; set `ALLOWED_HOSTS` explicitly.

20.5 Docker

One image, run with different commands for web / worker / beat:

DOCKERFILE

```
FROM python:3.13-slim
ENV PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
COPY --from=ghcr.io/astral-sh/uv:latest /uv /bin/uv
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev
COPY . .
# web (FastAPI): uv run gunicorn app.main:app -k uvicorn.workers.UvicornWorker -w 4
# web (Django): uv run gunicorn config.wsgi:application -w 4
# worker:      uv run celery -A myapp worker -c 4
# beat:        uv run celery -A myapp beat
```

COMMON MISTAKE

Deploy-day classics: (1) not restarting **Celery workers**, so they run old task code (Part 13); (2) too many Gunicorn workers for the box's memory — each worker is a full process holding the app; size workers to $\sim 2 \times \text{CPU} + 1$ and to available RAM, not arbitrarily high; (3) blocking calls in an async FastAPI app under load (Part 14), which no amount of workers fixes; (4) leaving `DEBUG=True` or a wildcard `ALLOWED_HOSTS` in production.

WHEN NOT TO REACH FOR KUBERNETES/ASYNC PREMATURELY

A single VM with Nginx + Gunicorn + Celery under systemd + managed Postgres/Redis runs a serious app comfortably and is easy to operate. Choose ASGI/async when you have genuine high-concurrency I/O; choose Django/WSGI when you want simplicity and the admin. Add orchestration when you truly need horizontal scale and rolling deploys — complexity is a cost paid every day.

EXERCISE 20.1

Containerize a FastAPI app: Dockerfile + Compose with `web` (Gunicorn+Uvicorn workers), `worker` (Celery), `beat`, `postgres`, `redis`, `nginx`. Add `/health` and `/ready` endpoints, `pydantic-settings` config validation, and a deploy script running migrations + restarting workers. Practice an expand→contract migration with a nullable column backfilled by a Celery task, keeping `/ready` green throughout.

Part 21 · Performance

Python is "slow" only when you let it be. The wins that matter for a backend are almost never about raw interpreter speed — they are about the database, avoiding blocking, and caching. This part orders the levers by impact and shows how to measure before you optimize.

21.1 The measurable wins, in order

Optimize in this order; earlier items dwarf later ones:

1. **Kill N+1 queries** (Part 8): `select_related / prefetch_related` (Django), `joinedload / selectinload` (SQLAlchemy). The single biggest win.
2. **Index columns you filter/sort/join on** (Part 8): a missing index turns a 2 ms query into a full table scan.
3. **Select fewer columns/rows**: `.only() / .values()` (Django), `load_only` (SQLAlchemy), and always paginate — don't fetch 100k rows to show 20.
4. **Don't block the event loop** (ASGI, Parts 3/14): one sync call serializes everything; offload blocking I/O to threads, CPU to processes.
5. **Cache expensive reads** (Part 15): with stampede protection.
6. **Move slow work off the request** (Part 13): Celery.
7. **Then** consider more workers, connection pooling, or PyPy/Rust extensions.

21.2 Finding N+1 and slow queries

PYTHON

```
# Django: see every query a request makes
# pip install django-debug-toolbar (shows query count + timings in the browser)
from django.db import connection
print(len(connection.queries))      # after a request, in tests

# Django: fail loud on N+1 in tests
from django.test.utils import CaptureQueriesContext
with CaptureQueriesContext(connection) as ctx:
    list(view_logic())
assert len(ctx.captured_queries) <= 3  # assert query budget
```

PYTHON

```
# SQLAlchemy: log all SQL
engine = create_async_engine(url, echo=True)  # dev only
```

PRODUCTION TIP

Ninety percent of "the API is slow" is an N+1 or a missing index, both visible in one glance at **django-debug-toolbar** (Django) or SQL echo (SQLAlchemy). Add a **query-count assertion** to your feature tests for hot endpoints so an accidental N+1 fails CI instead of shipping. Measure with a profiler (`py-spy` , `cProfile` , `scalene`) before optimizing code — guessing optimizes the wrong thing.

21.3 CPU-bound work and the GIL

For genuinely CPU-heavy work (parsing, crypto, image/ML), remember Part 3: threads don't help, processes do. Options in impact order: push it to a **Celery worker** (best — off the request entirely), a `ProcessPoolExecutor`, or a compiled extension (NumPy, or a Rust/C library) that releases the GIL internally.

PYTHON

```
# offload CPU work to processes (Part 14)
with ProcessPoolExecutor() as pool:
    result = await loop.run_in_executor(pool, cpu_heavy, data)
```

21.4 Serialization and connection pooling

- **Serialization** can dominate large responses: Pydantic v2 (Rust core) and DRF are both reasonable; for very large payloads use `orjson` (fast JSON) as the response renderer.
- **Connection pooling**: reuse DB connections (Django `CONN_MAX_AGE`, SQLAlchemy pool settings, or PgBouncer in front of Postgres) — opening a connection per request is a hidden tax under load.

PYTHON

```
# Django: keep DB connections warm
DATABASES["default"]["CONN_MAX_AGE"] = 60           # seconds; 0 = new conn per request
```

21.5 Memory and worker sizing

Each Gunicorn/Uvicorn worker is a full process holding the app in memory. Size `workers ≈ 2×CPU + 1`, bounded by RAM. Stream large datasets with generators / `.iterator()` (Part 2) instead of loading them all. Watch for per-request memory growth in long-lived workers (unlike classic PHP, these processes persist — Part 3 — so leaks accumulate, as they do in Node).

WHEN NOT TO OPTIMIZE

Don't add caching, async, multiprocessing, or PyPy to an app serving modest traffic at 30 ms responses. Fix correctness and clarity first; optimize when a profiler under realistic load shows a real bottleneck. Premature optimization buys complexity you maintain forever and speed you didn't need — and often *hides* the actual problem (an N+1 that a cache papers over).

EXERCISE 21.1

Take a deliberately slow endpoint (N+1 + missing index + fetching full rows). Profile it with django-debug-toolbar / SQL echo, then: add the right eager load, add the index, switch to `.only()` / `load_only`, and paginate. Show the before/after query count and latency (target $\geq 5\times$ fewer queries). Add a query-count assertion to a test so a future N+1 fails CI.

Part 22 · Observability

You cannot operate what you cannot see. In Node you reach for `pino / winston`, Prometheus client, OpenTelemetry, and Sentry. Python has direct analogs, and the patterns — structured logs, metrics, traces, health/readiness — are identical. This part maps them and gives you a production-ready baseline.

22.1 Structured logging

The most important habit: emit **JSON logs** with context (request id, user id, duration), not free-form strings. Python's stdlib `logging` plus `structlog` (or `python-json-logger`) gives you `pino`-style structured output.

PYTHON

```
import structlog
log = structlog.get_logger()

log.info("order.created", order_id=order.id, user_id=user.id, total_cents=total)
# -> {"event": "order.created", "order_id": 42, "user_id": 7, "total_cents": 1999,
#     "timestamp": "2026-...", "level": "info"}
```

Node	Python
pino / winston	<code>logging</code> + <code>structlog</code> / <code>python-json-logger</code>
child logger w/ context	<code>log.bind(request_id=...)</code>
log levels	<code>debug/info/warning/error/critical</code>
ship to stdout → collector	same (12-factor)

PRODUCTION TIP

Log to **stdout as JSON** and let your platform (Docker/K8s/systemd) ship it to a collector (Loki, CloudWatch, Datadog) — do not write log files inside the app (12-factor). Bind a **request id** at the start of each request (middleware / dependency) and include it in every log line so you can trace one request across services. Never log secrets, tokens, passwords, or full request bodies with PII.

22.2 Request context and correlation IDs

PYTHON

```
# FastAPI middleware: attach a request id and log timing
@app.middleware("http")
async def logging_mw(request: Request, call_next):
```

```

rid = request.headers.get("X-Request-ID", str(uuid4()))
log_ctx = structlog.contextvars.bind_contextvars(request_id=rid,
                                                  path=request.url.path)

start = time.perf_counter()
response = await call_next(request)
log.info("request.done", status=response.status_code,
        ms=round((time.perf_counter() - start) * 1000, 1))
response.headers["X-Request-ID"] = rid
return response

```

22.3 Metrics

Expose Prometheus metrics (request counts, latencies, in-flight, queue depth) via `prometheus-client` — the same model as Node's `prom-client`. Community packages (`prometheus-fastapi-instrumentator`, `django-prometheus`) wire the common ones automatically.

PYTHON

```

from prometheus_client import Counter, Histogram
REQUESTS = Counter("http_requests_total", "Requests", ["method", "path", "status"])
LATENCY = Histogram("http_request_seconds", "Latency", ["path"])
# increment/observe in middleware; expose /metrics for Prometheus to scrape

```

Track the golden signals: **latency, traffic, errors, saturation** — plus, for this stack specifically, **Celery queue depth and task failure rate** (via Flower or a Celery exporter) and DB connection-pool usage.

22.4 Distributed tracing

OpenTelemetry has first-class Python support with auto-instrumentation for FastAPI, Django, SQLAlchemy, `httpx`, Celery, and Redis — spans propagate across service and task boundaries so you can see a request fan out into DB calls and background jobs.

PYTHON

```

from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
FastAPIInstrumentor.instrument_app(app) # auto-spans for every request
# similarly: SQLAlchemyInstrumentor, HTTPXClientInstrumentor, CeleryInstrumentor

```

22.5 Error tracking and health checks

- **Error tracking:** Sentry has excellent Django/FastAPI/Celery SDKs — `sentry_sdk.init(...)` captures unhandled exceptions with stack traces, request context, and release tags. The direct analog of `@sentry/node`.

- **Health/readiness:** `/health` (process up) and `/ready` (dependencies reachable — DB, Redis) endpoints for the load balancer and orchestrator (Part 20). Keep `/health` cheap; put dependency checks in `/ready`.

PYTHON

```
import sentry_sdk
sentry_sdk.init(dsn=settings.sentry_dsn, traces_sample_rate=0.1,
                environment=settings.env)    # captures errors + performance
```

22.6 The observability checklist

Signal	Tool	Node analog
Structured logs	structlog → stdout	pino
Request correlation	request-id middleware	pino child logger
Metrics	prometheus-client	prom-client
Tracing	OpenTelemetry	OpenTelemetry
Errors	Sentry SDK	@sentry/node
Queue monitoring	Flower / Celery exporter	Bull Board
Health/readiness	<code>/health</code> , <code>/ready</code>	same
Uptime/alerts	your platform + alert rules	same

COMMON MISTAKE

Shipping with `print()` debugging and no request correlation, then being unable to reconstruct what happened during an incident. Set up structured JSON logging with a request id, Sentry for exceptions, and a `/ready` check on day one — not after the first outage. Unstructured logs at scale are nearly unsearchable when you need them most.

EXERCISE 22.1

Add to a FastAPI app: structlog JSON logging, a middleware that binds a request id and logs status+latency per request, a `/metrics` endpoint with request-count and latency, `/health` + `/ready` (checking DB and Redis), and Sentry error capture. Trigger an error and confirm it appears in Sentry with the request id, and that logs for that request share the id.

Part 23 · Capstone Projects

Three production-oriented projects that combine everything. The primary implementation is **FastAPI + SQLAlchemy (async) + Pydantic + Celery**; for each, the **Django/DRF equivalent** is noted so you can build it either way. Build them in order.

23.1 Project A — Task Management API

Users own **projects**, projects have **tasks** assigned to members, with activity tracking. Consolidates Parts 5-12, 18.

Schema (SQLAlchemy)

DIAGRAM

```
users          (id, email[unique], hashed_password, created_at)
projects       (id, owner_id→users, name, status[enum], archived_at?, created_at)
project_members (project_id, user_id, role[owner|member])    -- association table
tasks          (id, project_id→projects, assignee_id→users, title,
               status[todo|doing|done], priority, due_at?, created_at, deleted_at?)
activities     (id, subject_type, subject_id, causer_id→users, event, created_at)
```

Building blocks

- **Schemas:** `ProjectCreate / ProjectOut`, `TaskCreate / TaskOut` Pydantic models with `from_attributes=True` (Parts 7, 9).
- **Auth + authz:** `current_user` dependency (JWT, Part 10); a `require_project_member` dependency that loads the project and 403s non-members — the FastAPI analog of a DRF object permission / Laravel policy.
- **Use-case:** `CreateTask` in a transaction; on assignment, publish a `TaskAssigned` event whose handler dispatches a **Celery** task to email the assignee and write an `activities` row (Parts 12, 13) — never on the request path.
- **Data correctness:** eager-load `project.members` and `task.assignee` with `selectinload` to avoid N+1 (Part 8); index `(project_id, status)` and `assignee_id`; paginate every list (Part 9).
- **Django/DRF equivalent:** `ProjectViewSet / TaskViewSet` with serializers, `IsProjectMember` permission, `select_related / prefetch_related`, DRF pagination, and a `post_save` signal dispatching the Celery task.

Tests (Part 18)

Cover create (201), validation (422), non-member edit (403), missing project (404), pagination shape, and "assigning a non-member is rejected." Assert the assignee email task is **enqueued** (not run) with a mocked/eager Celery. Aim for a sub-second suite with a rolled-back DB and `dependency_overrides`.

BUILD A

Implement A end to end in FastAPI+SQLAlchemy with ≥ 15 tests, then port the models and one endpoint to Django/DRF. Prove members can create tasks but only owners can archive; assigning a non-member 422s; and the notification task is enqueued, not executed, under test.

23.2 Project B — E-commerce API

Money, inventory, transactions, and idempotency — correctness under concurrency. Consolidates Parts 8, 9, 11, 13, 15.

Schema

DIAGRAM

```
products      (id, name, slug[unique], price_cents, currency, active)
inventory     (id, product_id→products, quantity, version)      -- optimistic-lock column
carts         (id, user_id→users, created_at)
cart_items    (id, cart_id→carts, product_id→products, quantity, unit_price_cents)
orders        (id, user_id→users, status[pending|paid|failed|refunded],
               total_cents, currency, idempotency_key?[unique], created_at)
order_items   (id, order_id→orders, product_id, quantity, unit_price_cents)
payments      (id, order_id→orders, provider, provider_charge_id, status)
```

The critical path: place an order (atomic, idempotent, stock-safe)

PYTHON

```
async def place_order(db: AsyncSession, user: User, cart: Cart, idem_key: str) -> Order:
    # idempotency: replay if this key already produced an order (Part 9)
    existing = await db.scalar(select(Order).where(Order.idempotency_key == idem_key))
    if existing:
        return existing

    async with db.begin():
        # one transaction (Part 8)
        order = Order(user_id=user.id, status="pending",
                      idempotency_key=idem_key, currency="USD", total_cents=0)
        db.add(order); await db.flush()
        total = 0
        for item in cart.items:
            # pessimistic lock so two orders can't oversell the last unit (Part 8)
            inv = await db.scalar(
                select(Inventory).where(Inventory.product_id == item.product_id)
                .with_for_update())
            if inv.quantity < item.quantity:
                raise InsufficientStock(item.product_id) # -> 409 (Part 9)
            inv.quantity -= item.quantity
            db.add(OrderItem(order_id=order.id, product_id=item.product_id,
                             quantity=item.quantity,
```

```

unit_price_cents=item.unit_price_cents))
    total += item.quantity * item.unit_price_cents
    order.total_cents = total
    charge_order.delay(order.id)                # charge async on a worker
(Part 13)
    return order

```

Building blocks

- **Money as integer cents** everywhere (never floats); a `frozen Money` value object for arithmetic (Part 12).
- **Idempotency**: an `Idempotency-Key` dependency plus the DB unique constraint as a backstop (Parts 9, 11).
- **Payment**: a `PaymentGateway` Protocol with `Fake` / `Stripe` implementations (Part 12); charging is an **idempotent** Celery task that flips the order to paid and emits `OrderPaid` (Part 13).
- **Webhooks**: verify the provider HMAC on the raw body, dedupe by event id, reconcile status (Parts 9, 11).
- **Django/DRF equivalent**: `select_for_update()` in a `transaction.atomic()` block; the rest maps directly.

BUILD B

Implement checkout with pessimistic inventory locking and idempotent order creation. Write a concurrency test firing two orders for the last unit and assert exactly one succeeds (the other 409s). Verify a retried create with the same idempotency key yields one order. Simulate a payment webhook delivered twice and assert the order flips to `paid` exactly once.

23.3 Project C — Webhook / Integration Platform

Receive provider events, transform, and deliver to customer endpoints with retries and signatures — the reliability capstone. Consolidates Parts 3, 11, 13, 16.

Schema

DIAGRAM

```

sources          (id, name, signing_secret)
inbound_events   (id, source_id→sources, external_id[unique per source],
                  payload[json], received_at, processed_at?)    -- dedupe key
endpoints        (id, user_id→users, url, signing_secret, active)
subscriptions    (id, endpoint_id→endpoints, event_type)
deliveries       (id, endpoint_id→endpoints, inbound_event_id→inbound_events,
                  status[pending|delivered|failed], attempts, last_status_code,
                  next_attempt_at?)

```

Inbound (receive fast, process async)

PYTHON

```

@app.post("/webhooks/{source}")
async def receive(source: str, request: Request, db=Depends(get_db)):
    src = await db.scalar(select(Source).where(Source.name == source))
    if not src: raise HTTPException(404)
    body = await request.body()  # RAW bytes (Part
11)
    expected = hmac.new(src.signing_secret.encode(), body,
hashlib.sha256).hexdigest()
    if not hmac.compare_digest(expected, request.headers.get("X-Signature", "")):
        raise HTTPException(400, "bad signature")
    ext_id = json.loads(body)["id"]
    # dedupe: at-least-once delivery means the same event may arrive twice
    exists = await db.scalar(select(InboundEvent).where(
        InboundEvent.source_id == src.id, InboundEvent.external_id == ext_id))
    if exists:
        return Response(status_code=204)  # already have it
    event = InboundEvent(source_id=src.id, external_id=ext_id,
payload=json.loads(body))
    db.add(event); await db.commit()
    fan_out_event.delay(event.id)  # Celery (Part 13)
    return Response(status_code=204)  # ack fast (Part
3)

```

Outbound (deliver with retries + signatures)

PYTHON

```

@celery_app.task(bind=True, max_retries=8, acks_late=True,
                retry_backoff=True, retry_jitter=True)
def deliver_webhook(self, delivery_id: int):
    d = Delivery.objects.select_related("endpoint",
"inbound_event").get(pk=delivery_id)
    if d.status == "delivered":  # idempotent (Part 13)
        return
    payload = d.inbound_event.payload
    sig = hmac.new(d.endpoint.signing_secret.encode(),
                    json.dumps(payload).encode(), hashlib.sha256).hexdigest()
    try:
        resp = httpx.post(d.endpoint.url, json=payload,
                        headers={"X-Signature": sig},
                        timeout=httpx.Timeout(10.0, connect=3.0))  # (Part 16)
        d.attempts += 1
        if resp.is_success:
            d.status, d.last_status_code = "delivered", resp.status_code
        else:
            d.status, d.last_status_code = "failed", resp.status_code
            d.save(); raise self.retry()  # backoff until max_retries
        d.save()
    except httpx.TransportError as exc:
        d.attempts += 1; d.status = "failed"; d.save(); raise self.retry(exc=exc)

```

Production concerns

Everything rests on Part 3: receive on the web tier in milliseconds, do all real work on Celery workers. Dedupe inbound by `(source, external_id)`; sign outbound and let customers verify (Part 11); make delivery idempotent so retries never double-deliver; monitor with Flower; alert on deliveries stuck `failed` past all retries (a dead-letter). Rate-limit inbound per source; cap payload size.

BUILD C

Implement both pipelines. Test: an inbound event delivered twice is stored once; a flaky endpoint (500 then 200) is retried with backoff and ends `delivered` with the right attempt count; a tampered signature is 400; outbound requests carry a valid `X-Signature`. Confirm the receive endpoint returns 204 in well under 50 ms regardless of downstream speed.

YOU ARE NOW PRODUCTION-READY

Build all three — a permissioned CRUD platform, a concurrency-correct commerce flow, and a reliable async delivery system — in FastAPI+SQLAlchemy, and port at least one to Django/DRF. At that point you can build essentially any Python backend. What remains is depth: domain modeling, operational maturity, and reading framework internals when the docs run out.

Part 24 · Type Checking & Static Typing (mypy & pyright)

You already think in types. The good news: Python's type system will feel deeply familiar — it borrowed heavily from the same ideas as TypeScript, including structural typing and generics. The difference is enforcement. In TypeScript, types are *mandatory* to benefit and are erased at build time. In Python, annotations are **optional and never enforced at runtime by the interpreter** — a separate **type checker** (mypy or pyright) reads them, exactly like `tsc` reads your `.ts` files. The annotations are the `: string` you already write; the checker is `tsc --noEmit`.

24.1 The mental model, mapped to TypeScript

TypeScript	Python	Notes
<code>tsc --noEmit</code>	<code>mypy</code> / <code>pyright</code>	the checker; not the runtime
<code>x: string</code>	<code>x: str</code>	same idea, different syntax
<code>x?: number / number \ undefined</code>	<code>x: int \ None</code>	Python has no <code>undefined</code>
<code>string \ number</code>	<code>str \ int</code>	unions, same
<code>interface / type</code>	<code>Protocol</code> / <code>TypedDict</code> / <code>dataclass</code>	several tools, see below
generics <code><T></code>	<code>[T]</code> (3.12+) or <code>TypeVar</code>	same concept
<code>unknown</code>	<code>object</code>	safe top type
<code>any</code>	<code>Any</code>	the escape hatch; avoid
<code>as Foo</code>	<code>cast(Foo, x)</code>	unchecked assertion
<code>readonly</code>	<code>Final</code> / <code>tuple</code>	immutability hints
<code>keyof</code> , mapped types	(limited)	Python's system is less expressive here

The single most important thing to internalize: **types are checked in CI, not at runtime**. A wrong type doesn't raise on its own — mypy/pyright catches it before you ship. (If you *want* runtime enforcement of data shapes, that's Pydantic's job — Part 7 — not the type checker's.)

24.2 The syntax, side by side

TYPESCRIPT

```
function greet(name: string, times: number = 1): string {
    return (name + " ").repeat(times).trim();
}
const ids: number[] = [1, 2, 3];
const user: { id: number; name: string } = { id: 1, name: "Ada" };
```

PYTHON

```
def greet(name: str, times: int = 1) -> str:
    return " ".join([name] * times)

ids: list[int] = [1, 2, 3]
# for a fixed shape, use a dataclass or TypedDict rather than an inline object type
```

Built-in generics use the lowercase builtins (no imports since 3.9): `list[int]`, `dict[str, int]`, `tuple[int, str]`, `set[str]`. Callables and special forms come from `typing` or `collections.abc`:

PYTHON

```
from collections.abc import Callable, Iterable, Sequence

handler: Callable[[int, str], bool]          # (n: number, s: string) => boolean
def total(xs: Iterable[int]) -> int: ...      # accept anything iterable (like an
Iterable<T>)
```

NODE.JS MENTAL MODEL

Prefer the **abstract** collection types (`Iterable`, `Sequence`, `Mapping`) in function *parameters* and concrete ones (`list`, `dict`) in *return* types — the Python version of "accept the widest interface, return the narrowest." `Sequence[int]` accepts a list or a tuple; `list[int]` demands a list.

24.3 Optional and None — Python's one `null`

TypeScript juggles two empties (`null` and `undefined`); Python has exactly one: `None`. `Optional[X]` is just an alias for `X | None`, and the checker forces you to narrow it before use — the same flow-narrowing `tsc` does.

TYPESCRIPT

```
function find(id: number): User | undefined { ... }
const u = find(1);
u.name;           // Error: possibly undefined
if (u) u.name;    // ok — narrowed
```

PYTHON

```
def find(id: int) -> User | None: ...

u = find(1)
u.name      # mypy error: item "None" has no attribute "name"
if u is not None:
    u.name   # ok – narrowed
```

PRODUCTION TIP

Turn on **strict optional checking** (it's on by default in modern mypy and in pyright). The `X | None` discipline eliminates the Python equivalent of `Cannot read property 'x' of undefined` — the checker refuses to let you touch a possibly-`None` value. This one setting removes an entire class of production crashes.

24.4 Structural typing: `Protocol` = a TypeScript `interface`

This is the concept that will make you feel at home. TypeScript is **structurally** typed: anything with the right shape matches an `interface`, no `implements` needed. Python's `Protocol` is exactly that — duck typing, but statically checked.

TYPESCRIPT

```
interface Reader { read(): string; }
function consume(r: Reader) { r.read(); }
// any object with a matching read() is accepted – no `implements`
```

PYTHON

```
from typing import Protocol

class Reader(Protocol):
    def read(self) -> str: ...

def consume(r: Reader) -> None:
    r.read()

# any class with a matching read() method satisfies Reader – no inheritance needed
class FileReader:
    def read(self) -> str: return "..."

consume(FileReader())      # checks – structural match, just like TS
```

Contrast with **nominal** typing via base classes (`class X(Base)`), which is like `class X extends Base` — the name/lineage matters. Reach for `Protocol` when you want "any object shaped like this," and a base class (often an ABC — `abc.ABC`) when you want a named contract with shared implementation.

24.5 Typed dicts, dataclasses, and named shapes

For a fixed record shape you have three tools; pick by how the data behaves:

Tool	Best for	TS analog
<code>TypedDict</code>	a plain <code>dict</code> with known keys (e.g. JSON)	<code>interface Foo { ... }</code> on a <code>{}</code>
<code>@dataclass</code>	a real object with methods/defaults	a <code>class</code>
Pydantic <code>BaseModel</code>	data that must be validated at runtime	<code>zod / class-validator</code>

PYTHON

```
from typing import TypedDict
from dataclasses import dataclass

class UserJSON(TypedDict):           # a dict, but keys/types are checked
    id: int
    name: str

@dataclass
class User:                          # a class with __init__, __eq__, __repr__ for
    free
    id: int
    name: str
    active: bool = True
```

NODE.JS MENTAL MODEL

`TypedDict` describes a `dict` you already have (a parsed JSON body) without changing it at runtime — like typing a `Record` /object literal. `@dataclass` is the closest thing to a plain TS `class` with a generated constructor. When the data crosses a trust boundary (an HTTP body, a queue message), don't stop at `TypedDict` — validate it with Pydantic, because the type checker cannot guarantee what actually arrived at runtime.

24.6 Generics

Modern Python (3.12+) has clean generic syntax that reads almost like TypeScript:

TYPESCRIPT

```
function first<T>(xs: T[]): T | undefined { return xs[0]; }
class Box<T> { constructor(public value: T) {} }
```

PYTHON

```
def first[T](xs: list[T]) -> T | None:      # 3.12+ syntax
    return xs[0] if xs else None

class Box[T]:
```

```
def __init__(self, value: T) -> None:
    self.value = value
```

Pre-3.12 (still everywhere) uses an explicit `TypeVar` — worth recognizing:

PYTHON

```
from typing import TypeVar
T = TypeVar("T")
def first(xs: list[T]) -> T | None: ...
```

24.7 The escape hatches — and when they bite

PYTHON

```
from typing import Any, cast

data: Any = parse()           # like `any` – disables checking; spreads silently
user = cast(User, data)       # like `as User` – you assert, checker trusts you (no
                             # runtime check)
```

COMMON MISTAKE

Reaching for `Any` the moment the checker complains. `Any` is contagious in the same way `any` is in TypeScript: every value derived from it becomes unchecked, and the error just moves somewhere less obvious. Prefer `object` (the safe top type, like `unknown`) and narrow with `isinstance`, or write a `Protocol`. Reserve `cast` for cases where *you* genuinely know more than the checker (e.g. after validating), and remember it does **nothing** at runtime — it's a promise, not a conversion.

24.8 mypy vs pyright — which checker?

	mypy	pyright
From	Python core team	Microsoft (powers Pylance in VS Code)
Speed	fast	very fast (incremental)
Editor	via plugin	native in VS Code
Strictness	opt-in, granular	aggressive by default

Both are excellent; you can even run both. A pragmatic setup: **pyright in the editor** for instant feedback (you likely get it free with the Python VS Code extension), and **mypy in CI** as the gate. Configure in `pyproject.toml`:

TOML

```
[tool.mypy]
python_version = "3.13"
strict = true           # turn everything on (do this on new code)
```

```
warn_unused_ignores = true
warn_redundant_casts = true

# for pyright, a pyrightconfig.json or [tool.pyright] with typeCheckingMode =
"strict"
```

SHELL

```
uv run mypy src/           # the `tsc --noEmit` of your CI pipeline
uv run pyright             # or this
```

ADOPTING TYPES ON AN EXISTING CODEBASE

You don't have to boil the ocean. Type checking is **gradual** by design: unannotated code is simply skipped. Start with `strict = false`, type your most-used modules and all *new* code, and ratchet up per-module overrides over time — the exact `allowJs` /incremental-migration story you'd use moving a JS project to TypeScript.

WHEN NOT TO OBSESS OVER TYPES

Throwaway scripts, notebooks, and quick spikes don't need annotations — Python runs fine without them, and that's a feature. Save the strictness for library code, shared modules, and anything long-lived. The goal is fewer production bugs, not a green checker for its own sake.

EXERCISE 24.1

Add `mypy --strict` to a small module and fix every error without using `Any` or `cast`. Then write a Protocol named `Notifier` with a `send(self, msg: str) -> None` method, a couple of unrelated classes that satisfy it (no inheritance), and a function that accepts a `Notifier` — confirm the checker accepts all of them structurally, exactly like a TypeScript `interface`.

Part 25 · Configuration & Settings Management

In Node you probably reach for `process.env`, a `.env` file loaded by `dotenv`, and maybe a hand-rolled `config.js` that reads and coerces values. Python's modern answer is **pydantic-settings**: a typed, validated settings object that reads from environment variables and `.env` files, coerces types, and fails loudly at startup if anything is missing or malformed. It's `dotenv` + `zod` + your `config.js`, in one place.

25.1 The problem with raw `os.environ`

`os.environ` is Python's `process.env` — a plain string→string map. The footguns are identical to Node's:

PYTHON

```
import os
DEBUG = os.environ["DEBUG"]
# -> "false" ... a truthy string! bool("false") is True
PORT = os.environ.get("PORT")          # -> "8000" ... a string, not an int
TIMEOUT = os.environ.get("TIMEOUT")    # -> None if unset; crashes later, far from the
cause
```

Every value is a string, `"false"` is truthy, and a missing variable blows up deep in your code rather than at boot. You want to parse, validate, and type everything **once, at startup**.

25.2 pydantic-settings — typed config that validates itself

SHELL

```
uv add pydantic-settings
```

PYTHON

```
from pydantic import Field, PostgresDsn
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env", env_file_encoding="utf-8", extra="ignore"
    )

    debug: bool = False                                # "false"/"0"/"no" -> False, correctly
```

```

port: int = 8000 # coerced from string, validated as int
database_url: PostgresDsn # REQUIRED – startup fails if missing/
invalid
secret_key: str = Field(min_length=32) # validated, not just present
allowed_hosts: list[str] = ["localhost"] # "a,b,c" or JSON parsed into a list

settings = Settings() # reads env + .env, validates NOW

```

TYPESCRIPT

```

// the Node shape you're replacing
import "dotenv/config";
export const config = {
  debug: process.env.DEBUG === "true",
  port: Number(process.env.PORT ?? 8000),
  databaseUrl: process.env.DATABASE_URL!, // no validation; hope it's set
};

```

Field names map to env vars case-insensitively: `debug` reads `DEBUG`, `database_url` reads `DATABASE_URL`. Types are coerced and validated by Pydantic — so `PORT=abc` fails at boot with a precise message, not with a mysterious error on request #1.

NODE.JS MENTAL MODEL

BaseSettings is `dotenv` and `zod` fused: it *loads* the `.env` (like `dotenv/config`) **and** *validates* + *types* the result (like a `zod` schema over `process.env`). You import one `settings` object and get autocomplete and type-checking on every field — no more `process.env.FOO!` with a non-null assertion and a prayer.

25.3 Nested config and prefixes

Group related settings into sub-models, and namespace their env vars with a delimiter — the tidy version of `DB_HOST`, `DB_PORT`, `REDIS_URL` sprawl:

PYTHON

```

from pydantic import BaseModel
from pydantic_settings import BaseSettings, SettingsConfigDict

class DBSettings(BaseModel):
    host: str = "localhost"
    port: int = 5432
    pool_size: int = 10

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_nested_delimiter="__")
    db: DBSettings = DBSettings()

# env: DB__HOST=db.internal DB__PORT=5432 DB__POOL_SIZE=20

```

```
settings = Settings()
print(settings.db.host)
```

25.4 Per-environment settings (dev / staging / prod)

The 12-factor rule holds in both ecosystems: **config comes from the environment, not from code branches**. Keep one settings class and let the environment supply values; use distinct `.env` files per environment and never commit the real ones.

DIAGRAM

```
.env.example      ← committed: documents every variable, with dummy values
.env              ← local dev only, GIT-IGNORED
# staging/prod: real env vars injected by the platform (Docker, k8s, systemd)
```

PYTHON

```
import os
from functools import lru_cache

@lru_cache          # build once, reuse (a tiny singleton)
def get_settings() -> Settings:
    env_file = f".env.{os.environ.get('APP_ENV', 'dev')}"
    return Settings(_env_file=env_file)    # override which file to load
```

In FastAPI this pairs perfectly with dependency injection (Part 7) — inject `get_settings` so tests can override it:

PYTHON

```
from fastapi import Depends

@app.get("/health")
def health(settings: Settings = Depends(get_settings)):
    return {"env": settings.debug}
```

PRODUCTION TIP

In containers and orchestrators, don't ship a `.env` at all — inject real environment variables (Docker `-e /compose environment:`, k8s `env /secrets`, systemd `Environment=`). `pydantic-settings` reads those the same way it reads a file, so the *same* `Settings` class works everywhere with zero code changes — the essence of 12-factor config.

25.5 Secrets

Never put real secrets in a committed file. Beyond plain env vars, `pydantic-settings` can read Docker/k8s **secrets files** (each secret is a file whose name is the key), and integrates with cloud secret managers via a custom source.

PYTHON

```

class Settings(BaseSettings):
    model_config = SettingsConfigDict(secrets_dir="/run/secrets") # Docker/k8s
    secrets
    stripe_key: str # read from /run/secrets/stripe_key if present, else
    env

# use SecretStr so the value never prints in logs/tracebacks:
from pydantic import SecretStr
class S(BaseSettings):
    api_token: SecretStr # repr shows '*****'; .get_secret_value() to read

```

COMMON MISTAKE

Three Node-developer traps: (1) committing `.env` — gitignore it and commit only `.env.example`; (2) reading `os.environ` scattered across modules instead of one validated `Settings` — you lose the single source of truth and fail late instead of at boot; (3) logging the whole settings object and leaking secrets — wrap sensitive fields in `SecretStr` so they're masked in logs and tracebacks (see Part 26).

WHEN NOT TO ADD PYDANTIC-SETTINGS

A ten-line script that reads one `API_KEY` doesn't need a settings class — `os.environ["API_KEY"]` is fine. Reach for `pydantic-settings` once you have more than a couple of variables, multiple environments, or values that need coercion/validation — i.e. any real service.

EXERCISE 25.1

Build a `Settings` class with a required `database_url`, a `debug: bool` defaulting to `False`, a `port: int` defaulting to `8000`, and a nested `redis` sub-model. Confirm that (a) an unset `database_url` fails at startup with a clear message, (b) `DEBUG=0` and `DEBUG=false` both parse to `False` (the bug you'd hit with raw `process.env`), and (c) `PORT=notaport` fails loudly at boot rather than on first request. Then wrap a `SecretStr` field and verify it prints masked.

Part 26 · Logging & Error Tracking

In Node you likely use `pino` or `winston` and ship errors to Sentry. Python has a **batteries-included** `logging` module in the standard library (no dependency needed), plus `structlog` when you want ergonomic structured/JSON logs, and the same **Sentry** SDK you already know. The big adjustment: stop using `print()`. `print()` is `console.log` — fine for a script, wrong for a service.

26.1 The standard `logging` module

PYTHON

```
import logging

logger = logging.getLogger(__name__)    # one logger per module (use __name__)

logger.debug("verbose detail: %s", value)  # lazy %-formatting — only formats if
emitted
logger.info("user signed up", extra={"user_id": 42})
logger.warning("cache miss")
logger.error("payment failed", exc_info=True)  # include the traceback
logger.critical("out of disk")
```

Node (pino/winston)	Python <code>logging</code>	Notes
<code>logger.info()</code>	<code>logger.info()</code>	same 5 levels: debug/info/warn(ing)/error/critical
<code>console.log</code>	<code>print()</code>	avoid <code>print()</code> in services
<code>logger.child({req})</code>	<code>LoggerAdapter</code> / <code>structlog</code> bind	attach context
transports	handlers	where logs go (stdout, file, HTTP)
formatters	formatters	how logs look (text, JSON)
levels config	<code>logging.config.dictConfig</code>	central config

NODE.JS MENTAL MODEL

Python's model is **logger** → **handler** → **formatter**: a *logger* (named per module) creates records, a *handler* decides where they go (stdout, a file, the network — pino's "transports"), and a *formatter* decides how they look (plain text or JSON). You configure this once at startup, not per call.

26.2 Configure once, at startup

Configure logging in one place (your app entry point), the way you'd set up pino once. `dictConfig` is the declarative, production-friendly form:

PYTHON

```
import logging.config

logging.config.dictConfig({
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {
        "json": {"format": '{"level":"%(levelname)s","logger":"%(name)s","msg":"%(message)s"}'},
        "plain": {"format": "%(asctime)s %(levelname)-8s %(name)s: %(message)s"},
    },
    "handlers": {
        "console": {"class": "logging.StreamHandler", "formatter": "plain"},
    },
    "root": {"level": "INFO", "handlers": ["console"]},
})
```

PRODUCTION TIP

Log to **stdout**, not to files. In containers (Part 20) the platform captures stdout and ships it to your aggregator — the 12-factor rule, same as Node. Set the level from config (`LOG_LEVEL` , via Part 25's settings) so you can flip to `DEBUG` in one place without a redeploy. Never hand-roll log rotation inside the app; let the platform handle it.

26.3 Structured logging with structlog

Plain text is unsearchable at scale. You want **JSON logs with key/value context** — the reason pino won in Node. `structlog` is the ergonomic way to get that in Python:

SHELL

```
uv add structlog
```

PYTHON

```
import structlog

structlog.configure(
    processors=[
        structlog.contextvars.merge_contextvars,  # pulls in bound request context
        structlog.processors.add_log_level,
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.JSONRenderer(),      # emit JSON (swap for dev
console renderer)
    ],
```

```
)
log = structlog.get_logger()

log.info("order_created", order_id=123, total=49.99, currency="USD")
# -> {"event":"order_created","order_id":123,"total":49.99,"currency":"USD",
#     "level":"info","timestamp":"2026-..."}

```

TYPESCRIPT

```
// the pino you're replacing
logger.info({ orderId: 123, total: 49.99, currency: "USD" }, "order_created");

```

26.4 Request context (correlation IDs)

The killer feature: bind a request/correlation ID **once** and have it appear on every log line for that request — pino's `logger.child()`, done via `contextvars` (Part 14) so it works correctly even across `await` boundaries.

PYTHON

```
import structlog, uuid

async def logging_middleware(request, call_next):
    structlog.contextvars.clear_contextvars()
    structlog.contextvars.bind_contextvars(
        request_id=str(uuid.uuid4()),
        path=request.url.path,
    )
    return await call_next(request)

# anywhere deeper in the request — no need to pass the id around:
log.info("db_query", table="users") # automatically includes request_id + path

```

NODE.JS MENTAL MODEL

`bind_contextvars` is `logger.child({ requestId })`, and `contextvars` is the equivalent of `AsyncLocalStorage` — the context follows the logical task through every `await`, so a log deep in your service carries the request ID without threading it through every function signature.

26.5 Error tracking with Sentry

Same Sentry you use in Node, same concepts. The SDK auto-instruments Django, FastAPI, Celery, and more — you initialize it once and unhandled exceptions ship automatically with tracebacks and request context.

SHELL

```
uv add "sentry-sdk[fastapi]"

```

PYTHON

```
import sentry_sdk

sentry_sdk.init(
    dsn=settings.sentry_dsn,          # from Part 25 settings
    environment=settings.app_env,     # dev/staging/prod
    traces_sample_rate=0.1,           # perf tracing on 10% of requests
    send_default_pii=False,           # don't ship user PII by default
)

# unhandled exceptions are captured automatically; capture handled ones explicitly:
try:
    charge(order)
except PaymentError:
    sentry_sdk.capture_exception()    # like Sentry.captureException(e) in Node
    raise
```

COMMON MISTAKE

Node-developer traps when moving to Python logging: (1) using `print()` in a service — it can't be levelled, filtered, or structured, and bypasses your handlers; (2) calling `logging.info(...)` on the **root** logger instead of a module logger (`logging.getLogger(__name__)`), which loses the source and makes filtering impossible; (3) eager string formatting — `logger.debug(f"x={expensive()}")` runs `expensive()` even when `DEBUG` is off, whereas `logger.debug("x=%s", expensive())` still evaluates the arg, so guard truly costly work with `logger.isEnabledFor(logging.DEBUG)`; (4) logging secrets — mask them (Part 25's `SecretStr`) before they reach a log line.

WHEN NOT TO ADD STRUCTLOG / SENTRY

A CLI tool or one-off script is fine with the stdlib `logging` module and no error tracker — structured JSON and Sentry earn their keep in long-running services with real traffic. Don't add the moving parts until you have something worth observing.

EXERCISE 26.1

Configure `structlog` to emit JSON in production and a colored console renderer in development (branch on Part 25's `debug` setting). Add middleware that binds a `request_id` per request, then log from two different modules within one request and confirm both lines carry the same `request_id`. Force an unhandled exception and verify it reaches Sentry (use a test DSN) with the request context attached.

Part 27 · Python + LLMs

Python is the default language of the AI ecosystem, so this is one area where you're moving *toward* the center of gravity, not away from it. The provider SDKs (OpenAI, Anthropic) mirror their Node counterparts almost method-for-method, so the API surface will feel immediately familiar. What Python adds is the thing you already love from TypeScript — **typed, validated structured output** — via Pydantic, plus first-class async that fits FastAPI (Part 7) perfectly.

This chapter is about *calling* LLMs from a backend. Retrieval-augmented generation and model training have their own cookbooks; here we cover the application layer: API calls, structured output, tool calling, local models, and the production patterns (retries, cost, testing) that keep an LLM feature from becoming an outage or a surprise bill.

MODEL IDS CHANGE CONSTANTLY

Every code sample uses a placeholder like `MODEL = "claude-sonnet-4-5"` or `"gpt-4o"`.

Model IDs are versioned and rotate often — always check the provider's current model list (their docs) for the live identifier rather than trusting a string printed in a book. The *API shape* below is stable; the model *name* is not.

27.1 The SDKs, mapped to Node

Node	Python	Notes
<code>openai</code> (npm)	<code>openai</code> (PyPI)	same methods, snake_case
<code>@anthropic-ai/sdk</code>	<code>anthropic</code>	same
<code>await client.chat.completions.create</code>	<code>client.chat.completions.create</code>	sync or <code>await</code> (async client)
<code>for await (chunk of stream)</code>	<code>for event in stream: / async for</code>	streaming
<code>zodResponseFormat</code>	Pydantic model + <code>.parse()</code>	structured output
<code>AsyncOpenAI</code> (default)	<code>OpenAI</code> (sync) or <code>AsyncOpenAI</code>	Python gives you both explicitly

SHELL

```
uv add openai anthropic
```

27.2 A basic call — sync and async

TYPESCRIPT

```
// Node
import OpenAI from "openai";
const client = new OpenAI();
const res = await client.chat.completions.create({
  model: "gpt-4o",
  messages: [{ role: "user", content: "Explain a Python venv in one sentence." }],
});
console.log(res.choices[0].message.content);
```

PYTHON

```
# Python – sync (fine for scripts, CLIs, Celery tasks)
from openai import OpenAI
client = OpenAI() # reads OPENAI_API_KEY from env
res = client.chat.completions.create(
  model="gpt-4o",
  messages=[{"role": "user", "content": "Explain a Python venv in one sentence."}],
)
print(res.choices[0].message.content)
```

PYTHON

```
# Python – async (use this inside FastAPI so you don't block the event loop, Part 14)
from openai import AsyncOpenAI
client = AsyncOpenAI()

async def ask(q: str) -> str:
  res = await client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": q}],
  )
  return res.choices[0].message.content
```

The Anthropic SDK is the same idea; note `max_tokens` is **required** and the system prompt is a top-level parameter, not a message:

PYTHON

```
from anthropic import Anthropic
client = Anthropic()
msg = client.messages.create(
  model="claude-sonnet-4-5",
  max_tokens=1024,
  system="You are concise.",
  messages=[{"role": "user", "content": "Explain a Python venv in one sentence."}],
)
print(msg.content[0].text)
```

COMMON MISTAKE

Using the **sync** client inside an `async def` FastAPI route. It blocks the event loop for the *entire* multi-second model call, freezing every other request on that worker (Part 14). Inside async code, use `AsyncOpenAI` / `AsyncAnthropic` and `await`. In a sync context (a script, a Celery worker), the sync client is correct and simpler.

27.3 Structured output — Pydantic is your Zod

This is the pattern that makes LLMs *usable* in a backend: force the model to return data that matches a schema, validated, so you get a typed object instead of a string you have to `JSON.parse` and pray over. OpenAI parses straight into a Pydantic model:

PYTHON

```
from pydantic import BaseModel
from openai import OpenAI

class Invoice(BaseModel):
    vendor: str
    total: float
    currency: str
    line_items: list[str]

client = OpenAI()
res = client.chat.completions.parse(
    model="gpt-4o",
    messages=[{"role": "user", "content": f"Extract invoice fields:\n{raw_text}" }],
    response_format=Invoice,
    schema
)
invoice: Invoice = res.choices[0].message.parsed
print(invoice.total, invoice.currency)
```

note: .parse, not .create
<- the Pydantic model IS the schema
a validated Invoice, fully typed

TYPESCRIPT

```
// the Node/Zod equivalent you already know
const res = await client.chat.completions.parse({
  model: "gpt-4o",
  messages: [...],
  response_format: zodResponseFormat(InvoiceSchema, "invoice"),
});
```

For a provider-agnostic version (works with Anthropic, OpenAI, local models, etc.), the **instructor** library patches any client to accept a `response_model`:

PYTHON

```
import instructor
from anthropic import Anthropic
```

```

client = instructor.from_anthropic(Anthropic())
invoice = client.messages.create(
    model="claude-sonnet-4-5", max_tokens=1024,
    response_model=Invoice,                                # same Pydantic model, any
    provider
    messages=[{"role": "user", "content": raw_text}],
)

```

NODE.JS MENTAL MODEL

Pydantic here is doing exactly what Zod does with `zodResponseFormat` : the schema is sent to the model to constrain generation, and the response is validated against it on the way back. The difference is that in Python this same `Invoice` model is *also* your FastAPI request/response model, your ORM boundary, and your type — one definition, reused everywhere (Part 7).

27.4 Streaming

Stream tokens as they arrive — for a chat UI or Server-Sent Events from FastAPI. It's the `await (const chunk of stream)` you know:

PYTHON

```

# OpenAI, async streaming
stream = await client.chat.completions.create(
    model="gpt-4o", stream=True,
    messages=[{"role": "user", "content": "Write a haiku about the GIL."}],
)
async for chunk in stream:
    delta = chunk.choices[0].delta.content
    if delta:
        print(delta, end="", flush=True)

```

PYTHON

```

# Anthropic gives a typed streaming context manager
with client.messages.stream(model="claude-sonnet-4-5", max_tokens=512,
    messages=[{"role": "user", "content": "..."}]) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)

```

Wiring it to FastAPI as SSE is a few lines (Part 7):

PYTHON

```

from fastapi.responses import StreamingResponse

@app.post("/chat")
async def chat(body: ChatIn):
    async def gen():

```

```

stream = await client.chat.completions.create(
    model="gpt-4o", stream=True,
    messages=[{"role": "user", "content": body.message}])
async for chunk in stream:
    if (d := chunk.choices[0].delta.content):
        yield f"data: {d}\n\n"
return StreamingResponse(gen(), media_type="text/event-stream")

```

27.5 Tool calling and a minimal agent loop

Tool calling (a.k.a. function calling) lets the model ask *you* to run a function and feed the result back — the mechanism behind assistants and agents. The shape is identical to Node's: you describe tools, the model may return tool-call requests, you execute them and return results, and loop until the model produces a final answer.

PYTHON

```

tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current weather for a city.",
        "parameters": {
            # JSON Schema (a
Pydantic .model_json_schema() works)
            "type": "object",
            "properties": {"city": {"type": "string"}},
            "required": ["city"],
        },
    },
}]

def get_weather(city: str) -> str:
    return f"{city}: 22°C, clear" # your real implementation

def run_agent(question: str) -> str:
    messages = [{"role": "user", "content": question}]
    while True:
        res = client.chat.completions.create(
            model="gpt-4o", messages=messages, tools=tools)
        msg = res.choices[0].message
        if not msg.tool_calls: # no tool needed -> final
            answer = msg.content
            return msg.content
        messages.append(msg) # record the model's tool
        request = msg
        for call in msg.tool_calls:
            args = json.loads(call.function.arguments)
            result = get_weather(**args) # dispatch to the real
            function = result
            messages.append({
                "role": "tool",

```

```

        "tool_call_id": call.id,
        "content": result,
    })
    # loop: the model now sees the tool result and continues

```

NODE.JS MENTAL MODEL

This is the exact loop you'd write with the Node SDK — describe tools, check `tool_calls`, execute, append `role: "tool"` results, repeat. Nothing about it is Python-specific except the `snake_case`. For anything beyond a few tools, reach for a framework (LangChain, LlamaIndex, Pydantic AI, or the providers' own agent SDKs) rather than hand-maintaining the loop — the same call you'd make in Node once orchestration gets real.

PRODUCTION TIP

Always **cap the loop** (a max-iterations counter) so a misbehaving model can't spin forever running tools — each iteration is a paid API call. Treat every tool the model can invoke as untrusted input: validate arguments (Pydantic!), authorize the action, and never `eval` /shell-exec what the model hands you.

27.6 Local models — Ollama & Hugging Face

You don't always want a paid API. Two common local paths:

Ollama runs models locally and — conveniently — exposes an **OpenAI-compatible** endpoint, so you reuse the same SDK by pointing `base_url` at it. Zero new client to learn:

PYTHON

```

from openai import OpenAI
local = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama") # any key
string
res = local.chat.completions.create(
    model="llama3.1",
    messages=[{"role": "user", "content": "Summarize this changelog:\n" + text}],
)

```

Hugging Face `transformers` runs a model in-process — full control, no server, but you manage the weights and (ideally) a GPU:

PYTHON

```

from transformers import pipeline
pipe = pipeline("text-generation", model="meta-llama/Llama-3.1-8B-Instruct")
out = pipe("Explain async in one line:", max_new_tokens=60)
print(out[0]["generated_text"])

```

WHEN NOT TO RUN LOCAL MODELS

Local models trade a bill for operational weight: GPU provisioning, memory, model downloads, and slower cold starts. For most backend features a hosted API is cheaper *all-in* until you hit real scale, strict data-residency rules, or offline requirements. Prototype against a hosted API; move local when a concrete constraint forces it — not by default.

27.7 Production patterns: retries, rate limits, cost, and caching

LLM calls are slow, flaky, rate-limited, and metered. Treat them like the unreliable network calls they are (Part 16).

Retries & timeouts are built into the SDKs — configure them rather than hand-rolling:

PYTHON

```
from openai import OpenAI
client = OpenAI(max_retries=4, timeout=30.0)  # exponential backoff on 429/5xx,
built in
# per-call override:
client.with_options(timeout=10.0).chat.completions.create(...)
```

Token usage and cost come back on every response — log it (Part 26) and you can attribute spend per feature or per user:

PYTHON

```
res = client.chat.completions.create(model="gpt-4o", messages=msgs)
u = res.usage
log.info("llm_call", prompt_tokens=u.prompt_tokens,
        completion_tokens=u.completion_tokens, total=u.total_tokens)
```

Caching: identical prompts should not be paid for twice. Cache on a hash of (model + messages) in Redis (Part 15); and use provider **prompt caching** for large, reused system prompts/context to cut both latency and cost.

PYTHON

```
import hashlib, json
def cache_key(model: str, messages: list) -> str:
    blob = json.dumps({"m": model, "msgs": messages}, sort_keys=True)
    return "llm:" + hashlib.sha256(blob.encode()).hexdigest()
# check Redis before calling; store the completion after.
```

COMMON MISTAKE

Shipping an LLM feature with **no timeout, no retry cap, no cost logging, and no cache**. The failure modes are all avoidable: a hung request holds a worker for minutes; an un-capped retry storm amplifies a provider outage; an unlogged feature produces a bill nobody can explain; and re-answering identical prompts burns money for nothing. Wire all four in from day one.

27.8 Testing LLM code without burning tokens

Never hit the real API in your test suite — it's slow, costs money, and is non-deterministic. **Mock the client** (Part 18) and assert on *your* logic (the prompt you built, the tool dispatch, the parsing), not the model's creativity.

PYTHON

```
def test_agent_dispatches_tool(mocker):
    fake = mocker.patch("myapp.llm.client")
    # first response asks for a tool; second returns the final answer
    fake.chat.completions.create.side_effect = [tool_call_response(),
    final_response()]
    answer = run_agent("weather in Cairo?")
    assert answer == "It's 22°C in Cairo."
    assert fake.chat.completions.create.call_count == 2
```

PRODUCTION TIP

Keep a tiny, opt-in **integration test** (skipped in normal CI, run manually or nightly with a real key and a strict token budget) that hits the live API to catch provider/schema drift — the model equivalent of a smoke test. Everything else mocks.

EXERCISE 27.1

Build a FastAPI endpoint that takes raw text and returns a **validated** Invoice (Section 27.3) using structured output. Add: an async client so the route doesn't block (27.2), a Redis cache keyed on the input (27.7), token/cost logging (27.6), and a `max_retries` + timeout config. Then write tests that (a) mock the client and assert the parsing, and (b) confirm a second identical request is served from cache without a second API call.

Part 28 · Master Cheat Sheet & Closing Roadmap

Bookmark this part. It compresses the whole book into lookup tables: **Node.js** → **Python** → **Django** → **FastAPI** across every subsystem, plus the closing roadmap.

24.1 Tooling & ecosystem

Node.js	Python
npm / pnpm	uv (recommended) / Poetry / pip
package.json	pyproject.toml
package-lock.json	uv.lock / poetry.lock
node_modules/	the virtualenv (.venv)
npm ci	uv sync --frozen
npx tool	uvx tool / uv run tool
ESLint / Prettier	ruff (lint + format)
tsc (type check)	mypy / pyright
node REPL	python / manage.py shell
Jest / Vitest	pytest
Supertest	APIClient (DRF) / TestClient (FastAPI)
nock / msw	respx / responses
PM2	Gunicorn/Uvicorn + systemd/Supervisor

24.2 Language

JavaScript / TypeScript	Python
const x = 1	x = 1
`Hi \${name}`	f"Hi {name}"
{ a: 1 }	{"a": 1} (dict)
[1,2,3]	[1, 2, 3] (list)
new Set() / new Map()	set() / dict
x === y	x == y (value) / x is y (identity)
x ?? y	x if x is not None else y
x?.y	getattr(x, "y", None)

JavaScript / TypeScript	Python
<code>arr.map(f)</code>	<code>[f(x) for x in arr]</code>
<code>arr.filter(p)</code>	<code>[x for x in arr if p(x)]</code>
<code>arr.reduce(...)</code>	<code>sum()</code> / <code>functools.reduce</code>
<code>switch</code>	<code>match/case</code> (structural)
<code>(x) => x*2</code>	<code>lambda x: x*2</code>
<code>...rest</code> / <code>...spread</code>	<code>*args</code> / <code>*seq</code> (and <code>**kwargs</code> / <code>**dict</code>)
<code>class</code> / <code>interface</code>	<code>class</code> / <code>Protocol</code> / <code>ABC</code>
<code>readonly data</code>	<code>@dataclass(frozen=True)</code>
decorators	decorators (<code>@</code>)
<code>try/catch(e)</code>	<code>try/except TypeError as e</code>
<code>function*</code> / <code>yield</code>	<code>def gen(): yield</code>
<code>for await...of</code>	<code>async for ... in</code>
<code>import/export</code>	<code>import</code> / <code>from x import y</code>

24.3 Concurrency & runtime

Node.js	Python
event loop (libuv)	asyncio event loop
<code>async function</code> / <code>await</code>	<code>async def</code> / <code>await</code>
a <code>Promise</code>	a coroutine / <code>Task</code> / <code>Future</code>
<code>Promise.all</code>	<code>asyncio.gather</code> / <code>TaskGroup</code>
<code>Promise.allSettled</code>	<code>gather(return_exceptions=True)</code>
<code>Promise.race</code>	<code>wait(..., FIRST_COMPLETED)</code>
<code>setTimeout</code>	<code>asyncio.sleep</code> / <code>call_later</code>
<code>AbortController</code>	<code>Task.cancel()</code> / <code>asyncio.timeout</code>
Worker Threads (CPU)	<code>ProcessPoolExecutor</code> / Celery (GIL!)
<code>child_process</code>	<code>subprocess</code> / <code>asyncio subprocess</code>
single-threaded by design	GIL: 1 thread runs bytecode; use processes for CPU
one runtime model	three: asyncio, threads, processes

24.4 Web frameworks & HTTP

Express / Node	Django	DRF	FastAPI
<code>express.Router</code>	<code>urlpatterns</code> / <code>include</code>	<code>DefaultRouter</code>	<code>APIRouter</code>

Express / Node	Django	DRF	FastAPI
<code>router.get('/x/:id')</code>	<code>path('x/<int:id>')</code>	ViewSet	<code>@app.get('/x/{id}')</code>
handler <code>(req, res)</code>	view function/class	ViewSet method	<code>async def</code> endpoint
middleware	middleware class	—	<code>@app.middleware</code> / <code>deps</code>
Zod / Joi	Forms	serializer	Pydantic
DTO / serializer	serializer	serializer	Pydantic <code>*Out</code>
DI (Nest)	—	—	<code>Depends</code>
CRUD routes	—	<code>ModelViewSet</code> + router	explicit endpoints
Swagger (manual)	—	browsable API	auto OpenAPI <code>/docs</code>
runtime	WSGI (sync)	WSGI (sync)	ASGI (async)

24.5 Data, jobs, infrastructure

Node.js	Django	FastAPI
Prisma	Django ORM	SQLAlchemy
<code>findMany({where})</code>	<code>.objects.filter(...)</code>	<code>select().where(...)</code>
<code>include</code> (eager)	<code>select_related</code> / <code>prefetch_related</code>	<code>joinedload</code> / <code>selectinload</code>
migrate	<code>makemigrations</code> + <code>migrate</code>	Alembic
<code>\$transaction</code>	<code>transaction.atomic()</code>	<code>session.begin()</code>
<code>FOR UPDATE</code>	<code>select_for_update()</code>	<code>with_for_update()</code>
Axios / fetch	requests / httpx	httpx (async)
<code>Promise.all</code> HTTP	threads	<code>asyncio.gather</code> + <code>httpx</code>
BullMQ	Celery (+ Beat)	Celery / arq / Dramatiq
Bull Board	Flower	Flower
node-cron	Celery Beat	Celery Beat
Redis (ioredis)	<code>django.core.cache</code> / <code>redis-py</code>	<code>redis.asyncio</code>
in-proc cache Map	external Redis (no in-proc!)	external Redis
S3 SDK	boto3 / <code>django-storages</code>	boto3 / <code>aioboto3</code>
<code>getSignedUrl</code>	<code>generate_presigned_url</code>	<code>generate_presigned_url</code>
Socket.IO	Channels (ASGI)	WebSockets (ASGI) / Channels
bcrypt	Django hashers	passlib / argon2
jsonwebtoken	simplejwt	pyjwt / python-jose
pino / winston	structlog	structlog
Sentry	sentry-sdk	sentry-sdk
Prometheus client	django-prometheus	prometheus-client

24.6 The runtime split, in one table

Concept	Node	Django (WSGI)	FastAPI (ASGI)
Model	one event loop	many sync worker procs	event loop per worker
Concurrency	non-blocking	add processes	non-blocking + workers
Blocking code	offload	fine per worker	must not block loop
CPU parallelism	Workers/procs	procs / Celery	procs / Celery (GIL)
Server	node/PM2	Gunicorn	Uvicorn (+Gunicorn)
Async DB/HTTP	native	optional	required for benefit
Worker code refresh	restart	restart	restart
Celery worker refresh	—	restart	restart

24.7 Closing Roadmap — from here to fluent

Weeks 1-2 · Language & FP (Parts 1-2). Port 20+ Node utilities to Python; rewrite a transform module with comprehensions + `itertools`. Turn on `mypy` / `ruff`. *Checkpoint:* you write comprehensions by reflex, not `map().filter()`.

Week 3 · Runtime (Part 3). Benchmark I/O-bound vs CPU-bound across async, threads, processes; explain the GIL and WSGI-vs-ASGI to a colleague. *Checkpoint:* you always know which concurrency tool fits.

Weeks 4-5 · Frameworks & data (Parts 4-8). Scaffold a Django app *and* a FastAPI app; build Users/Projects/Tasks CRUD in both ORMs with zero N+1s. *Checkpoint:* you build a resource in either stack without looking up steps.

Week 6 · APIs, auth, security (Parts 9-11). Pagination/filtering, JWT, permissions, rate limiting, the security + SSRF checklist. *Checkpoint:* your API is safe to expose publicly.

Weeks 7 · Async & jobs (Parts 13-14, 16). Move slow work to Celery; write correct async endpoints with async DB + `httpx`; know when async is overkill. *Checkpoint:* nothing slow or blocking runs on the request path.

Week 8 · Testing & production (Parts 18, 20, 22). `pytest` with fixtures and factories; containerize web + worker + beat; add structured logs, health checks, and Sentry. *Checkpoint:* you can ship and operate with confidence.

Weeks 9-12 · Capstones (Part 23). Build all three in FastAPI+SQLAlchemy; port one to Django/DRF. *Checkpoint:* you are an independent, production-grade Python backend developer in both ecosystems.

Then keep going: read framework internals when docs run out, adopt async and tracing where they pay off, deepen domain modeling (Part 12). You started fluent in Node; you now hold the thread connecting Django and FastAPI back to everything you already knew. Welcome to Python.